基于分位数风险裕度与情景滚动优化的微网购电与储能协同调度

摘　要

针对负荷、光伏与电价不确定条件下的微网经济调度问题，本文以 10 分钟为步长，建立“预测—购电—储能—安全执行”的统一滚动优化框架，并以真实供需和真实结算价格回放评价策略。四问逐层引入确定性信息、供需误差、日内光伏预报和波动电价，核心区别在于**决策时可用的信息**以及**购电合同能否调整**。

针对问题一，在典型日负荷、光伏和电价完全已知的条件下建立**确定性线性规划**。最优策略全天购电 **5.95 万 kWh**、购电费 **3.51 万元**，较无储能基准节省 26.90%，且无弃光。电池在低价和光伏富余时段充电、在高价时段放电，并于 24 时恢复至 6000kWh。

针对问题二，采用前一日、前一周和近七日中位数构造**滚动预测**，并由普通购电与 5 倍紧急购电的边际成本推出 **80% 临界分位数**，形成日前安全裕度；日内按“有余则充、有缺先放、仍缺再补”执行安全反馈。为检验该策略，本文另以决策前已完整揭晓的 12 段历史误差序列构造 12 条负载——光伏联合情景，在每日 0 时以情景平均紧急购电费用最小为目标制定购电计划，并在日内按 4 小时窗口滚动调整储能，记为“**12 情境滚动模型**”。2025 年 2 月 1 日至 12 月 31 日共 334 天，主策略总费用 **1463.26 万元**，其中计划费用 **1413.45 万元**、紧急费用 49.81 万元；与 12 情境滚动模型相比，费用下降 1.30%，紧急购电量下降 67.71%。这说明在 5 倍紧急价格下，直接用 80% 分位数形成合同安全裕度，比 12 条等权历史情景更能覆盖高净负荷尾部。

针对问题三，采用 **12 情境随机规划与 4 小时滚动控制**，在 6、12、18 时根据最新光伏预报调整尚未执行的购电合同。全部更新方案全年费用 **1466.21 万元**，较仅 0 时预报**节省 39.57 万元**、2.63%，紧急购电量下降 25.99%。结果表明，日内预报的价值不仅在于降低计划费用，也在于及时修正高价缺口；但调整并非越多越好，需按完整结算费用评价。

针对问题四，考虑到制定购电计划时无法提前获得未来真实电价，本文以过去 28 天中相同星期的电价均值、最近 7 天电价均值以及附件 1 参考电价加权得到**基础电价预测**；再用决策时刻以前的**历史对数价格误差**修正基础值，并按与负荷、光伏相同的历史误差块构造 12 条价格情景，使每条情景同时保留负荷、光伏和价格的共同变化。允许日内调整时，费用由 **1558.41 万元降至 1538.52 万元**，紧急购电量下降 27.40%。

逐段核验表明，全部策略满足能量守恒、储电连续、功率容量、信息因果和费用规则；四种全年主策略相对各自无储能基准均节费 21.03%—22.74%。

关键词：储能调度；线性规划；分位数风险；情景随机规划；滚动控制

一、问题重述

本题研究微网中光伏、外部电网购电与储能设备的协同调度问题，在满足负载需求及储能设备运行约束的条件下，以降低购电成本为目标。

问题一在已知负载、光伏及电价的条件下，制定单日最优购电与储能方案；问题二考虑负载和光伏的不确定性，利用历史数据预测并制定日前购电计划，同时通过储能和紧急购电进行日内调节；问题三进一步引入 6 时、12 时和 18 时发布的光伏预测，对未执行的购电计划进行动态调整；问题四在此基础上考虑实时电价波动，对不同购电调整策略进行优化与比较。

二、问题分析

四个问题均以微网能量平衡和储能状态变化为基础，其难点在于储能使各时段决策相互关联，同时负荷、光伏和电价的不确定性会影响购电策略。

1. 问题一：数据完全已知，可建立确定性优化模型，在满足储能容量、功率及日初日末电量一致等约束下，使全天购电费用最小。
2. 问题二：未来负荷和光伏存在预测误差，需要在计划购电费用与高价紧急购电风险之间进行权衡，并结合日内储能实时调节。
3. 问题三：利用日内多次更新的光伏预测，对尚未执行的购电计划进行滚动调整，并综合考虑调整成本与紧急购电成本。
4. 问题四：进一步加入电价预测，在供需和价格均不确定的情况下联合优化购电与储能，并比较允许和不允许日内调整两种策略的经济性。

三、模型假设、符号与数据处理

3.1 模型假设

1. 每个 10 分钟区间内以平均功率代表负载和光伏。运行控制能够监测当前供需，并在该区间内完成补缺和充放电；这一近似不描述秒级波动和响应延迟。
2. 外网能够提供题目允许的计划电量和紧急电量，负载不能削减。题目没有给出的线路容量、购电爬坡和固定交易费用不另加限制。
3. 无向外网售电收入，无法利用的余电可以免费处置，自放电和退化费用不计入。
4. 电池只用当时的剩余供电充电，紧急购电仅补足负载缺口。购电、充电和放电量均在电池外部计量，储电量表示电池内部的能量。

3.2 时间离散与数据口径

将自然日划分为 $T = 144$ 个区间，每段长度 $\Delta = 1/6$ 小时。功率与区间电量分别描述瞬时变化率和区间累计量，例如 600 kW 持续 10 分钟对应 100 kWh。因此，将附件中的负载功率 L_t 与光伏功率 V_t 转换为

$$\ell_t = \Delta L_t, \qquad v_t = \Delta V_t, \qquad n_t = \ell_t - v_t. \tag{1}$$

其中 n_t 为净负荷，即扣除光伏后的外部供电需求；$n_t < 0$ 表示存在可用于充电的光伏余量，故不作非负截断。

附件输入时刻为 00:10 至 24:00，本文将其解释为对应 10 分钟区间的结束时刻。原结果模板的部分购电时段标记为 00:10–00:20 至次日 00:00–00:10，与储能部分采用的 00:00–24:00 自然日口径不一致。本文统一采用当日 00:00–24:00 的 144 个区间，在保留原工作表结构的同时按映射表填入结果，用于消除原材料的时间标记歧义。

图1按全年日期和日内时刻展示负载、光伏发电与实时电价的分布，其中颜色表示相应功率或电价水平。该图直观反映了供需和价格在日内及全年尺度上的变化，为后续预测、购电和储能调度提供数据背景。

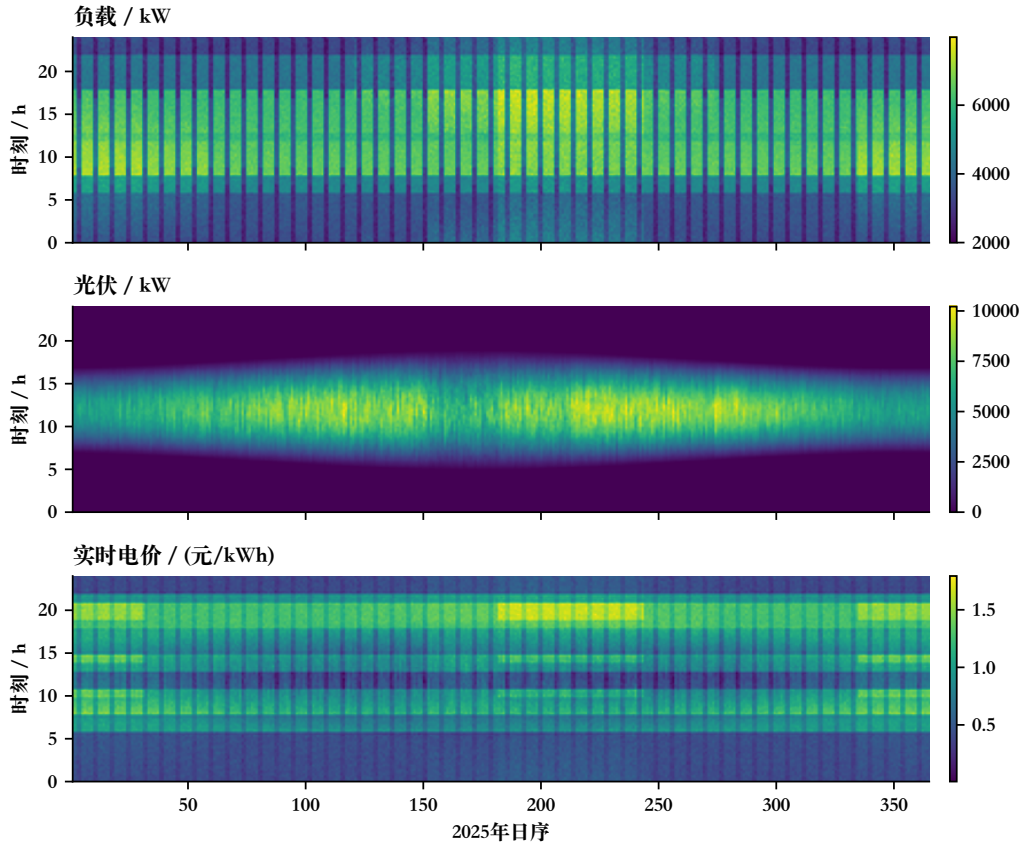


图 1　全年负载、光伏发电与实时电价的日内分布

3.3　符号说明与信息边界

表1列出各问共用符号。带帽符号表示预测值，带横线符号表示优化得到的参考值，上标 act 表示实际执行量。以 r 表示当前决策时刻，以 h 表示预测时域内的相对区间索引。

表 1　主要符号及单位

符号	含义	单位
ℓ_t, v_t, n_t	负载电量、光伏电量、净负荷	kWh
G_t^0, G_t	午夜（零点）原计划购电量、经允许调整后的交付量	kWh
X_t, W_t	紧急购电量、无法利用的余电量	kWh
C_t, D_t	送入电池的充电量、由电池送出的放电量	kWh
E_t	第 t 段开始时电池内部储电量	kWh
$p_t, \widehat{p}_{t\|r}$	实际结算价格、时刻 r 对 t 的预测价格	元/kWh
η, B	单程效率 0.9、每段充放电上限 $5000/6$	无量纲、kWh
K, H_R, H_M	可能情况的条数、长期与短期安排的区间数	条、段
$\bar{C}_t, \bar{D}_t, \bar{E}_t$	优化得到的参考充电、放电和储电量	kWh
$J^{\mathrm{ref}}, C^{\mathrm{act}}$	预测条件下的目标值、实际支付费用	元

3.4　微网通用物理模型

3.4.1　微网能量流关系

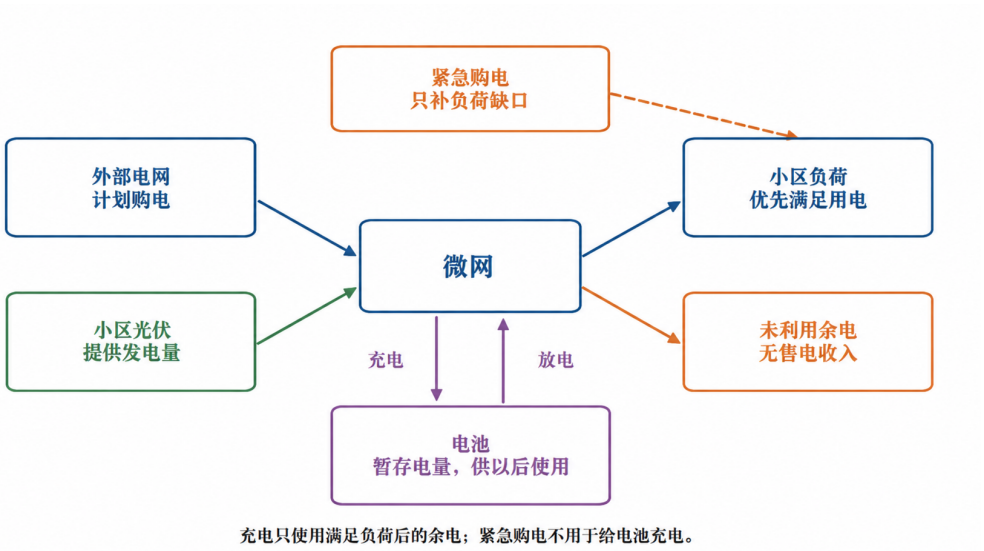


图 2　微网能量流及储能调度关系

图2展示了微网各类电量的流向：光伏发电和计划购电共同满足小区负荷，剩余电量可充入电池以供后续时段使用；当两者与电池放电仍不足以覆盖负荷时，采用紧急购电补足缺口。未利用的余电不产生售电收入，且紧急购电仅用于满足负荷，不用于电池充电。

对任一区间，外网交付、紧急补购、光伏和电池放电是来源，负载、电池充电和弃置余电是去向，因此

$$G_t + X_t + v_t + D_t = \ell_t + C_t + W_t, \qquad G_t, X_t, C_t, D_t, W_t \geq 0, \tag{2}$$

$$E_{t+1} = E_t + \eta C_t - D_t/\eta, \tag{3}$$

$$1200 \leq E_t \leq 10800, \qquad 0 \leq C_t, D_t \leq B = 5000/6. \tag{4}$$

式(2)保证供电；式(3)记录充放电损耗；式(4)保证储电量和功率不超限。100 kWh 送入电池，只增加 90 kWh 储电量；从电池向外送出 100 kWh，需要消耗 $100/0.9$ kWh 内部能量。

3.4.2 通用约束表达

由于余电 W_t 可以免费处置，只需在优化中保证供电不少于需要，即

$$G_t + X_t + D_t - C_t \geq n_t. \tag{5}$$

求解完成后由式(2)计算余电。允许余电处置不等同于退还购电费，已承诺购电仍按规则结算。后续各问仅调整供需预测、购电调整权限与费用结构，通用物理约束保持不变。

四、问题一模型的建立与求解

4.1 问题分析与建模思路

问题一的 144 段负荷、光伏和电价均已知，是确定性多时段优化。储能把当前充放电与后续可用库存连接起来：若从低价时段购入 1 kWh 充电，经过两次效率损失后最多在高价时段送出 $\eta^2 = 0.81$ kWh；光伏富余、功率上限和容量边界还会共同改变实际调度。目标与约束均为线性关系，故采用线性规划统一决定购电、充放电和储电状态。

4.2 确定性优化模型

沿用式(5)的供电条件及式(3)–(4)的储能约束，问题一以 G_t, C_t, D_t 和区间起点储电量 E_t 为变量，不设置紧急购电。对 $t = 1, \ldots, 144$，模型为

$$\min_{G,C,D,E} \quad \sum_{t=1}^{144} p_t G_t, \tag{6}$$

$$\begin{aligned} \text{s.t.} \quad & G_t + D_t - C_t \geq n_t, \qquad G_t \geq 0, \\ & E_{t+1} = E_t + \eta C_t - D_t/\eta, \\ & 0 \leq C_t, D_t \leq B, \qquad 1200 \leq E_t \leq 10800, \\ & 1200 \leq E_{145} \leq 10800, \qquad E_1 = E_{145} = 6000. \end{aligned} \tag{7}$$

供电约束的松弛量

$$W_t = G_t + D_t - C_t - n_t \geq 0$$

就是未被负载或电池利用的余电；它没有售电收入，也不会抵扣已经发生的购电费。终端等式则防止模型通过在最后时段耗尽原有电量得到不可重复的一次性低费用。

4.3 充放电互斥性与求解方法

上述目标和约束均为线性关系，属于凸优化中的线性规划 [1]。电池同段充、放电的直接约束 $C_t D_t = 0$ 是非线性的，但在本题允许免费处置余电的条件下，可以从任一同时充放电的可行解构造同费用的互斥解。

若 $C_t, D_t > 0$，令

$$\delta_t = \min\{C_t, D_t/\eta^2\}, \qquad C'_t = C_t - \delta_t, \qquad D'_t = D_t - \eta^2\delta_t. \tag{8}$$

变换后至少一个变量为零且功率上限仍成立，同时

$$\eta C'_t - D'_t/\eta = \eta C_t - D_t/\eta, \qquad D'_t - C'_t = D_t - C_t + (1-\eta^2)\delta_t.$$

前式保证储电量轨迹不变，后式说明供电松弛只会增加；购电量 G_t 未变，故费用不增加。由此至少存在一个同段充放电互斥的最优解。若允许售电或出现负电价，则需重新检查这一结论。

程序使用 HiGHS 求解线性规划 [2]，完成净化后重新回代全部物理约束；另求解带充放电状态变量的混合整数模型作独立核对。

4.4 调度结果与运行规律

最优全天购电量为 59482.6990 kWh，购电费为 35126.9486 元。图3依次展示供需、储电量以及充放电功率与电价：购电不必逐段等于净负荷，电池在低价阶段和光伏富余阶段吸收电量，在较高电价阶段放电，最终回到 6000 kWh。

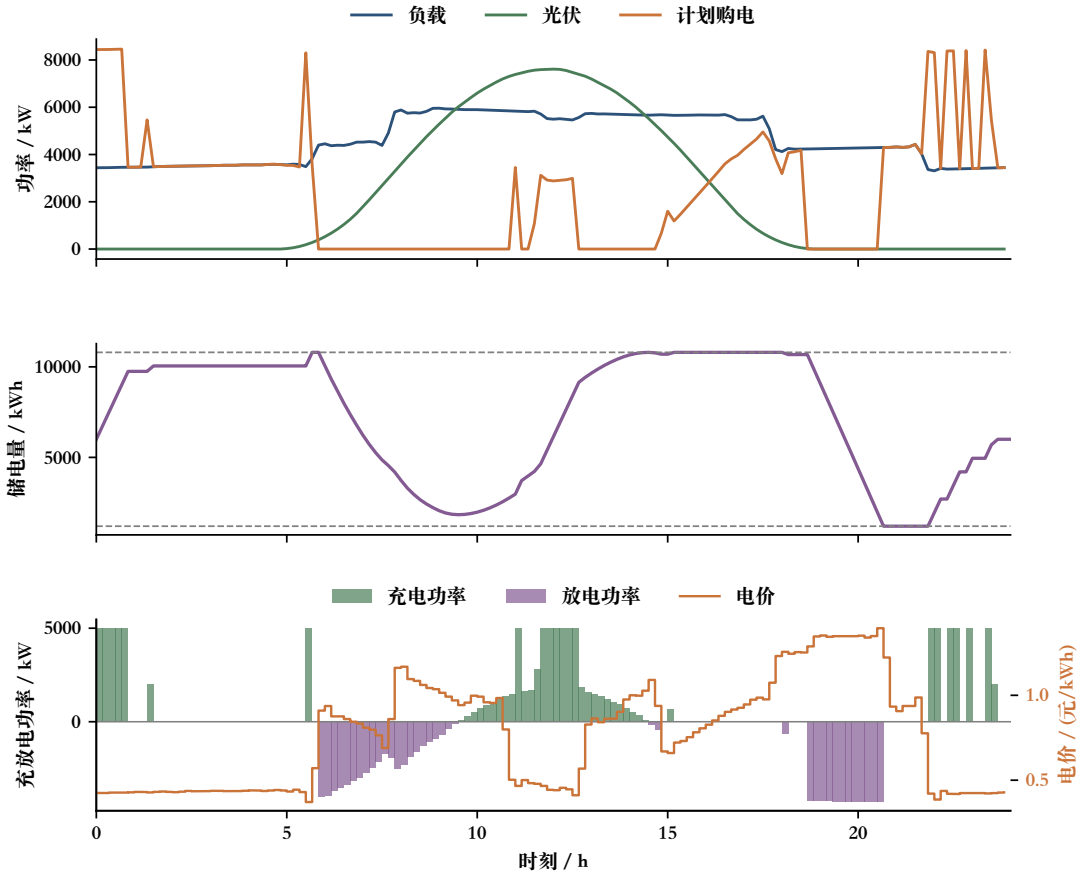


图 3 问题一负载、光伏、购电、储电量及分时价格下的充放电轨迹

表中指定 10 分钟区间按第3.2节的自然日口径读取，例如 12:00–12:10 采用附件中 12:10 这一时刻的数据；完整 144 段结果写入 `result1.xlsx`。充放电量均按电池外部计量，因而 4 小时储电量的净变化不等于该段“充电量减放电量”。

表 2 问题一指定时间段购电量及全天合计

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	480.41	14:00-14:10	0.00
16:00-16:10	445.43	18:00-18:10	531.89	20:00-20:10	0.00
全天购电量		59482.70	全天购电费		35126.95

表 3 问题一储能设备分段充放电量及端点储电量（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4500.00	0.00	04:00-08:00	833.33	6365.84
08:00-12:00	4787.96	1703.00	12:00-16:00	5286.04	91.10
16:00-20:00	0.00	5780.13	20:00-24:00	5333.33	2859.87
00:00 储电量		6000.00	24:00 储电量		6000.00

00:00–04:00 累计充电 4500.00 kWh、没有放电，电池内部增加 4050.00 kWh；16:00–20:00 累计放电 5780.13 kWh，体现了跨时段转移。某个 4 小时汇总中充、放电量均为正，只表示不同 10 分钟区间先后发生充电和放电，不违反单段互斥。

4.5 可行性与经济性检验

将 144 段结果逐一代回能量平衡与储电量递推式，并与无储能基准 $G_t = \max(n_t, 0)$ 比较，得到表4。表中余电、残差和重叠量均由求解账本自动计算，而非在正文手工录入。

表 4 问题一可行性与经济性检验

检验指标	数值	检验指标	数值
无储能购电费/元	48052.05	优化后购电费/元	35126.95
节省费用/元	12925.10	相对节费率	26.90%
最低储电量/kWh	1200.00	最高储电量/kWh	10800.00
24:00 储电量/kWh	6000.00	弃置余电/kWh	0.00
最大能量平衡残差/kWh	2.274×10^{-13}	最大储电量递推残差/kWh	2.103×10^{-12}
最大同段充放电重叠/kWh	0.00	LP–MILP 费用差/元	0.00

与无储能方案相比，储能减少购电费 12925.10 元，节费率为 26.90%。储电量始终位于安全区间并准确恢复到初值，弃置余电在数值容差内为零；线性规划与强制互斥模型费用一致，支持上述互斥化处理和调度结果。

五、问题二模型的建立与求解

5.1 决策时序与两类模型框架

问题二要求每天 0 时提交当日计划购电量，提交后不得回改；日内只能根据实际余缺调整储能动作，并以 5 倍价格补足最终缺口。为处理负载和光伏预测误差，本文在相同的物理约束和实际结算口径下构建两类方案：一类是“成本驱动的风险裕度日前规划—日内即时反馈控制模型”（简称“风险裕度—即时反馈模型”），用滚动残差分位数形成购电安全裕度；另一类是“12 情境随机规划—4 小时滚动控制模型”，用联合误差情境显式描述未来供需的不确定性。

两类模型是对不确定性和日内储能控制的两种处理路径。经相同评价期的回放比较，本文将费用和紧急购电表现更优的风险裕度—即时反馈模型作为问题二的最终方案，12 情境模型则作为并列对照，并凭借其便于接入滚动预报和价格情境的结构，作为问题三、四的扩展基础。图4具体表示风险裕度—即时反馈模型的信息流：前 24 小时计划正式提交，后 24 小时仅用于避免电池在日界处短视。

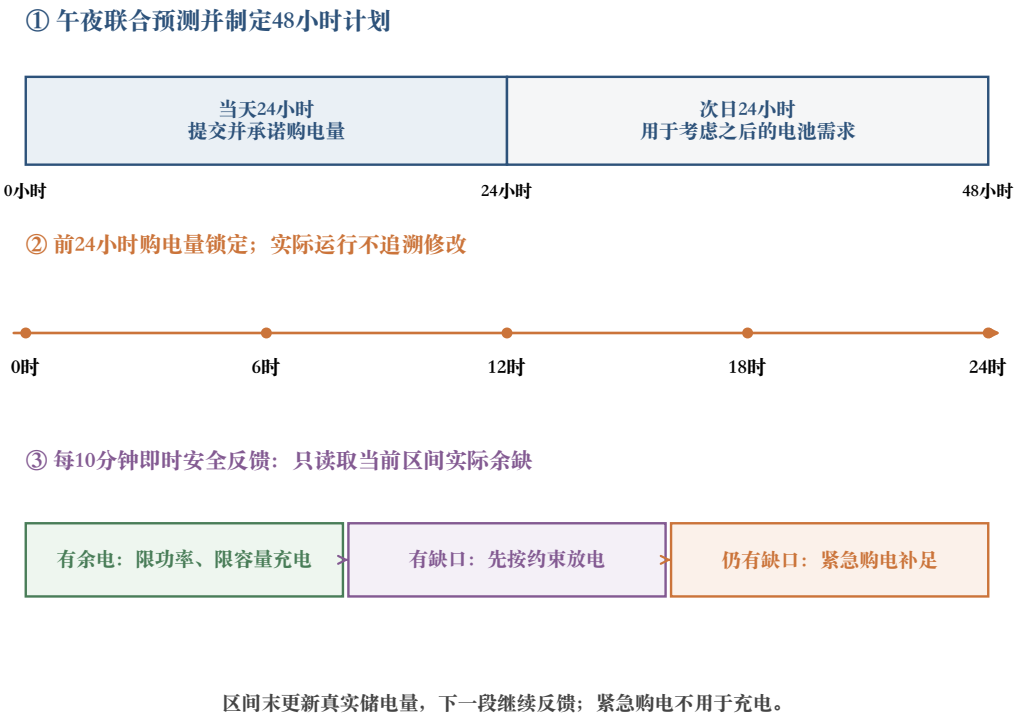


图 4　风险裕度—即时反馈模型的 48 小时计划与 10 分钟安全反馈

全年从 1 月 1 日 0 时、储电量 6000 kWh 连续运行。1 月既是历史逐步积累期，也是实际运行的预热期；2 月 1 日不重置电池，正式 334 日直接继承 1 月末状态，这样避免人为选择有利初始储电量。

5.2 联合因果预测与成本驱动的风险裕度

对运行日 d 的负载和光伏，分别建立三个可解释基预测：前一日同段、前一周同段、最近 7 日逐段中位数。记其向量为 $\boldsymbol{b}_{d,t}^{L}, \boldsymbol{b}_{d,t}^{V}$，用此前最多 14 日数据求非负且和为 1 的权

重：

$$(\boldsymbol{\alpha}_d,\boldsymbol{\beta}_d)=\arg\min_{\boldsymbol{\alpha},\boldsymbol{\beta}}\sum_{j=d-14}^{d-1}\sum_{t}\left|n_{j,t}-(\boldsymbol{\alpha}^{\top}\boldsymbol{b}^{L}_{j,t}-\boldsymbol{\beta}^{\top}\boldsymbol{b}^{V}_{j,t})\right|,\quad \mathbf{1}^{\top}\boldsymbol{\alpha}=\mathbf{1}^{\top}\boldsymbol{\beta}=1. \tag{9}$$

由此得到 $\widehat{\ell}_{d,t}=\boldsymbol{\alpha}_d^{\top}\boldsymbol{b}^{L}_{d,t}$、$\widehat{v}_{d,t}=\boldsymbol{\beta}_d^{\top}\boldsymbol{b}^{V}_{d,t}$。历史不足 7 日时退化为前一日曲线，首日使用附件 1；次日预测用当日预测值代替尚未发生的当日实测值，保证 48 小时计划严格因果。滚动训练和评价始终按时间顺序进行 [3]。

记已揭晓净负荷残差为 $e_{j,t}=n_{j,t}-(\widehat{\ell}_{j,t}-\widehat{v}_{j,t})$。从此前最多 30 日残差计算 q 分位数，并以“逐 10 分钟分位数”和“所在小时池化分位数”各 50% 收缩，得到稳定裕度 $m^{(q)}_{d,t}$；风险净负荷为

$$N^{(q)}_{d,t}=\left[\widehat{\ell}_{d,t}-\widehat{v}_{d,t}+m^{(q)}_{d,t}\right]_{+}. \tag{10}$$

分位数在评价前由题设价格关系确定。忽略储能考察单区间，若净负荷随机变量为 N、计划购电为 G，期望费用 $f(G)=pG+5p\mathbb{E}[N-G]_{+}$。内点一阶条件为

$$p-5p\,\Pr(N>G^{*})=0,\qquad F_N(G^{*})=0.8, \tag{11}$$

故正式取 $q=80\%$。这是一条事前风险折中依据，不是看完全年结果后挑选最便宜分位数。

5.3 48 小时计划购电模型

每天 0 时单独求解一次 48 小时参考计划，并在该次求解内以 $t=1,\dots,288$ 作局部编号：$t=1$ 对应当天 00:00–00:10，$t=145$ 对应次日 00:00–00:10。将今、明两日的风险净负荷 $N_t^{(0.8)}$ 拼接后，令 G_t,C_t,D_t,W_t 分别表示第 t 段内的计划购电量、充电量、放电量和未利用余电量，E_t 表示第 t 段开始时的储电量；p_t 为该段电价。于是建立

$$\min\quad \sum_{t=1}^{288}p_tG_t, \tag{12}$$

$$\begin{aligned}\text{s.t.}\quad & G_t+D_t-C_t-W_t=N_t^{(0.8)},\\ & E_{t+1}=E_t+\eta C_t-D_t/\eta,\\ & 0\le C_t,D_t\le B,\quad W_t,G_t\ge 0,\quad 1200\le E_t\le 10800,\qquad t=1,\dots,288,\\ & E_1=E_0^{\text{act}},\qquad E_{289}=6000.\end{aligned} \tag{13}$$

其中，E_0^{act} 是当天 0 时从前序实际执行账本读出的已知储电量参数，E_1 则是本次局部计划窗口第一段开始时的状态变量；等式 $E_1=E_0^{\text{act}}$ 将二者衔接。由递推式可知，E_{145} 是前 144 段结束、即次日 0 时的储电量，E_{288} 是最后一个 10 分钟区间开始时的储电量，而 E_{289} 才是 288 段全部执行后的 48 小时末端储电量。因此令 $E_{289}=6000$ kWh，用于保留跨日库存价值，避免模型在最后一段透支电池；这不表示每天 24 时强制复位。

正式提交的仅是前 144 段（当天）的 G_t；后 144 段仅作为跨日库存价值的参考，到

次日 0 时会以新的历史信息和新的 E_0^{act} 重新求解。模型为线性规划，可在全年逐日稳定求解。

5.4 10 分钟即时安全反馈

对当前区间 r，令 $a_r = G_r + v_r - \ell_r$。若 $a_r \geq 0$，在功率与容量允许范围内尽量充电；若 $a_r < 0$，先放电，再用紧急购电补足：

$$\begin{aligned} &C_r = \min\{a_r, B, (10800 - E_r)/\eta\}, \qquad D_r = X_r = 0, && a_r \geq 0, \\ &D_r = \min\{-a_r, B, \eta(E_r - 1200)\}, \quad X_r = -a_r - D_r, \quad C_r = 0, && a_r < 0, \\ &W_r = [a_r - C_r]_+, \qquad E_{r+1} = E_r + \eta C_r - D_r/\eta. \end{aligned} \tag{14}$$

该规则只读取本区间持续监测得到的平均供需，不回改 G_r。它自动满足充放电互斥、禁止紧急购电充电，并把未满足负荷精确记入 X_r。下一段从真实 E_{r+1} 继续，形成跨日连续闭环。

5.5 风险裕度-即时反馈模型结果与敏感性

问题二实际费用按

$$C_2^{\text{act}} = \sum_t p_t G_t + 5 \sum_t p_t X_t \tag{15}$$

核算。正式 334 日总费用为 14632621.39 元，其中计划费用 14134516.24 元、紧急购电费用 498105.16 元；紧急购电量 81988.50 kWh。2 月 1 日初始储电量为 9418.37kWh，年末为 8738.40kWh，均由 1 月连续运行和后续反馈自然得到。

表5给出事前设定的 70%、80%、90% 三组完整回放。80% 相对 70% 仅增加 0.49% 总费用，却使紧急购电量下降 39.80%，在经济性与供电风险之间形成明显折中；90% 继续降低紧急量，但计划费用上升更多。因此，风险裕度—即时反馈模型采用由 5 倍紧急价格推导的 80% 临界分位数，而不以事后最低总费用取代事前规则。

表 5 问题二滚动残差分位数敏感性（2 月至 12 月）

分位数	计划费用/元	紧急购电量/kWh	总费用/元
70%	13733589.03	136190.00	14561692.22
80%	14134516.24	81988.50	14632621.39
90%	14731430.95	36517.04	14955181.09

5.6 12 情境随机规划与 4 小时滚动控制模型

该方案的核心思路是把问题二分成“日前定合同、日内调储能、实时保供电”三个层次。每日 0 时，48 小时情境规划同时权衡普通购电费和不同供需情境下的紧急购电风险，只将前 24 小时的购电量作为不可更改的当日合同；进入日内后，每 10 分钟根据最新历史和真实库存重算未来 4 小时的储能轨迹，但只执行第一个 10 分钟动作；本区间供需实现后，再由安全层修正该动作并计算紧急购电。三层的作用如表6所示。

表 6　12 情境方案的决策时序与各层作用

层次	求解时刻	主要输入与输出	实际采用部分
日前规划	每日 0 时	由当时可得历史构造 12 条未来路径，得到购电计划及参考储能轨迹	提交前 24 小时购电量；后 24 小时只作为跨日参考
日内控制	每 10 分钟开始前	读取当前真实储电量，在固定购电合同下重算未来 24 段充放电轨迹	只采用首段充、放电目标，下一段重新求解
安全执行	供需实现后	用实际负载、光伏和设备边界修正首段目标	有余电才充电，有缺口才放电，仍不足时紧急购电

5.6.1　负载–光伏联合情境

在任一决策时刻 r，先以同星期历史、近期均值和附件 1 先验形成未来基准预测 $\widehat{\ell}_{r+h|r}$、$\widehat{v}_{r+h|r}$。再从决策时刻以前最多 60 日中选取 $K=12$ 个已经完整揭晓的连续误差块；第 k 个块同时提供负载误差 $\varepsilon_h^{L,k}$ 和光伏误差 $\varepsilon_h^{V,k}$，由此构造

$$\ell_{r+h}^{(k)}=\left[\widehat{\ell}_{r+h|r}+\varepsilon_h^{L,k}\right]_+,\qquad v_{r+h}^{(k)}=\left[\widehat{v}_{r+h|r}+\varepsilon_h^{V,k}\right]_+,\qquad n_{r+h}^{(k)}=\ell_{r+h}^{(k)}-v_{r+h}^{(k)}. \tag{16}$$

同一情境中的两类误差取自同一历史起点，且保留整段误差序列，不逐时打乱，因而能够同时保留负载与光伏的相关性以及误差的时间连续性。12 条情境等权使用；所有误差块均在 r 之前完整结束，因此模型不会读取决策时尚未发生的数据。每日 0 时规划取 $H_R=288$ 段，日内控制取 $H_M=24$ 段，分别对应 48 小时和 4 小时。

5.6.2　第一层：每日 0 时的 48 小时情境规划

在本次 48 小时窗口内，以 $t=1,\dots,H_R$ 作局部编号。G_t 为计划购电量，$\bar{C}_t,\bar{D}_t,\bar{E}_t$ 为参考充电量、放电量和储电量；$X_t^{(k)},W_t^{(k)}$ 分别为情境 k 下可能需要的紧急购电量和未利用余电量。模型为

$$\begin{aligned}
\min\quad & \sum_{t=1}^{H_R}p_tG_t+\frac{1}{K}\sum_{k=1}^{K}\sum_{t=1}^{H_R}5p_tX_t^{(k)}, && && (17)\\
\text{s.t.}\quad & G_t+X_t^{(k)}+\bar{D}_t-\bar{C}_t-W_t^{(k)}=n_t^{(k)}, && t=1,\dots,H_R,\ k=1,\dots,K,\\
& \bar{E}_{t+1}=\bar{E}_t+\eta\bar{C}_t-\bar{D}_t/\eta, && t=1,\dots,H_R,\\
& 0\le\bar{C}_t,\bar{D}_t\le B,\quad G_t\ge 0, && t=1,\dots,H_R,\\
& 1200\le\bar{E}_t\le 10800, && t=1,\dots,H_R+1,\\
& X_t^{(k)},W_t^{(k)}\ge 0, && t=1,\dots,H_R,\ k=1,\dots,K,\\
& \bar{E}_1=E_0^{\text{act}},\qquad \bar{E}_{H_R+1}=6000. &&
\end{aligned}$$

目标函数第一项是确定支付的计划购电费，第二项是 12 条等概率路径下的平均紧急购电费。关键在于，$G_t,\bar{C}_t,\bar{D}_t,\bar{E}_t$ 对所有情境均相同，表示在未来误差尚未揭晓时就要给出一条共同的计划；只有 $X_t^{(k)}$ 和 $W_t^{(k)}$ 随情境变化，用来核算不同路径下可能出现的缺

电和余电。因此，模型不会利用“提前知道将发生哪条情境”的信息来选择储能动作。

48 小时末端的 6000 kWh 只用于防止有限时域模型在末端透支电池，并不要求每天 24 时恢复到 6000 kWh。求解后仅提交 $t = 1, \dots, 144$ 的 G_t，次日部分和整条 $\bar{E}_t$ 均为日内控制提供参考；下一日 0 时仍从实际库存重新规划。

5.6.3 第二层：固定合同下的 4 小时滚动控制

设 r 为当前 10 分钟区间的开始时刻。此时当日购电合同 G 已经确定，不能因新预测而修改。记 $\widetilde{G}_{r+h}$ 为本次 4 小时控制窗口内固定使用的购电量：位于当日的区间采用已提交合同，若窗口跨到次日，则采用午夜规划中尚未提交的次日参考值。模型以实际库存 E_r^{act} 为起点，重新构造未来 $H_M = 24$ 段的 12 条联合情境，求解共享的充放电轨迹 $\bar{C}_{r,h}^M, \bar{D}_{r,h}^M, \bar{E}_{r,h}^M$：

$$
\begin{aligned}
\min \quad & \frac{1}{K} \sum_{k=1}^{K} \sum_{h=0}^{H_M-1} 5 p_{r+h} X_{r,h}^{(k)}, \\
\text{s.t.} \quad & \widetilde{G}_{r+h} + X_{r,h}^{(k)} + \bar{D}_{r,h}^M - \bar{C}_{r,h}^M - W_{r,h}^{(k)} = n_{r+h}^{(k)}, \\
& \bar{E}_{r,h+1}^M = \bar{E}_{r,h}^M + \eta \bar{C}_{r,h}^M - \bar{D}_{r,h}^M / \eta, \qquad \bar{E}_{r,0}^M = E_r^{\text{act}}, \\
& \bar{E}_{r,H_M}^M = E_r^{\text{tar}}, \quad 0 \le \bar{C}_{r,h}^M, \bar{D}_{r,h}^M \le B, \quad 1200 \le \bar{E}_{r,h}^M \le 10800, \\
& X_{r,h}^{(k)}, W_{r,h}^{(k)} \ge 0, \qquad h = 0, \dots, H_M - 1,\ k = 1, \dots, K.
\end{aligned}
\tag{18}
$$

式(18)中没有再写计划购电费，是因为 $\widetilde{G}_{r+h}$ 在本次求解中已固定，该项对储能决策是常数；此时优化的目的，就是利用电池尽量降低未来 4 小时的期望紧急购电费。

短窗口末端目标取自 48 小时参考轨迹。若实际库存已经偏离参考轨迹，原目标可能在 4 小时内无法到达，因此先按功率与容量限制投影：

$$
\begin{aligned}
L_r &= \max\{1200, E_r^{\text{act}} - H_M B/\eta\}, \qquad U_r = \min\{10800, E_r^{\text{act}} + H_M B \eta\}, \\
E_r^{\text{tar}} &= \min\{U_r, \max\{L_r, \bar{E}_{r+H_M}^{48}\}\}.
\end{aligned}
\tag{19}
$$

其中 $\bar{E}_{r+H_M}^{48}$ 是午夜 48 小时规划给出的同一时刻参考库存。投影使末端约束位于当前库存理论上可达的区间内，从而避免短窗口因库存偏差而不可行。每次求解后只取 $h = 0$ 的 $\bar{C}_{r,0}^M, \bar{D}_{r,0}^M$ 作为本区间目标；到 $r + 1$ 时读入新的实际库存并重复上述过程，这就是 4 小时滚动控制。

5.6.4 第三层：按实际供需安全执行

滚动优化使用的是预测情境，不能直接保证首步目标一定符合本区间的实际余缺。令 $a_r = G_r + v_r - \ell_r$，则安全层执行

$$
\begin{aligned}
C_r^{\text{act}} &= \min\left\{\bar{C}_{r,0}^{M}, [a_r]_+, B, \frac{10800 - E_r^{\text{act}}}{\eta}\right\},\\
D_r^{\text{act}} &= \min\left\{\bar{D}_{r,0}^{M}, [-a_r]_+, B, \eta(E_r^{\text{act}} - 1200)\right\},\\
X_r^{\text{act}} &= [-a_r - D_r^{\text{act}}]_+, \qquad W_r^{\text{act}} = [a_r - C_r^{\text{act}}]_+,\\
E_{r+1}^{\text{act}} &= E_r^{\text{act}} + \eta C_r^{\text{act}} - D_r^{\text{act}}/\eta.
\end{aligned}
\tag{20}
$$

因此，实际有余电时才允许充电，实际有缺口时才允许放电，放电后仍未满足的负荷才由紧急购电补足。该规则保证紧急购电不会用于充电，并使实际能量平衡、功率限制和储电量边界始终成立。需要特别区分：48 小时规划决定当日购电合同，4 小时模型只给出储能目标，安全层才产生最终实际充放电量与紧急购电量。

5.6.5 结果比较

12 情境模型属于有限情境随机规划与滚动控制的结合 [4–6]。其 334 日实际总费用为 14824938.70 元，紧急购电量为 253922.05 kWh。相同设备参数、评价日期和结算口径下，风险裕度—即时反馈模型少支出 192317.31 元，即 1.30%；紧急购电量减少 171933.55 kWh，即 67.71%，因此本文选其作为问题二最终方案。

这一结果表明，在本年度数据和题设 5 倍紧急购电价格下，12 条等权历史情境对高净负荷尾部的覆盖仍然有限；直接用 80% 分位数形成合同安全裕度，实际回放表现更好。该结论是同一数据集上的模型比较，并不意味着分位数模型在其他年份或其他应急价格下必然占优。12 情境模型能够保留联合误差路径，且便于在新预报到达后重构未来情境，因此仍作为问题三、四的扩展基础；其参数、逐时账本和复现入口均保留在支撑材料中。

5.7 指定日期结果与可行性

以下四个日期的计划购电、充放电、储电量和紧急购电均由风险裕度—即时反馈模型账本自动聚合。6 月 21 日光伏充足，实际电费最低；12 月 21 日净负荷较高，实际电费最高。四日中只有 9 月 23 日发生紧急购电，集中在 20:20–20:30 且仅 6.38 kWh，说明 80% 裕度覆盖了绝大多数展示日偏差，但仍保留少量尾部缺口。

表 7　问题二：2025-03-20 购电结果（电量：kWh；费用：元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	833.33	14:00-14:10	0.00
16:00-16:10	501.00	18:00-18:10	714.11	20:00-20:10	0.00
全天计划购电量	72305.96	全天计划电费	43696.42		

含紧急补电的实际总购电量 72305.96 kWh，实际总电费 43696.42 元。

表 8　问题二：2025-03-20 储能充放电与端点储电量（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	1454.95	189.73	04:00-08:00	854.40	5125.76
08:00-12:00	6291.46	2777.38	12:00-16:00	3552.03	401.28
16:00-20:00	315.14	5575.58	20:00-24:00	8416.68	2953.12
00:00 储电量	9137.54	24:00 储电量	9019.48		

表 9　问题二：2025-06-21 购电结果（电量：kWh；费用：元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	833.33	14:00-14:10	0.00
16:00-16:10	248.26	18:00-18:10	88.89	20:00-20:10	0.00
全天计划购电量	47997.33	全天计划电费	27128.83		

含紧急补电的实际总购电量 47997.33 kWh，实际总电费 27128.83 元。

表 10　问题二：2025-06-21 储能充放电与端点储电量（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	0.00	0.00	04:00-08:00	348.72	1723.41
08:00-12:00	1778.95	0.00	12:00-16:00	0.00	0.00
16:00-20:00	349.47	5068.83	20:00-24:00	8977.35	2485.90
00:00 储电量	10800.00	24:00 储电量	10800.00		

表 11 问题二：2025-09-23 购电结果（电量：kWh；费用：元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	833.33	14:00-14:10	0.00
16:00-16:10	579.88	18:00-18:10	784.29	20:00-20:10	0.00
全天计划购电量	75781.45	全天计划电费	46640.00		

含紧急补电的实际总购电量 75787.82 kWh，实际总电费 46683.00 元。

表 12 问题二：2025-09-23 储能充放电与端点储电量（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	2100.81	30.83	04:00-08:00	447.57	5667.98
08:00-12:00	6293.08	1896.62	12:00-16:00	3417.33	512.57
16:00-20:00	101.55	5636.83	20:00-24:00	8551.57	2990.29
00:00 储电量	8734.41	24:00 储电量	8960.55		

表 13 问题二：2025-12-21 购电结果（电量：kWh；费用：元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	997.81	14:00-14:10	0.00
16:00-16:10	829.72	18:00-18:10	737.89	20:00-20:10	0.00
全天计划购电量	99340.27	全天计划电费	64230.27		

含紧急补电的实际总购电量 99340.27 kWh，实际总电费 64230.27 元。

表 14 问题二：2025-12-21 储能充放电与端点储电量（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	1403.13	77.22	04:00-08:00	826.51	1441.55
08:00-12:00	5926.72	6721.57	12:00-16:00	6108.17	2213.11
16:00-20:00	0.00	5646.15	20:00-24:00	8497.40	2842.53
00:00 储电量	9576.86	24:00 储电量	9015.80		

表 15 问题二：指定日期紧急购电时段与电量（kWh）

2025-03-20		2025-06-21		2025-09-23		2025-12-21	
时间段	电量	时间段	电量	时间段	电量	时间段	电量
无	0.00	无	0.00	20:20-20:30	6.38	无	0.00

在四个展示日期中，6 月 21 日实际电费最低，为 2.71 万元；当日负荷为 8.14 万 kWh，光伏发电量为 6.21 万 kWh。12 月 21 日负荷增至 12.44 万 kWh，光伏发电量降至 3.53 万 kWh，实际电费为 6.42 万元。对比表明，外网购电决策同时受分时电价及净负荷水平影响。

储能表按 4 小时汇总多个 10 分钟区间，因此同一汇总时段可同时出现正的累计充电量与累计放电量。单日累计充电量也可能超过电池容量，这反映日内多次循环；容量

约束限制任一时刻的储电量，而非全天累计能量吞吐量。全年逐段验证能量守恒、储电量递推、容量与功率边界、跨日连续、充放电互斥、禁止紧急购电充电及训练信息边界；检验结果见第八节。

六、问题三模型的建立与求解

6.1 模型选择依据与总体思路

问题三的难点是新信息分批到达，而每次只能修改尚未执行的合同；购电调整、储能库存和紧急补购又通过未来供需共同耦合。单点滚动优化会忽略尾部误差，完整多阶段情景树则随信息节点迅速膨胀。因此，本文采用 **12 情境随机规划与 4 小时滚动控制**：预报发布时重构联合误差路径并优化未执行合同，发布时点之间每 10 分钟重算储能轨迹、只执行首步，最后由安全层保证实际供电。该结构保留误差相关性和时间连续性，又能把分段线性调整费用直接纳入线性规划 [4–6]。

$K = 12$ 与 4 小时窗口由计算量和收益共同确定：情境数由 6 增至 12 可降低 17.78 万元，继续增至 24 仅再降低 4.10 万元；控制窗口由 2 小时增至 4 小时降低 2.32 万元，继续增至 6 小时仅再降低 0.99 万元。问题三的预报价值只与同一模型下“仅 0 时预报”比较，避免把更换模型的影响误认为新预报收益。

6.2 调整机制与决策变量

问题三在上述框架中引入 6 时、12 时和 18 时发布的新光伏预报，并据此调整尚未执行的购电计划。购电量的调整方向由最新预报、附件 1 规定的标准购电价格及调整结算规则共同决定：当预计缺电风险上升时，可上调计划购电量，以减少价格更高的紧急购电；当预计净需求下降时，则可下调计划购电量，从而减少部分原计划费用。

每日 0 时，模型利用当时最新发布的光伏预报制定日前购电计划 G^0；在其余预报发布时点，仅对当前及后续尚未交付区间的有效购电量 G 进行调整。购电计划仅在规定的预报发布时点更新，储能决策则以 10 分钟为间隔执行，因此二者具有不同的决策时间尺度。各版预报严格按照其发布时间纳入模型，并仅在相应的 24 小时预测范围内进行插值；超出该范围的时段仍采用历史预测结果。

6.3 调整费用与上下调边际

令 $a_t^+ = [G_t - G_t^0]_+$ 为上调量，$a_t^- = [G_t^0 - G_t]_+$ 为下调量。实际电费为

$$C_3^{\text{act}} = \sum_t p_t \left(G_t^0 - 0.5 a_t^- + 1.5 a_t^+ + 5 X_t^{\text{act}} \right). \tag{21}$$

式中四项依次表示原计划费用、取消部分对应的费用扣减、新增购电费用和紧急购电费用。该结算方式也可表述为：下调时，保留量 G_t 按标准购电价格支付，取消量 $G_t^0 - G_t$ 按 $0.5p_t$ 支付违约费用；上调时，原计划量按标准购电价格支付，新增量按 $1.5p_t$ 支付。对每个时段，仅按照最终生效购电量与午夜原计划之间的差额结算一次，不累计计算历次调整产生的中间差额。

例如，设标准购电价格为 1 元/kWh，午夜原计划购电量为 100 kWh。上调至 120

kWh 时，计划及调整费用为 $100+1.5\times 20=130$ 元；下调至 80 kWh 时，保留的 80 kWh 支付 80 元，取消的 20 kWh 按 50% 支付 10 元，合计 $80+0.5\times 20=90$ 元。这也等于 $100-0.5\times 20$ 元。

按照上述结算规则，每下调 1 kWh 购电量可节省 $0.5p_t$，因此优化模型不应预先排除下调决策。为说明上下调决策的边际条件，暂不考虑储能，并以 N 表示随机净需求。当 $G<G^0$ 时，增加 1 kWh 计划购电量的边际费用为 $0.5p_t$，与其减少缺电风险所带来的 $5p_t\Pr(N>G)$ 相平衡，对应净需求的 90% 分位数；当 $G>G^0$ 时，边际费用为 $1.5p_t$，对应净需求的 70% 分位数。因此，在单时段情形下，当 G^0 低于净需求的 70% 分位数时，应上调购电量；当其高于 90% 分位数时，应下调购电量；当其位于二者之间时，则维持原计划不变。全年模型还包含储能及其跨时段耦合关系，实际调整方案由完整优化模型求得。

6.4 滚动调整优化模型

每次新预报发布后，模型首先更新未来光伏预测及其对应的 12 种误差情景，随后以当前实际储电量为初始状态，重新求解后续时域的购电与储能计划。记 $\mathcal{A}_r$ 为当日尚未执行的时间区间集合，$\mathcal{B}_r$ 为次日的参考时间区间集合，则滚动优化目标为

$$\begin{aligned}\min \quad J_3(r)=&\sum_{t\in\mathcal{A}_r}p_t(G_t^0-0.5a_t^-+1.5a_t^+)+\sum_{t\in\mathcal{B}_r}p_tG_t\\&+\frac{1}{K}\sum_{k=1}^{K}\sum_{t\in\mathcal{A}_r\cup\mathcal{B}_r}5p_tX_t^{(k)}.\end{aligned}\tag{22}$$

模型保留情景供电约束、储电量递推关系和全部设备运行限制，并对当日尚未执行的区间增加以下约束：

$$G_t-G_t^0=a_t^+-a_t^-,\qquad a_t^+,a_t^-\ge 0,\qquad t\in\mathcal{A}_r.$$

优化以 E_r^{act} 为起点，并将 48 小时参考时域的末端储电量保持为 6000 kWh。由于次日购电计划尚未提交，因此 $\mathcal{B}_r$ 中的购电量仅按附件 1 规定的标准购电价格参与参考优化；待次日 0 时，再重新生成该日的正式购电计划。

求解完成后，模型使用新的 G 替换尚未执行区间的原购电计划，而已经发生的购电行为及相应费用保持不变。问题三沿用 12 情境模型的 4 小时滚动优化机制和安全执行规则。由此，新预报不仅能够改变后续时段的有效购电计划，还会影响储能在不同时段之间的能量配置。

6.5 预报发布时点选择

为识别不同日内预报更新的边际作用，本文设置五种方案：仅使用 0 时预报；在 0 时预报的基础上，分别增加 6 时、12 时或 18 时的一次更新；以及采用 6 时、12 时和 18 时的全部更新。各方案采用相同的设备参数、预测基线、结算规则和评价日期，仅改变允许使用的预报发布时点。

正式评价期内，仅使用 0 时预报时的总费用为 15057834.98 元；采用全部三次日内更新后降至 14662124.53 元，共节省 395710.45 元，即 2.63%。在三种单次更新方案中，增加 6 时、12 时和 18 时更新后的费用分别为 14895395.53 元、14742351.59 元和 14945491.35 元，其中 12 时更新收益最大；在 0 时和 12 时更新基础上采用全部更新，还可减少 80227.06 元。因此，本文将全部日内更新方案作为问题三主策略。

日内预报更新主要通过三种途径降低运行费用：第一，当预计缺电风险上升时，以 1.5 倍标准购电价格上调计划购电量，从而减少按 5 倍标准购电价格结算的紧急购电；第二，当预计供电余量增加时，下调计划购电量；第三，根据光伏预测的变化重新优化后续储能轨迹。当预报发布时间较晚或储电量达到边界时，更新所带来的边际收益可能较小。相关研究同样强调，应以最终调度成本评价预测结果的改进 [7]。

6.6 策略比较与结果分析

问题三在正式评价期内的费用为 14662124.53 元，紧急购电量为 179940.20 kWh。其严格控制组为同一模型中仅使用 0 时预报的方案；全部日内更新相对该控制组节省 395710.45 元，即 2.63%，并使紧急购电量减少 63190.63 kWh，即 25.99%。因此，新预报的价值不仅体现为更低的合同费用，也体现为对高价缺口的及时修正。

从调整方向看，334 天内累计上调 19.53 万 kWh、下调 76.39 万 kWh，最终有效购电量较午夜计划净减少 56.86 万 kWh。下调量明显大于上调量，说明日内信息主要用于撤回偏保守的日前承诺；但选择性上调仍不可缺少，它以 1.5 倍价格替代了部分 5 倍紧急购电。单次更新中 12 时费用最低，表明接近日间光伏峰值的新信息最有价值；全部三次更新仍优于只在 12 时更新，说明早晚偏差也会通过储能库存影响后续费用。

需要说明的是，问题二最终采用的风险裕度—即时反馈模型与问题三采用的 12 情境滚动框架在误差处理和日内控制方式上不同，因此二者的费用差异不能直接归因于预报更新。问题三的预报信息价值应通过本节设置的内部控制实验进行评价；跨问题的结果仅在统一运行汇总表中并列展示，不能替代问题三内部方案之间的比较。

`result3.xlsx` 分别记录零点原计划、调整后的最终有效购电量以及储能的实际运行情况。下文选取与问题二相同的四个指定日期展示调度结果，以便在统一的数据口径下进行对照。

表 16　问题三：2025-03-20 购电结果（电量：kWh；费用：元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00 / 0.00	12:00-12:10	541.79 / 526.47	14:00-14:10	0.00 / 0.00
16:00-16:10	663.16 / 536.02	18:00-18:10	700.81 / 700.81	20:00-20:10	0.00 / 0.00
全天计划购电量	68952.66	全天计划电费			42479.66

表内“计划 / 调整”分别为午夜承诺与最终承诺；调整后计划电量 64941.93 kWh。含紧急补电的实际总购电量 65755.86 kWh，实际总电费 44999.49 元。

表 17　问题三：2025-03-20 储能充放电与端点储电量（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	1549.05	2.46	04:00-08:00	816.75	5968.14
08:00-12:00	4403.72	2606.16	12:00-16:00	6105.10	211.15
16:00-20:00	176.54	5578.81	20:00-24:00	6264.07	2895.28
00:00 储电量	8683.81	24:00 储电量	6887.52		

表 18　问题三：2025-06-21 购电结果（电量：kWh；费用：元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00 / 0.00	12:00-12:10	0.00 / 0.00	14:00-14:10	0.00 / 0.00
16:00-16:10	342.55 / 269.74	18:00-18:10	374.12 / 374.12	20:00-20:10	0.00 / 0.00
全天计划购电量	44609.30	全天计划电费			26672.36

表内“计划 / 调整”分别为午夜承诺与最终承诺；调整后计划电量 44189.74 kWh。含紧急补电的实际总购电量 44191.12 kWh，实际总电费 26497.44 元。

表 19　问题三：2025-06-21 储能充放电与端点储电量（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	101.66	0.00	04:00-08:00	35.85	3932.23
08:00-12:00	4746.06	0.00	12:00-16:00	2510.02	0.00
16:00-20:00	68.37	4783.60	20:00-24:00	7273.58	2485.90
00:00 储电量	8484.76	24:00 储电量	9300.38		

表 20　问题三：2025-09-23 购电结果（电量：kWh；费用：元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00 / 0.00	12:00-12:10	414.77 / 414.77	14:00-14:10	0.00 / 0.00
16:00-16:10	583.87 / 565.77	18:00-18:10	806.95 / 806.95	20:00-20:10	15.36 / 15.36
全天计划购电量		74486.36	全天计划电费		46592.17

表内“计划 / 调整”分别为午夜承诺与最终承诺；调整后计划电量 71224.59 kWh。含紧急补电的实际总购电量 71395.52 kWh，实际总电费 46315.30 元。

表 21　问题三：2025-09-23 储能充放电与端点储电量（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	1402.39	0.00	04:00-08:00	852.65	5994.99
08:00-12:00	5286.90	1896.62	12:00-16:00	4909.01	403.62
16:00-20:00	29.22	5581.27	20:00-24:00	8092.46	2861.02
00:00 储电量		8811.07	24:00 储电量		8729.19

表 22　问题三：2025-12-21 购电结果（电量：kWh；费用：元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00 / 0.00	12:00-12:10	990.38 / 927.08	14:00-14:10	0.00 / 0.00
16:00-16:10	878.45 / 878.45	18:00-18:10	721.01 / 721.01	20:00-20:10	0.00 / 0.00
全天计划购电量		97358.05	全天计划电费		62475.96

表内“计划 / 调整”分别为午夜承诺与最终承诺；调整后计划电量 94869.44 kWh。含紧急补电的实际总购电量 95485.81 kWh，实际总电费 64220.68 元。

表 23　问题三：2025-12-21 储能充放电与端点储电量（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	1276.58	41.71	04:00-08:00	1699.98	1582.24
08:00-12:00	5076.31	7444.26	12:00-16:00	7971.67	2209.51
16:00-20:00	0.00	5647.81	20:00-24:00	8021.08	2825.53
00:00 储电量		8908.71	24:00 储电量		8604.14

表 24　问题三：指定日期紧急购电时段与电量（kWh）

2025-03-20		2025-06-21		2025-09-23		2025-12-21	
时间段	电量	时间段	电量	时间段	电量	时间段	电量
00:20-00:30	28.44	01:20-01:30	1.38	01:40-01:50	3.11	01:10-01:20	22.42
01:10-02:00	76.56			04:50-05:10	43.21	02:00-02:20	13.60
02:30-02:50	8.48			05:50-06:20	26.94	03:00-03:20	73.08
03:00-03:50	85.56			06:50-07:10	27.53	03:30-04:00	104.38
04:50-05:10	23.43			07:40-07:50	15.33	04:10-04:20	2.49
05:50-06:00	16.49			18:20-18:40	30.82	05:00-05:20	4.17

表 24 问题三：指定日期紧急购电时段与电量（续）

2025-03-20		2025-06-21		2025-09-23		2025-12-21	
时间段	电量	时间段	电量	时间段	电量	时间段	电量
08:10-08:20	35.90			19:20-19:30	6.21	05:40-06:00	33.30
08:50-09:10	90.11			20:20-20:30	6.38	07:50-08:00	10.09
09:50-10:00	45.21			20:50-21:00	11.40	08:20-08:30	21.33
14:10-14:30	38.84					08:40-09:00	24.35
14:40-14:50	5.54					09:20-09:50	28.34
15:20-16:20	162.60					10:30-10:50	110.12
16:50-17:00	1.84					13:50-14:00	10.60
18:40-19:10	89.09					17:20-17:30	28.54
19:50-20:20	62.58					18:30-19:00	37.25
21:20-21:30	23.43					19:50-20:10	78.41
23:40-23:50	19.82					23:40-23:50	13.92

在问题三主策略中，9 月 23 日紧急购电量为 170.93 kWh、实际电费为 46315.30 元；3 月 20 日紧急购电量为 813.93 kWh、实际电费为 44999.49 元。不同日期的更新收益还取决于负荷、光伏、库存与调整价格，故总体经济性以“全部更新—仅 0 时”完整 334 天控制实验评价。

购电表斜线两侧分别表示午夜原计划与最终有效购电量，二者之差为最终调整量。储能表记录实际充放电量，紧急购电表记录安全执行后仍需补足的电量，三类结果共同构成当日实际供电账本。

七、问题四模型的建立与求解

7.1 电价预测与实际结算

问题四在问题二和问题三的基础上进一步考虑实时电价的变化。由于制定购电计划时无法提前获得未来的真实电价，因此需要根据已经发生的历史电价对未来价格进行预测。

对于未来各时段的电价，本文利用过去 28 天中相同星期的电价均值、最近 7 天的电价均值以及附件 1 电价，分别赋予 0.60、0.30 和 0.10 的权重，得到基础预测 [8]:

$$\hat{p}_t^{\text{base}} = 0.60p_t^{\text{week}} + 0.30p_t^{\text{recent}} + 0.10p_t^{\text{prior}},$$

其中，p_t^{week} 表示近期相同星期对应时段的历史电价，p_t^{recent} 表示最近 7 天对应时段的平均电价，p_t^{prior} 表示附件 1 给出的参考电价。

再用决策时刻以前的历史误差修正基础值，得到 $\hat{p}_{r+h|r}$，并构造 K 条可能价格路径。第 k 种情景下

$$p_{r+h}^{(k)} = \exp\left(\log \hat{p}_{r+h|r} + \varepsilon_h^{\log p,k}\right),$$

其中 $\varepsilon_h^{\log p,k}$ 为第 k 个历史块中的对数价格误差。本文取 $K = 12$，且负载、光伏和电

价误差来自同一段历史，使每条情景保留三者的共同变化；实际电价始终只用于交付结算和事后评价。

7.2 两类交易机制的统一模型

先用一个费用系数 g_t 表示普通购电及调整部分：尚可自由安排购电的区间取 $g_t = G_t$；问题三式的当天调整区间取 $g_t = G_t^0 - 0.5a_t^- + 1.5a_t^+$。波动电价下的长计划目标统一为

$$\min \quad J_4 = \frac{1}{K}\sum_{k=1}^{K}\sum_{t} p_t^{(k)}\left(g_t + 5X_t^{(k)}\right). \tag{23}$$

不允许日内调整时，午夜确定当天 G^0 后不再改变；允许调整时，在规定发布时刻按问题三的原计划基准更新 G。供电、储能、末端和执行规则均与各自对应的固定电价方案一致，区别是目标中的价格也随情景变化。短期 4 小时控制同样用对应价格情景计算平均应急费用。

实际结算仍分别采用式(15)和式(21)，其中 p_t 取交付区间真实价格。预测价格仅用于决策，实际价格用于结算，二者对应的目标值与实付费用严格区分。

7.3 运行结果与策略比较

对 334 天连续回放，不允许日内调整时总费用为 15584051.46 元、紧急购电量为 243834.28 kWh；允许调整后分别降至 15385170.51 元和 177011.89 kWh。后者节省 198880.94 元，即 1.28%，并减少紧急购电 66822.40 kWh，即 27.40%。允许调整策略的有效购电量较仅零点计划净减少 626936.33 kWh，说明波动价格下日内更新同样主要用于撤回不再经济的承诺，同时通过选择性上调控制高价缺口。

在允许调整的模型内部，全部更新较仅 0 时预报节省 433183.45 元，即 2.74%。另设“提前知道未来真实电价、供需仍按因果预测”的信息对照：不允许和允许调整两种策略分别仅再节省 174674.23 元和 152912.24 元，比例为 1.12% 和 0.99%。这表明在本年度数据中，价格预测改进仍有价值，但收益小于储能和日内供需信息带来的改善。

然而，提前知道真实电价后，紧急购电量并不一定减少。这是因为模型可能选择在部分低价时段减少提前购电，而在必要时通过紧急购电补足需求。只要其他购电费用的下降幅度大于紧急购电费用的增加，总费用仍可能降低。因此，评价调度策略时应以最终总费用为主要指标，而不能只比较紧急购电量。

附录进一步给出同时已知未来供需与价格且放宽交易收费的全年下界，用于刻画理论改进空间。两组完整结果分别写入 result4-2.xlsx 和 result4-3.xlsx。以下保留题目要求的四个日期及全部指定时段；图5进一步比较四种主策略的费用组成与累计费用。

表 25　波动电价下的问题二：2025-03-20 购电结果（电量：kWh；费用：元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	632.45	14:00-14:10	0.00
16:00-16:10	520.26	18:00-18:10	0.00	20:00-20:10	0.00
全天计划购电量		67093.45	全天计划电费		42792.02

含紧急补电的实际总购电量 67605.92 kWh，实际总电费 44783.14 元。

表 26　波动电价下的问题二：2025-03-20 储能充放电与端点储电量（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	3805.70	2.46	04:00-08:00	1645.77	5087.86
08:00-12:00	4491.76	2777.38	12:00-16:00	4749.27	278.94
16:00-20:00	0.00	7016.93	20:00-24:00	876.11	1514.21
00:00 储电量		6601.65	24:00 储电量		2082.53

表 27　波动电价下的问题二：2025-06-21 购电结果（电量：kWh；费用：元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	284.04	18:00-18:10	517.50	20:00-20:10	0.00
全天计划购电量		53122.60	全天计划电费		25788.96

含紧急补电的实际总购电量 53123.97 kWh，实际总电费 25791.88 元。

表 28　波动电价下的问题二：2025-06-21 储能充放电与端点储电量（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	913.55	0.00	04:00-08:00	2468.75	1382.92
08:00-12:00	5384.69	0.00	12:00-16:00	1859.95	0.00
16:00-20:00	0.00	4821.73	20:00-24:00	7617.18	2485.90
00:00 储电量		2772.33	24:00 储电量		9535.88

表 29　波动电价下的问题二：2025-09-23 购电结果（电量：kWh；费用：元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	515.66	14:00-14:10	0.00
16:00-16:10	504.21	18:00-18:10	0.00	20:00-20:10	15.36
全天计划购电量	70395.79	全天计划电费	45803.17		

含紧急补电的实际总购电量 71102.67 kWh，实际总电费 49163.70 元。

表 30　波动电价下的问题二：2025-09-23 储能充放电与端点储电量（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4999.31	0.00	04:00-08:00	1784.61	5532.06
08:00-12:00	5464.78	1896.62	12:00-16:00	4206.87	295.28
16:00-20:00	10.05	5883.10	20:00-24:00	2305.56	2511.95
00:00 储电量	4502.00		24:00 储电量	3486.06	

表 31　波动电价下的问题二：2025-12-21 购电结果（电量：kWh；费用：元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	950.06	14:00-14:10	0.00
16:00-16:10	852.85	18:00-18:10	721.01	20:00-20:10	0.00
全天计划购电量	92733.00	全天计划电费	70272.31		

含紧急补电的实际总购电量 93419.88 kWh，实际总电费 73738.12 元。

表 32　波动电价下的问题二：2025-12-21 储能充放电与端点储电量（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	795.66	56.20	04:00-08:00	122.78	890.40
08:00-12:00	5339.70	7351.15	12:00-16:00	7509.89	2220.11
16:00-20:00	0.00	5647.94	20:00-24:00	7413.74	2855.56
00:00 储电量	10095.28		24:00 储电量	8024.03	

表 33　波动电价下的问题二：指定日期紧急购电时段与电量（kWh）

2025-03-20		2025-06-21		2025-09-23		2025-12-21	
时间段	电量	时间段	电量	时间段	电量	时间段	电量
00:20-00:30	28.44	01:20-01:30	1.38	01:40-01:50	3.11	01:10-01:20	22.42
01:40-02:00	14.29			05:00-05:10	6.60	02:00-02:20	13.60
02:30-02:50	8.48			06:10-06:30	55.69	03:00-03:20	73.08
03:00-03:50	85.56			06:40-07:00	43.03	03:30-04:00	104.38
04:50-05:20	32.21			07:40-07:50	22.80	04:10-04:20	2.49
05:50-06:00	9.24			14:30-14:50	118.13	05:00-05:20	4.17
15:20-16:20	163.70			15:00-15:20	82.72	05:40-06:20	50.23
16:30-17:00	16.10			15:50-16:50	296.34	07:50-08:00	21.62

表 33　波动电价下的问题二：指定日期紧急购电时段与电量（续）

2025-03-20		2025-06-21		2025-09-23		2025-12-21	
时间段	电量	时间段	电量	时间段	电量	时间段	电量
18:00-18:10	5.51			17:20-17:40	30.03	08:20-08:30	17.01
18:40-19:10	75.79			18:20-18:40	30.82	10:20-10:50	149.71
20:00-20:10	22.59			19:20-19:30	6.20	12:50-13:10	91.18
23:30-23:50	50.55			20:50-21:00	11.40	17:20-17:30	28.54
						18:30-19:00	34.29
						19:50-20:10	69.29
						22:50-23:00	4.86

表 34　波动电价下的问题三：2025-03-20 购电结果（电量：kWh；费用：元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00 / 0.00	12:00-12:10	541.79 / 526.47	14:00-14:10	210.87 / 0.00
16:00-16:10	663.16 / 536.02	18:00-18:10	0.00 / 0.00	20:00-20:10	0.00 / 0.00
全天计划购电量		65743.85	全天计划电费		42415.05

表内“计划 / 调整”分别为午夜承诺与最终承诺；调整后计划电量 61890.69 kWh。含紧急补电的实际总购电量 62638.74 kWh，实际总电费 44552.67 元。

表 35　波动电价下的问题三：2025-03-20 储能充放电与端点储电量（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	3805.70	2.46	04:00-08:00	845.67	5941.08
08:00-12:00	4422.46	2606.16	12:00-16:00	6117.63	176.57
16:00-20:00	126.65	6990.12	20:00-24:00	882.50	1487.03
00:00 储电量		6601.65	24:00 储电量		2067.30

表 36　波动电价下的问题三：2025-06-21 购电结果（电量：kWh；费用：元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00 / 0.00	12:00-12:10	0.00 / 0.00	14:00-14:10	0.00 / 0.00
16:00-16:10	342.55 / 269.74	18:00-18:10	555.19 / 536.81	20:00-20:10	0.00 / 0.00
全天计划购电量		49373.40	全天计划电费		25001.05

表内“计划 / 调整”分别为午夜承诺与最终承诺；调整后计划电量 48935.47 kWh。含紧急补电的实际总购电量 48935.47 kWh，实际总电费 24865.86 元。

表 37　波动电价下的问题三：2025-06-21 储能充放电与端点储电量（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	101.66	1.38	04:00-08:00	1706.05	1384.83
08:00-12:00	5359.33	0.00	12:00-16:00	3392.65	0.00
16:00-20:00	33.51	4797.71	20:00-24:00	7587.52	2485.90
00:00 储电量		2806.35	24:00 储电量		9535.88

表 38　波动电价下的问题三：2025-09-23 购电结果（电量：kWh；费用：元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00 / 0.00	12:00-12:10	414.77 / 414.77	14:00-14:10	0.00 / 0.00
16:00-16:10	583.87 / 565.77	18:00-18:10	0.00 / 0.00	20:00-20:10	15.36 / 15.36
全天计划购电量		73286.61	全天计划电费		48223.44

表内“计划 / 调整”分别为午夜承诺与最终承诺；调整后计划电量 70260.50 kWh。含紧急补电的实际总购电量 70397.23 kWh，实际总电费 47935.26 元。

表 39　波动电价下的问题三：2025-09-23 储能充放电与端点储电量（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	5004.27	0.00	04:00-08:00	1992.23	5862.47
08:00-12:00	5204.67	1896.62	12:00-16:00	4844.71	403.62
16:00-20:00	29.22	5894.43	20:00-24:00	2284.93	2511.95
00:00 储电量		4528.39	24:00 储电量		3542.31

表 40　波动电价下的问题三：2025-12-21 购电结果（电量：kWh；费用：元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00 / 0.00	12:00-12:10	990.38 / 990.38	14:00-14:10	0.00 / 0.00
16:00-16:10	878.45 / 878.45	18:00-18:10	721.01 / 721.01	20:00-20:10	0.00 / 0.00
全天计划购电量		95423.82	全天计划电费		72807.04

表内“计划 / 调整”分别为午夜承诺与最终承诺；调整后计划电量 93052.64 kWh。含紧急补电的实际总购电量 93606.81 kWh，实际总电费 74417.66 元。

表 41　波动电价下的问题三：2025-12-21 储能充放电与端点储电量（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	795.66	56.20	04:00-08:00	52.02	907.24
08:00-12:00	5120.72	7417.56	12:00-16:00	7896.45	2215.28
16:00-20:00	0.00	5647.94	20:00-24:00	7413.74	2855.56
00:00 储电量		10095.28	24:00 储电量		8024.03

表 42　波动电价下的问题三：指定日期紧急购电时段与电量（kWh）

2025-03-20		2025-06-21		2025-09-23		2025-12-21	
时间段	电量	时间段	电量	时间段	电量	时间段	电量
00:20-00:30	28.44	无	0.00	01:40-01:50	3.11	01:10-01:20	22.42
01:40-02:00	14.29			05:00-05:10	31.93	02:00-02:20	13.60
02:30-02:50	8.48			06:10-06:20	10.41	03:00-03:20	73.08
03:00-03:50	85.56			06:50-07:10	27.53	03:30-04:00	104.38
04:50-05:20	32.18			07:40-07:50	15.33	04:10-04:20	2.49
05:50-06:00	16.49			18:20-18:40	30.82	05:00-05:20	4.17

表 42　波动电价下的问题三：指定日期紧急购电时段与电量（续）

2025-03-20		2025-06-21		2025-09-23		2025-12-21	
时间段	电量	时间段	电量	时间段	电量	时间段	电量
08:10-08:20	35.90			19:20-19:30	6.20	05:40-06:00	33.30
08:50-09:10	90.11			20:50-21:00	11.40	07:50-08:00	10.09
09:50-10:00	45.21					08:20-08:30	21.33
14:10-14:30	38.84					08:40-09:00	24.35
14:40-14:50	1.77					09:30-09:50	6.28
15:20-16:20	162.55					10:30-10:50	96.87
16:50-17:00	1.84					13:50-14:00	4.82
18:00-18:20	8.28					17:20-17:30	28.54
18:40-19:10	77.76					18:30-19:00	34.29
20:00-20:20	49.78					19:50-20:10	69.29
23:30-23:50	50.55					22:50-23:00	4.86

在相同实际电价序列下，6 月 21 日允许调整策略的费用为 24865.86 元，较不调整策略减少 926.02 元，且当日未发生紧急购电。12 月 21 日允许调整后，紧急购电量由 686.88 kWh 降至 554.17 kWh，但实际电费增加 679.55 元。经济性评价需综合原计划费、调整费与紧急购电费，不能仅依据紧急购电量。

两类策略的日初储电量分别由此前连续运行状态决定，指定展示日期不重新设定统一初值。因此，日表用于解释实际运行特征，策略总体比较仍采用全期费用，并单独校正期初与期末储电量差异。

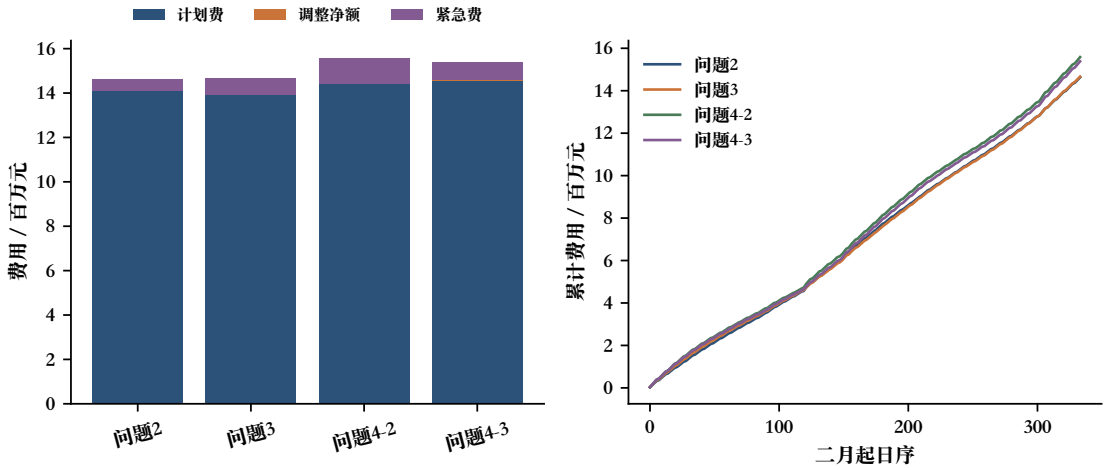


图 5　334 天内四种主策略的实际费用组成与累计费用

八、模型检验与结果分析

8.1　物理可行性检验

求解器得到最优解后，还需要检查结果是否满足能量平衡和储能运行约束。为此，对每个时段分别计算能量平衡误差和储电量变化误差：

$$r_t^{\text{bal}} = G_t + X_t + v_t + D_t - \ell_t - C_t - W_t,$$

$$r_t^{\text{soc}} = E_{t+1} - E_t - \eta C_t + \frac{D_t}{\eta}.$$

若模型运行正确，上述两个误差都应接近于 0。同时检查储电量是否处于规定范围内、充放电功率是否超过上限、是否出现同一时段同时充放电，以及紧急购电是否被用于电池充电。

检验结果表明，各方案均满足上述要求。所有结果中最大能量平衡误差为 $2.274 \times 10^{-13}\,\text{kWh}$，最大储电量变化误差为 $2.104 \times 10^{-12}\,\text{kWh}$，均可认为接近于 0；同时未出现储能越界、功率超限和同一时段同时充放电等情况。因此，模型计算结果满足实际运行的基本约束。

8.2 储电量差异与费用比较

不同策略在评价期结束时可能具有不同的剩余储电量。如果只比较已经支付的电费，可能会出现某个方案通过大量消耗原有储能而表现出较低费用的情况。

因此，在比较不同策略时，除报告实际费用外，进一步对期初和期末储电量差异进行统一校正：

$$\lambda = \frac{\min_t p_t^{(\text{附件 1})}}{0.9},$$

$$C_{\text{inv}} = C_{\text{act}} + \lambda(E_{\text{start}} - E_{\text{end}}).$$

其中，C_{act} 为实际运行费用，E_{start} 和 E_{end} 分别表示评价期开始和结束时的储电量。该校正值仅用于不同策略之间的公平比较，不计入题目要求的实际购电费用。

进一步对每日费用差进行重复抽样，计算平均节省费用的 95% 区间。结果表明，问题二主模型、问题三采用全部预报更新以及问题四允许日内调整三种改进方案，其平均节省费用的区间下限均大于 0，说明这些改进在本年度数据上均表现出较稳定的节费效果。

8.3 敏感性分析

为判断模型中的主要设置是否合理，分别改变部分条件并重新计算，包括去除储能、使用单一预测代替 12 种情景、改变情景数量、改变光伏预报更新时间以及调整滚动控制时间范围等。

结果表明，四种主策略相对各自无储能方案均节费 21.03%–22.74%，说明跨时段储能是节省开支的重要方法。问题二取消预测安全裕度后，总费用由 1463.26 万元上升至 1510.36 万元，表明仅围绕平均预测签订合同会显著增加高价紧急购电。

同时，对情景数量、控制时长和预报更新时间进行调整后，模型总体结论没有发生明显改变，说明所建立的购电与储能调度策略具有一定的稳定性。

此外，本文还对储能效率和购电调整费用的不同理解进行了重新计算。结果表明，这些参数和交易规则会对最终费用产生一定影响，因此在实际应用中应根据具体设备参数和交易规则重新确定模型参数。

主要比较结果汇总如下。全部 33 组实验的原始费用、紧急电量和储电量校正结果放入附录B。这些实验用于检查结论，不用正式 334 天的结果事后重选风险裕度的分位数参数。

表 43　二月至十二月实际运行汇总

策略	总费用/万元	紧急量/MWh	期初储电/kWh	期末储电/kWh
问题二	1463.26	81.99	9418.37	8738.40
问题三	1466.21	179.94	9109.33	8563.70
问题四-2	1558.41	243.83	7944.69	9258.51
问题四-3	1538.52	177.01	7944.69	9258.51

表 44　储能与预测裕度的费用对照（万元）

策略	无储能	简化预测/无裕度	本文策略
问题二	1859.39	1510.36	1463.26
问题三	1897.66	1624.09	1466.21
问题四-2	1973.43	1869.85	1558.41
问题四-3	1965.48	1719.17	1538.52

各列采用同一行对应的交易规则和实际数据；储电量差异的校正结果见附录。

表 45　平均每日节费及其估计区间

比较	平均日节省/元	95% 下限	95% 上限
问题二：80% 分位数模型相对 12 情境模型	575.80	72.64	1087.44
问题三：全部更新相对仅 0 时	1184.76	940.95	1439.37
问题四：可调整相对不调整	595.45	204.99	1049.18

正数表示节费，负数表示多花钱；三组区间下限均大于 0，支持本年度样本中存在正节费；区间不代表跨年度保证。

表 46　每次执行前对当前 10 分钟的预测误差（正式 334 日）

策略	净负荷 MAE/kWh	净负荷 RMSE/kWh	电价 MAE	电价 RMSE
问题二	46.2861	66.2495	0.0000	0.0000
问题三	47.5674	60.7167	0.0000	0.0000
问题四-2	58.4101	79.8666	0.0501	0.0674
问题四-3	47.5674	60.7167	0.0501	0.0674

电价误差单位为元/kWh；问题二记录午夜参考预测，其他策略记录当前区间实现前的控制层首步预测。

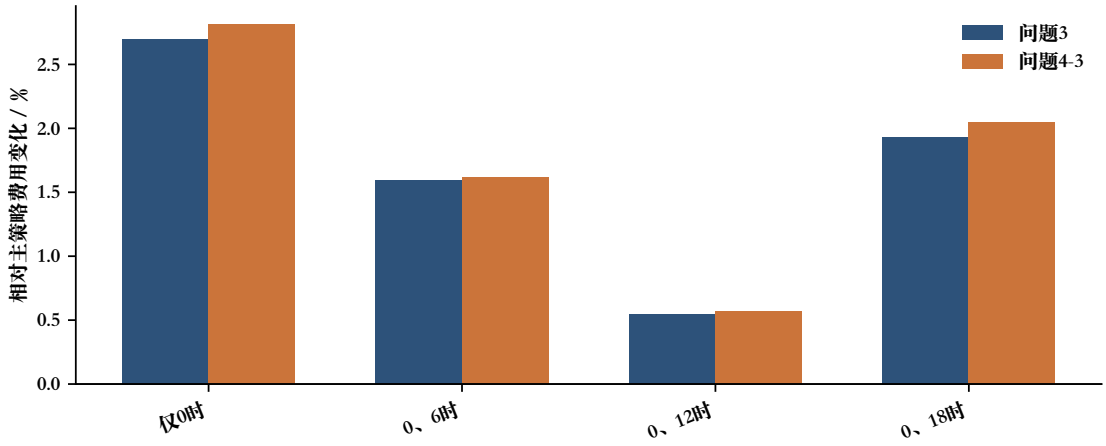


图 6 问题三及其波动价格版本中，不同预报更新组合相对完整更新的实际费用变化

九、结论与模型评价

9.1 主要结论

四问结果形成一条清晰主线：确定信息下，储能通过低价与光伏富余充电、高价放电获得主要节费；供需未知时，由 5 倍紧急价格导出的 80% 分位数在计划费和缺电风险间给出事前折中；日内预报到达后，调整主要撤回偏保守合同，同时以少量上调替代昂贵补购；价格波动下，评价标准仍是含计划、调整和紧急费用的实际总成本，而非单独追求最少紧急电量。全过程回放、控制实验和统计区间共同支持这些结论。

9.2 优点、限制与改进

模型以同一能量账本贯通四问，严格区分计划、参考轨迹和实际动作，并把因果预测、合同结算与设备约束落实到每个 10 分钟区间；五个工作簿和 33 组实验均可由程序独立复算。

局限主要有四点：问题二即时反馈按当前余缺贪心充放电，可能未把电量保留给未来更高价格或更大缺口；固定 80% 分位数在极端天气下未必合适；12 条历史情景不能覆盖全部尾部事件；10 分钟平均模型未考虑电池退化、线路容量和更短时标响应。后续可采用随季节和天气变化的分位数、带终端价值的自适应反馈、更丰富的情景缩减，并用多年度数据检验泛化能力。上述改进需要新增数据或设备参数，不能由本年度样本内回看替代。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于资料检索、模型讨论、代码辅助、结果核验和论文表述优化，详细使用情况见支撑材料。

参考文献

[1] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004. URL `https://web.stanford.edu/~boyd/cvxbook/`.

[2] Qi Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018. doi: 10.1007/s12532-017-0130-5.

[3] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, 3 edition, 2021. URL `https://otexts.com/fpp3/tscv.html`.

[4] Alexander Shapiro, Darinka Dentcheva, and Andrzej Ruszczynski. *Lectures on Stochastic Programming: Modeling and Theory*. Society for Industrial and Applied Mathematics, 2009. doi: 10.1137/1.9780898718751.

[5] James B. Rawlings, David Q. Mayne, and Moritz M. Diehl. *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, 2 edition, 2017. URL `https://sites.engineering.ucsb.edu/~jbraw/mpc/`.

[6] Alessandra Parisio, Evangelos Rikos, and Luigi Glielmo. Stochastic model predictive control for economic/environmental operation management of microgrids: An experimental case study. *Journal of Process Control*, 43:24–37, 2016. doi: 10.1016/j.jprocont.2016.04.008.

[7] Simone Mazzola, Claudio Vergara, Marco Astolfi, Vivian Li, Ignacio Perez-Arriaga, and Ennio Macchi. Assessing the value of forecast-based dispatch in the operation of off-grid rural microgrids. *Renewable Energy*, 108:116–125, 2017. doi: 10.1016/j.renene.2017.02.040.

[8] Jesus Lago, Grzegorz Marcjasz, Bart De Schutter, and Rafal Weron. Forecasting day-ahead electricity prices: A review of state-of-the-art algorithms, best practices and an open-access benchmark. *Applied Energy*, 293:116983, 2021. doi: 10.1016/j.apenergy.2021.116983.

附录 A：支撑材料文件说明

本页列出支撑包中的全部文件及用途。支撑包仅保留可运行建模代码、题目要求的结果文件与 AI 工具使用详情；题目原始附件不重复打包。全部文件均采用匿名名称，路径相对于支撑包根目录。

文件或目录	内容和用途
`README.md`	支撑材料文件列表、数据放置、环境与执行命令
`requirements.txt`	Python 依赖及固定版本
`run.py`	问题一至问题四、结果导出与核验的统一入口
`src/microgrid/`	数据、预测、优化、仿真、导出与核验的 9 个完整 Python 源文件
五个 `result*.xlsx`	题目指定的全部结果工作簿，位于支撑包根目录
AI 工具使用详情.pdf	AI 名称、用途、提示过程、采纳与核验范围

题目要求的结果文件：`result1.xlsx`、`result2.xlsx`、`result3.xlsx`、`result4-2.xlsx`、`result4-3.xlsx`。问题一结果覆盖 144 段；其余文件覆盖 2 月 1 日至 12 月 31 日 334 天，储能结果为实际动作与实际储电量。

复算顺序：按 README 将官方附件置于 `C 题/附件/`，安装依赖后依次执行 `q1`、`calibrate`、`select`、`main`、`export --results-only` 和 `verify --require-workbooks`。计算产生的逐时账本、校准记录与核验文件属于本地中间文件，不放入提交包。后续附录先给技术细节和完整实验表，再列全部自编程序。本轮 AI 差异须由队员据实复核。

附录 B：技术补充与复现细节

本附录说明正文模型如何转换为可重复执行的计算过程。公式中的计划购电量 G_t^0, G_t、充放电量 C_t, D_t、紧急购电量 X_t 及余电量 W_t 均为 10 分钟区间的交流侧电量，储电量记为 E_t；单向效率为 $\eta = 0.9$。完整程序紧接本附录列出，运行顺序与依赖版本见支撑材料 `README.md`。

B.1 问题二风险裕度—即时反馈模型的实现

问题二正式入口为 `simulate_quantile_q2`。每天只拟合一次预测权重：前一日、前一周和最近 7 日中位数构成三列基预测，最近 14 日内全部 10 分钟样本共同最小化净负荷绝对误差；负载权重和光伏权重分别非负且和为 1。历史不足 8 日时采用前一日曲线，首日采用附件 1。最近 30 日已揭晓净负荷残差用于计算 70%、80% 或 90% 分位数；逐段分位数与同小时池化分位数各取一半。次日预测以当日预测代替未揭晓的当日实际曲线，故 48 小时输入不含未来实测值。

48 小时稀疏线性规划以计划购电费为目标，固定起点真实储电量和末端 6000 kWh，仅提交前 144 段。日内每 10 分钟直接按当前实际余缺执行：余量为正时充到功率或容量边界，余量为负时先放到功率或储电下界，仍有缺口才记紧急购电；不调用另一个 MPC 函数。配置字段 `model=rolling_quantile`、`feedback=immediate_safe` 和 `control_hours=0` 共同标识该正式路径。

70%、90%、无分位数裕度和无储能方案均使用同一实现，只改变一个预先声明的参数。作为并列候选的 12 情境随机规划方案由 `scenario_reference` 独立运行，其预测审计与逐日结果置于 `q2_model_comparisons/`；完整逐时账本可由同一入口重建。

B.2 12 情境模型及问题三、四的预测与情景生成

26 维特征构造。 为说明特征的时间关系，令 D 为本次午夜预测的日序号，a 为目标日序号，$s = 0, \ldots, 143$ 为日内区间。把三个预测目标写为

$$z_{a,s} = (L_{a,s}, V_{a,s}, \log p_{a,s})^{\mathsf{T}}, \qquad A = \mathrm{diag}(5000, 5000, 1). \tag{24}$$

其中 L, V 单位为 kW，A 用于统一残差的数值尺度。朴素预测的负载、光伏基线见正文；对数价格基线依次以 $0.60, 0.30, 0.10$ 加权过去 28 天同星期均值、最近 7 天均值和附件 1 先验。历史不足时取已有天数；无同星期样本时改用最近样本；首日直接使用附件 1。

每个目标区间的输入 $x_{D,a,s}$ 含四组各 3 维的曲线特征：基线、起点前最近完整日、目标日的 7 天滞后、附件 1 先验，均左乘 A^{-1}。最近完整日固定为 $D-1$；只有 $0 \le a-7 < D$ 时才取实际 7 天滞后，否则使用先验。其余 14 维为

$$(\sin\theta_s, \cos\theta_s, \sin 2\theta_s, \cos 2\theta_s, \sin\phi_a, \cos\phi_a, a - D, \text{目标星期的 7 维独热编码}), \tag{25}$$

其中 $\theta_s = 2\pi(s+1)/144$，$\phi_a = 2\pi a/365.25$。独热编码是以一个 1 和六个 0 表示星期类别。预测次日时，最近日特征仍取 $D-1$，不会改取尚未结束的 D 日。固定电价问题在全部特征和残差计算中使用附件 1 日价格曲线；附件 4 实际价格仅进入问题四。

岭回归与梯度提升残差校正。 设训练目标为 $y_i = A^{-1}(z_i - \widehat{z}_i^{\text{base}})$，以 y_i^{T} 为行组成矩阵 Y，列向量 μ_Y 记录三个目标的训练均值。岭回归先用训练样本均值和标准差标准化 X，标准差小于 10^{-12} 的列以 1 替代除数，再求解

$$\widehat{W} = \arg\min_{W} \left\| Y - \mathbf{1}\mu_Y^{\mathsf{T}} - X_{\text{std}} W \right\|_F^2 + 30\|W\|_F^2, \tag{26}$$

$$\widehat{z} = \widehat{z}^{\text{base}} + A(\mu_Y + \widehat{W}^{\mathsf{T}} x_{\text{std}}). \tag{27}$$

截距不受惩罚。实现求解加上 $30I$ 的正规方程，并检查系数有限性。

提升树分别拟合三个残差目标。以某一目标为例，从训练均值 F_0 开始，用回归树逐轮拟合尚未解释的残差：

$$F_m(x) = F_{m-1}(x) + 0.08\, g_m(x), \qquad m = 1, \ldots, 60. \tag{28}$$

对于给定的树分区，记所拟合目标的残差为 y_i^*，叶值对应平方损失与 L2 惩罚 $\frac{1}{2}\sum_i \{y_i^* - F_{m-1}(x_i) - g_m(x_i)\}^2 + \frac{5}{2}\sum_j b_j^2$，其中 b_j 为叶节点的输出；分裂由直方图算法搜索。固定最多 15 个叶节点、每叶最少 36 个样本、L2 参数 5，随机种子 2026，关闭早停。树的输出同样乘回 A 并加到基线上。

滚动训练与预测缓存。 在第 D 日，训练样本的历史预测起点依次取 $\max(1, D-60), \ldots, D-1$，每个起点生成本日及次日目标，但仅保留目标日严格小于 D 的样本。这样，训练用到的实际结果均来自已经结束的完整日。岭回归保留全部 144 个日内点；提升树在拼接后的训练序列上每隔一个点取样，预测仍覆盖全部 144 个点。少于 7 天历史时，两种复杂模型均退回朴素基线。

基础模型每日拟合一次，并由同一午夜起点生成本日、次日及第三日共 432 步预测；第三日沿用同一模型和已知日历特征。这份缓存足以覆盖日内任一时刻发出的最长 48 小时查询。训练提前日数只有 0 和 1，因此跨到第三日的部分是模型外推。每 10 分钟控制更新实际库存，使用当前已发布的光伏预报；基础负载和价格模型不在日内反复重训。

官方预报融合与历史误差块。 每次只取允许发布时刻中最新且已经发布的一版光伏预报。其 24 个点对齐到发布时间后的第 1 至 24 小时，以最近已结束区间的实测光伏为插值锚点，形成 10 分钟预测；首个起点没有实测时使用附件 1 末点。超出该版预报覆盖范围后使用历史模型。消融实验中，未列入该实验的发布时刻对预测和购电调整同时不可用。

令当前绝对区间起点为 $r = 144D + s$，预测长度为 H。历史块起点取 $j = 144d + s$，其中

$$\max(0, D-60) \le d < D, \qquad j + H \le r. \tag{29}$$

后一条件保证块内 H 个实际区间全部结束。历史基线取第 d 日原始午夜生成的完整三日预测，再截取 s 起的 H 步；若使用官方预报，则只叠加在历史起点 j 已经发布的版

本。三条误差序列为负载和光伏的加性误差、价格的对数误差，按同一历史块配对，按正文情景公式转换为净负荷电量与正价格。

把合格块按时间排序，取等距索引 round(linspace$(0, M-1, K)$)，其中 M 为合格块数。12 情境模型及问题三、四取 $K = 12$，等权使用选中的块；$M < K$ 时允许重复，无合格块时使用零残差。点预测消融 $K = 1$ 单独规定为零残差。若 2 月启动的模型尚无自己的 1 月预测记录，就在各历史起点严格重建朴素预测，并标记为因果朴素初始化；不会用 2 月拟合的模型回填 1 月预测。

模型选择与数值保护。 12 情境模型及问题三、四的三个候选各自从 1 月 1 日 6000 kWh 开始假想运行，以 1 月 15 日至 31 日的库存校正费用选模。它们 1 月 15 日的库存可以不同，比较时保存各自期初、期末值。具体规则是在费用不超过候选最小值的 1.005 倍的模型中，依朴素、岭回归、提升树的顺序取最简单者。其正式 1 月始终按朴素策略预热，2 月继承该实际库存；候选实验不会改写这段运行。问题二风险裕度—即时反馈模型不使用这套选模结果。

负载和光伏预测截到非负区间。进入对数的价格设置 10^{-6} 下限；点预测和情景预测的对数价格在指数变换前截到 $[-9, 3]$，即价格约束于 $[1.23 \times 10^{-4}, 20.09]$ 元/kWh。此界远宽于附件 4 的正常取值范围，用于防止外推造成数值溢出。任何非有限预测或回归系数均触发异常，不以替换结果的方式继续运行。

B.3 参考末端、数值恢复与实际运行记录

末端库存可达域。 正文的短时域末端目标来自 48 小时参考轨迹。若实际库存偏离参考，原目标可能无法在剩余 H_M 步内达到，因此先将它投影到

$$[\max\{1200, E_r - H_M B/\eta\}, \min\{10800, E_r + H_M B\eta\}], \qquad B = 5000/6 \text{ kWh}. \tag{30}$$

区间两端分别对应持续最大功率放电和充电，考虑了容量、功率与效率。投影后作为本次参考优化的硬末端条件。它只表达设备的库存可达性；实际安全层还受当期余电和缺口限制，可能无法跟踪参考末端，下一步应从实际 E 重新规划。2 小时控制窗口更短，投影对避免功率限制导致的不可行尤为必要。本段只适用于 12 情境模型及问题三、四，不适用于问题二风险裕度—即时反馈模型。

求解结果数值复核。 稀疏线性规划调用 HiGHS，原始及对偶可行性容差均为 10^{-8}；混合整数核验的相对间隙设为 10^{-8}。程序要求求解状态为最优，并执行正文所述的同充同放消除，再由 C, D 重算整条 E、各情景 X, W 及目标值。恢复后的参考约束最大偏差不得超过 2×10^{-5}，重算目标与求解器目标的差不得超过 $10^{-4} + 10^{-7} \max\{1, |J_{\text{raw}}|\}$。这一步验证的是参考问题，实际电费仍由逐区间执行和结算产生。输入库存仅允许在边界外 10^{-7} 的舍入范围内截回容量界，更大越界直接报错。

运行记录内容。 工程中，每个策略的 `ledger.npz` 保存全年逐区间数组，其字段含义如下。所有电量字段以 kWh 计，价格为元/kWh，费用为元。

记录内容	字段与含义
交易和实际供需	plan 为 G^0，effective 为 G；load,pv,price 为结算区间的实际负载电量、光伏电量和价格。
储能和补缺	charge,discharge,emergency,waste 对应 C, D, X, W；soc_start,soc_end 为两端 E。
费用分项	base_fee,up_fee,down_fee,emergency_fee 记录原计划费、最终上调费、最终下调费、应急费；cost 为其和。
控制和预测摘要	情景模型中 target_charge,target_discharge 为安全层执行前的首步建议；问题二风险裕度—即时反馈模型中二者记录即时反馈实际值。expected_net,expected_price 为执行前参考预测；issue_index 为最近承诺更新时间。

revisions.json.gz 记录每次调整时刻及调整后的剩余日承诺向量，保存到小数点后 9 位。forecast_diagnostics.json 保存日级训练审计、调用次数及冷启动记录数；summary.json 保存策略配置和分评价窗口汇总。审计字段 training_actual_end_exclusive 是可用历史的排他上界：索引达到该值的区间尚不能用于训练。issue_index 也不等于每次预测所用的完整预报元数据。

提交包将五种策略的完整账本保存为 q1_ledger.npz 至 q4_3_ledger.npz。各数组按 10 分钟顺序排列，首项对应 2025 年 1 月 1 日 00:00–00:10；该格式保留全部字段并减少压缩包体积。提交包中的调整记录与预测审计也加相同策略前缀并以 gzip 压缩，例如 q2_revisions.json.gz 与 q2_forecast_diagnostics.json.gz。

在线检查与独立复核。 预测返回时检查历史误差块结束位置不晚于当前起点、预报发布时间不晚于当前时刻。执行时即时检查库存边界、能量平衡及充放电与应急充电重叠；独立验证程序再从实际账本重算守恒、库存连续、功率界、安全层动作和各费用分项，物理容差为 10^{-6} kWh。工作簿另按原模板日期、时段及单元格逐项核对，连续应急区间先展开再与账本比较。

程序未保存全年每次预测的完整情景及全部信息元数据，也未持久化每次线性规划的完整目标记录。因此，信息边界结论由实现约束、即时断言、日级审计和测试共同支持；保存的账本足以重算实际物理量与电费，不能单独复原所有中间预测。真实组件的 3 日端到端测试将同一历史前缀后的负载、光伏、价格及尚未发布预报共同扰动，检查此前的计划、储能建议、实际库存和费用不变。指定日期的 LP/MILP 对照则是使用已实现净负荷的事后数值核验，不参与过去的运行决策。

B.4 统计口径与信息下界

预测评价指标口径。 午夜 24 小时预测对正式期 334 天的全部日内点计算负载、光伏及价格 MAE；问题二账本保存午夜联合预测，情景模型的每 10 分钟首步预测由账本中的情景均值计算净负荷电量及价格 MAE、RMSE。附件 3 原始预报按发布时间与提前 1 至 24 小时分别统计，只使用实际光伏大于 100 kW 且存在实测的白天目标。后者剔除跨入 2026 年的无实测点。三者的提前量、样本与单位不同，图中的白天预报误差不能代

替整日能量误差；光伏大量零值也使全时段 MAPE 不适合作为主要指标。

库存校正与配对区间估计。 评价费用 C 之外，以固定 $\lambda = \min p^{(\text{附件 1})}/0.9$ 计算 $C^{\text{inv}} = C + \lambda(E_{\text{start}} - E_{\text{end}})$。两端库存同时计入，适用于正式期和 1 月候选比较；该校正不进入题目要求的实际电费。比较两种策略的日费用时，先按相同日期求差 $\delta_d = C_d^{\text{前者}} - C_d^{\text{后者}}$，再做循环区块重采样：每次均匀抽取 $\lceil 334/7 \rceil = 48$ 个起点，各带连续 7 日并在序列末尾循环，拼接后截到 334 日。固定种子 2026，重复 2000 次，取样本日均差的 2.5% 和 97.5% 分位数。正值表示后者节省，负值表示后者更贵。循环处理和单年季节性限制了区间的外推能力，该区间用于描述本年差异。

未来信息对照实验。 仅价格全知实验用真实未来价格替换评价年内的预测价格，净负荷预测、参考结构及安全层保持原配置；超出 2025 年的参考部分仍使用预测价。它检验价格信息在这套近似策略中的作用，费用不必单调下降。

完全信息下界使用全年 365 天真实净负荷与相应真实价格，所有购电按基础价计费，初始 $E = 6000$，末端只受容量界约束。令其购电变量为 g_t，保留相同储能物理约束并满足 $g_t + v_t + D_t \geq \ell_t + C_t$。任一主结算实际策略都能取 $g_t = G_t + X_t$ 形成可行解，且

$$C - \sum_t p_t(G_t + X_t) = \sum_t p_t \left[\tfrac{1}{2}(G_t - G_t^0)^+ + \tfrac{1}{2}(G_t^0 - G_t)^+ + 4X_t\right] \geq 0. \tag{31}$$

因此该放松规划的最优值不超过相同全年范围的主策略费用。差额同时包含未来信息、交易限制和控制近似的影响；它不能与 334 日费用直接相减，也不能全部视为可实现的节省。

B.5 校准、预测诊断与完整实验表

下表保留候选校准和各变体的原始数值。33 组实验包括问题二的无储能、无分位数裕度、70%、90% 和 12 情境模型 5 组，问题三与问题四的点预测、储能、更新时点、情景数、控制时域、效率与结算解释等 28 组。参数变体分别运行并按自己的效率及结算条款核验；主参数和事前规则不因正式期结果而改变。

表 47 情景滚动模型一月后半月预测方法选择（储电量校正后的费用，元）

策略	季节朴素	岭回归	梯度提升	入选方法
问题二 12 情境模型	993038.99	1021829.62	1007275.41	季节朴素
问题三	974841.35	994073.34	973691.02	季节朴素
问题四-2	1146794.67	1199761.83	1177412.12	季节朴素
问题四-3	1107397.57	1139949.50	1121541.43	季节朴素

表 48　午夜对当天 24 小时的平均绝对预测误差

策略	负荷/kW	光伏/kW	电价/(元/kWh)
问题二	181.34	156.59	0.000000
问题三	282.38	197.49	0.000000
问题四-2	282.38	167.60	0.050916
问题四-3	282.38	197.49	0.050916

表 49　33 组对照与参数检验完整结果（二月至十二月）

策略变体	实际费用/万元	紧急量/MWh	库存校正费用/万元
问题二：无分位数裕度	1510.36	315.31	1510.37
问题二：无储能	1859.39	258.47	1859.39
问题二：70% 分位数	1456.17	136.19	1456.19
问题二：90% 分位数	1495.52	36.52	1495.55
问题二：12 情境随机规划模型	1482.49	253.92	1482.52
问题三：原计划全额另加违约费	1484.12	165.99	1484.14
问题三：2 小时	1468.53	178.81	1468.54
问题三：6 小时	1465.22	180.46	1465.26
问题三：仅 0 时	1505.78	243.13	1505.81
问题三：0、12 时	1474.24	197.08	1474.26
问题三：0、18 时	1494.55	217.10	1494.57
问题三：0、6 时	1489.54	198.05	1489.56
问题三：24 情景	1462.12	171.67	1462.15
问题三：6 情景	1483.99	223.53	1484.00
问题三：无储能	1897.66	257.66	1897.66
问题三：点预测	1624.09	833.48	1624.09
问题三：往返效率 90%	1419.29	177.16	1419.33
问题四-2：价格全知	1540.94	267.45	1540.93
问题四-2：无储能	1973.43	319.20	1973.43
问题四-2：点预测	1869.85	1192.80	1869.82
问题四-3：原计划全额另加违约费	1557.73	163.61	1557.67
问题四-3：2 小时	1539.54	176.11	1539.48
问题四-3：6 小时	1538.07	177.72	1538.02
问题四-3：仅 0 时	1581.84	243.03	1581.78
问题四-3：0、12 时	1547.26	194.42	1547.20
问题四-3：0、18 时	1570.00	216.90	1569.95
问题四-3：0、6 时	1563.37	196.79	1563.32
问题四-3：24 情景	1529.55	169.15	1529.54
问题四-3：6 情景	1562.38	219.05	1562.33
问题四-3：价格全知	1523.23	192.97	1523.22

表 49　33 组对照与参数检验完整结果（续）

策略变体	实际费用/万元	紧急量/MWh	库存校正费用/万元
问题四-3：无储能	1965.48	256.10	1965.48
问题四-3：点预测	1719.17	849.87	1719.15
问题四-3：往返效率 90%	1491.10	172.31	1491.04

逐时独立验证的最大能量平衡误差为 2.274×10^{-13} kWh；问题一 LP 与 MILP 费用差为 0.000000000 元。全年 365 日完全信息松弛下界：固定电价 13751266.38 元，波动电价 14494771.23 元。该下界允许年末自由储电，与 334 日表内费用不能直接相减。对应 365 日实际费用：问题二 16757631.12 元，问题三 16684412.30 元，波动电价下分别为 18076656.90 元、17650372.98 元。差额同时包含信息、交易限制和策略近似的影响。

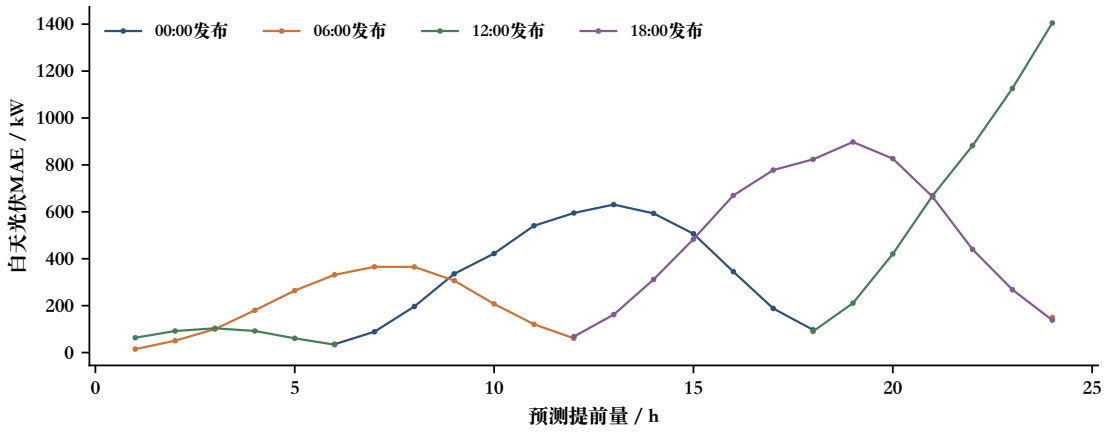


图 7　附件 3 光伏预报的平均绝对误差随提前小时数变化：正式 334 日，仅取实际光伏大于 100 kW 的时段

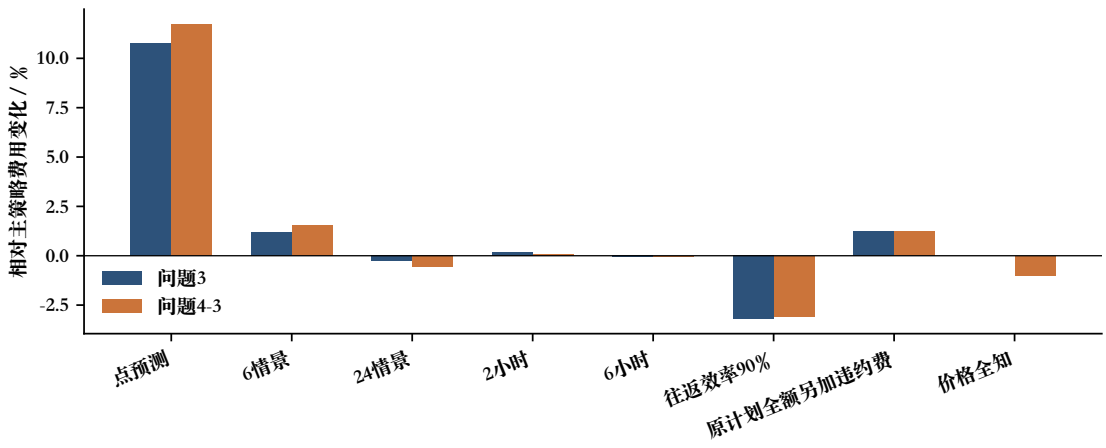


图 8　情景数、控制范围及费用解释变化时，实际费用相对主策略的变化

附录 C：完整程序代码

本附录列示与支撑材料同名的完整自编程序。依赖环境、输入放置方式及运行顺序见支撑材料 README.md。

run.py

```
#!/usr/bin/env python3
"""Reproduce calculations: .venv/bin/python run.py --help."""
import os
for key in ('OMP_NUM_THREADS','OPENBLAS_NUM_THREADS','MKL_NUM_THREADS','VECLIB_MAXIMUM_THREADS'):
    os.environ.setdefault(key,'1')
from pathlib import Path
import argparse
import json
import sys
from dataclasses import replace
sys.path.insert(0,str(Path(__file__).resolve().parent/'src'))
from microgrid.data import load_data
from microgrid.simulate import Policy, base_policies, simulate, solve_question_one,
    perfect_information_bound, write_json
from microgrid.quantile import simulate_quantile_q2


def main():
    ap=argparse.ArgumentParser(description=__doc__)
    ap.add_argument('stage',choices=['q1','calibrate','select','main','experiment','bound','export','verify'
    ,'all'])
    ap.add_argument('--policy',default='all',choices=['all','q2','q3','q4_2','q4_3'])
    ap.add_argument('--family',default='all',choices=['all','naive','ridge','boost'])
    ap.add_argument('--days',type=int,default=365)
    ap.add_argument('--experiment',default='point',choices=['point','no_storage','no_margin','q70','q90','
    scenario_reference','issue0','issue06','issue012','issue018','k6','k24','h2','h6','roundtrip90','
    full_plan_plus_penalty','known_price'])
    ap.add_argument('--quantile',type=float,choices=[.7,.8,.9],default=.8,
                    help='Q2 primary rolling residual quantile (default: 0.80)')
    ap.add_argument('--out',default='artifacts/runs')
    ap.add_argument('--require-workbooks',action='store_true')
    ap.add_argument('--paper-only',action='store_true',help='refresh paper tables and figures while
    retaining result workbooks and ledgers')
    ap.add_argument('--results-only',action='store_true',help='export only the five required result
    workbooks')
    args=ap.parse_args()
    if args.paper_only and args.stage!='export':ap.error('--paper-only is only available with export')
    if args.results_only and args.stage!='export':ap.error('--results-only is only available with export')
    if args.paper_only and args.results_only:ap.error('--paper-only and --results-only cannot be combined')
    if args.stage=='all':
        if args.out!='artifacts/runs' or args.days!=365 or args.policy!='all':
            ap.error('all requires the default output directory, all policies, and 365 days')
        import subprocess
        subprocess.run([sys.executable,str(Path(__file__).resolve().parent/'reproduce.py')],check=True)
        return
    data=load_data()
    selection_file=Path('artifacts/selection.json')
    selected=json.loads(selection_file.read_text())['selected'] if selection_file.exists() else {}
    if args.stage in ('q1','all'):
        print(solve_question_one(data,Path(args.out)/'q1'),flush=True)
    if args.stage=='calibrate':
```

```
        families=['naive','ridge','boost'] if args.family=='all' else [args.family]
        for family in families:
            for policy in base_policies():
                if args.policy!='all' and policy.name!=args.policy: continue
                p=replace(policy,name=policy.name+'_'+family,family=family,warmup_naive=False)
                simulate(p,data,end_day=31,output_dir='artifacts/calibration')
    if args.stage=='select':
        scores={}; chosen={}
        for policy in base_policies():
            vals={}
            for family in ('naive','ridge','boost'):
                file=Path('artifacts/calibration')/(policy.name+'_'+family)/'summary.json'
                vals[family]=json.loads(file.read_text())['january_validation']['inventory_corrected_cost']
            minimum=min(vals.values())
            choice=next(f for f in ('naive','ridge','boost') if vals[f]<=minimum*1.005)
            scores[policy.name]=vals; chosen[policy.name]=choice
        write_json(selection_file,{'selected':chosen,'validation_inventory_cost':scores,
                                   'rule':'Jan15-31 chronological cost, choose simplest within 0.5%;
     operational January remains naive.'})
        print(chosen)
    if args.stage in ('main','all'):
        if not selected: raise RuntimeError('Run calibration and selection first.')
        for policy in base_policies(selected):
            if args.policy!='all' and policy.name!=args.policy: continue
            if policy.name=='q2':
                simulate_quantile_q2(args.quantile,data,end_day=args.days,
                                     output_dir=args.out,name='q2')
            else:
                simulate(policy,data,end_day=args.days,output_dir=args.out)
    if args.stage=='experiment':
        names=[args.policy] if args.policy!='all' else ['q2','q3','q4_2','q4_3']
        for policy in base_policies(selected):
            if policy.name not in names: continue
            name=policy.name+'_'+args.experiment
            if policy.name=='q2':
                if args.experiment in ('q70','q90','no_margin','no_storage'):
                    q={'q70':.7,'q90':.9}.get(args.experiment,.8)
                    simulate_quantile_q2(q,data,end_day=args.days,
                                         output_dir='artifacts/experiments',name=name,
                                         use_margin=args.experiment!='no_margin',
                                         storage=args.experiment!='no_storage')
                    continue
                if args.experiment=='point':
                    simulate_quantile_q2(.8,data,end_day=args.days,
                                         output_dir='artifacts/experiments',name=name,
                                         use_margin=False)
                    continue
                if args.experiment=='scenario_reference':
                    simulate(replace(policy,name=name),data,end_day=args.days,
                             output_dir='artifacts/experiments')
                    continue
                raise ValueError('Q2 experiment is not applicable to the primary quantile model')
            changes={'point':{'scenarios':1}, 'no_storage':{'storage':False},
                     'issue0':{'updates':()}, 'issue06':{'updates':(6,)},
                     'issue012':{'updates':(12,)}, 'issue018':{'updates':(18,)},
                     'k6':{'scenarios':6},'k24':{'scenarios':24},
                     'h2':{'control_hours':2},'h6':{'control_hours':6},
                     'roundtrip90':{'eta':.9**.5},
                     'full_plan_plus_penalty':{'settlement':'full_plan_plus_penalty'},
```

```
                    'known_price':{'perfect_price':True}}
            p=replace(policy,name=name,**changes[args.experiment])
            simulate(p,data,end_day=args.days,output_dir='artifacts/experiments')
    if args.stage=='bound':
        print(perfect_information_bound(data,Path('artifacts/perfect_bound.json')),flush=True)
    if args.stage in ('export','all'):
        from microgrid.export import export_all
        export_all(paper_only=args.paper_only,results_only=args.results_only)
    if args.stage in ('verify','all'):
        from microgrid.verify import verify_all
        print(verify_all(require_workbooks=args.require_workbooks),flush=True)


if __name__=='__main__':
    main()
```

src/microgrid/data.py

```
"""Read the supplied workbooks without changing their values or templates."""
from __future__ import annotations

from dataclasses import dataclass
from datetime import datetime, time
from pathlib import Path
from typing import Any

import numpy as np
import openpyxl


def canonical_intervals() -> list[str]:
    """The model uses [00:00,00:10), ..., [23:50,24:00)."""
    def clock(minutes: int) -> str:
        return f"{minutes // 60:02d}:{minutes % 60:02d}"
    return [f"{clock(10*i)}-{clock(10*(i+1))}" for i in range(144)]


def parse_time_minutes(value: Any) -> int:
    if isinstance(value, (datetime, time)):
        return value.hour * 60 + value.minute
    text = str(value).strip()
    carry = 1440 if "+1" in text else 0
    text = text.replace("+1", "")
    hour, minute = text.split(":")[:2]
    return int(hour) * 60 + int(minute) + carry


@dataclass
class DataBundle:
    dates: np.ndarray
    load: np.ndarray
    pv: np.ndarray
    price: np.ndarray
    prior_load: np.ndarray
    prior_pv: np.ndarray
    prior_price: np.ndarray
    forecasts: np.ndarray
    source_dir: Path | None = None
```

```
41
42      @property
43      def flat_load(self) -> np.ndarray:
44          return self.load.reshape(-1)
45
46      @property
47      def flat_pv(self) -> np.ndarray:
48          return self.pv.reshape(-1)
49
50      @property
51      def flat_price(self) -> np.ndarray:
52          return self.price.reshape(-1)
53
54      @property
55      def n_days(self) -> int:
56          return len(self.dates)
57
58      @property
59      def n_steps(self) -> int:
60          return self.n_days * 144
61
62      @property
63      def start_date(self) -> np.datetime64:
64          return self.dates[0].astype("datetime64[D]")
65
66      def audit(self) -> dict[str, Any]:
67          return {
68              "days": self.n_days,
69              "start": str(self.dates[0]), "end": str(self.dates[-1]),
70              "steps_per_day": 144, "step_hours": 1 / 6,
71              "input_timestamp_convention": "right endpoint of preceding 10-minute interval",
72              "output_interval_convention": "00:00-00:10 through 23:50-24:00",
73              "template_offset_minutes_corrected": 10,
74              "load_min_kw": float(self.load.min()), "load_max_kw": float(self.load.max()),
75              "pv_min_kw": float(self.pv.min()), "pv_max_kw": float(self.pv.max()),
76              "price_min": float(self.price.min()), "price_max": float(self.price.max()),
77              "load_energy_kwh": float(self.load.sum() / 6),
78              "pv_energy_kwh": float(self.pv.sum() / 6),
79              "output_days": int(np.sum(self.dates >= np.datetime64("2025-02-01"))),
80          }
81
82
83  def _rows(path: Path, sheet: str | None = None) -> list[tuple]:
84      book = openpyxl.load_workbook(path, read_only=True, data_only=True)
85      try:
86          return list((book[sheet] if sheet else book.active).values)
87      finally:
88          book.close()
89
90
91  def load_data(path: str | Path = "C题/附件") -> DataBundle:
92      path = Path(path)
93      if not (path / "附件1.xlsx").is_file():
94          candidate = path / "C题" / "附件"
95          if candidate.is_dir():
96              path = candidate
97      prior_rows = _rows(path / "附件1.xlsx")
98      times = [parse_time_minutes(row[0]) for row in prior_rows[1:]]
99      if times != list(range(10, 1441, 10)):
```

```
        raise ValueError("附件1时间点不是00:10至24:00的144个连续时点")
    prior = np.asarray([row[1:4] for row in prior_rows[1:]], dtype=float)
    load_rows = _rows(path / "附件2.xlsx", "小区负载")
    pv_rows = _rows(path / "附件2.xlsx", "光伏发电实际功率")
    price_rows = _rows(path / "附件4.xlsx")
    dates = np.asarray([np.datetime64(row[0], "D") for row in load_rows[1:]])
    arrays = []
    for rows in (load_rows, pv_rows, price_rows):
        matrix_dates = np.asarray([np.datetime64(row[0], "D") for row in rows[1:]])
        if not np.array_equal(matrix_dates, dates):
            raise ValueError("附件2与附件4日期不一致")
        if [parse_time_minutes(v) for v in rows[0][1:]] != times:
            raise ValueError("输入工作表时间轴不一致")
        arrays.append(np.asarray([row[1:] for row in rows[1:]], dtype=float))
    if len(np.unique(dates)) != len(dates) or np.any(np.diff(dates).astype(int) != 1):
        raise ValueError("输入日期存在重复或缺失")
    predictions = np.full((len(dates), 4, 24), np.nan)
    date_map = {str(date): i for i, date in enumerate(dates)}
    issue_date = None
    for row in _rows(path / "附件3.xlsx")[1:]:
        if row[0]:
            issue_date = np.datetime64(datetime.strptime(str(row[0]), "%Y-%m-%d"), "D")
        if issue_date is None:
            raise ValueError("附件3首条预报缺少日期")
        minute = parse_time_minutes(row[1])
        if minute not in (0, 360, 720, 1080):
            raise ValueError("附件3存在非规定发布时刻")
        day = date_map[str(issue_date)]
        issue = minute // 360
        if np.isfinite(predictions[day, issue]).any():
            raise ValueError("附件3存在重复预报")
        predictions[day, issue] = row[2:26]
    for array in (*arrays, prior, predictions):
        if not np.isfinite(array).all() or np.any(array < 0):
            raise ValueError("输入存在缺失、非有限或负数")
    if arrays[0].shape != (len(dates), 144) or np.any(arrays[2] <= 0):
        raise ValueError("输入形状错误或电价非正")
    return DataBundle(dates, arrays[0], arrays[1], arrays[2],
                      prior[:, 1], prior[:, 2], prior[:, 0], predictions, path)


def template_manifest(path: str | Path = "C题/附件/附件5") -> dict[str, Any]:
    """Report the original template labels and their documented correction."""
    result: dict[str, Any] = {
        "model_intervals": canonical_intervals(),
        "original_purchase_intervals_shifted_minutes": 10,
        "correction": "Keep the 144 data positions; label their preceding intervals, beginning at 00:00.",
        "files": {},
    }
    for file in sorted(Path(path).glob("*.xlsx")):
        book = openpyxl.load_workbook(file, data_only=True)
        result["files"][file.name] = {
            sheet.title: {
                "rows": sheet.max_row, "columns": sheet.max_column,
                "merged_cells": [str(v) for v in sheet.merged_cells.ranges],
                "header": [str(cell.value) if cell.value is not None else None for cell in sheet[1]],
            }
            for sheet in book
        }
```

```
        book.close()
    return result
```

src/microgrid/export.py

```
"""Create the five required template workbooks and every paper number from ledgers."""
from pathlib import Path
import csv
import json
import math
import gzip
import hashlib
import datetime as dt
import numpy as np
import pandas as pd
from openpyxl import load_workbook
from openpyxl.styles import Font, Alignment
from openpyxl.utils import get_column_letter
from .data import load_data
from .simulate import STEPS, write_json

ROOT = Path(__file__).resolve().parents[2]
SELECTED_DATES = ['2025-03-20','2025-06-21','2025-09-23','2025-12-21']
CHINESE = {'q2':'问题二','q3':'问题三','q4_2':'波动电价下的问题二','q4_3':'波动电价下的问题三'}
DISPLAY_NAMES = {'q2':'问题二','q3':'问题三','q4_2':'问题四-2','q4_3':'问题四-3'}
OUTPUT_NAMES = {'q1':'result1.xlsx','q2':'result2.xlsx','q3':'result3.xlsx',
                'q4_2':'result4-2.xlsx','q4_3':'result4-3.xlsx'}
VARIANT_LABELS = {'issue0':'仅0时','issue06':'0、6时','issue012':'0、12时','issue018':'0、18时',
                  'point':'点预测','no_storage':'无储能','k6':'6情景','k24':'24情景',
                  'h2':'2小时','h6':'6小时','roundtrip90':r'往返效率90\%',
                  'full_plan_plus_penalty':'原计划全额另加违约费','known_price':'价格全知',
                  'no_margin':'无分位数裕度','q70':r'70\%分位数','q90':r'90\%分位数',
                  'scenario_reference':'12情境随机规划模型'}


def clock(slot):
    return f'{slot//6:02d}:{slot%6*10:02d}'


def period(start,end):
    return clock(start)+'–'+clock(end)


def events(values, eps=1e-7):
    on=np.asarray(values)>eps
    starts=np.flatnonzero(on & ~np.r_[False,on[:-1]])
    ends=np.flatnonzero(on & ~np.r_[on[1:],False])+1
    return [(int(a),int(b),float(values[a:b].sum())) for a,b in zip(starts,ends)]


def fetch(name, folder='artifacts/runs'):
    base=ROOT/folder/name
    with np.load(base/'ledger.npz') as archive:
        log={k:archive[k].copy() for k in archive.files}
    return log,json.loads((base/'summary.json').read_text())


def numeric_style(sheet, first_col=2):
```

```
    sheet.freeze_panes='B2' if sheet.max_column>20 else 'A2'
    sheet.row_dimensions[1].height=30
    for cell in sheet[1]:
        cell.font=Font(name='Arial',size=10,bold=True)
        cell.alignment=Alignment(horizontal='center',vertical='center',wrap_text=True)
    for row in sheet.iter_rows(min_row=2):
        for cell in row:
            cell.font=Font(name='Arial',size=10)
            cell.alignment=Alignment(horizontal='right' if isinstance(cell.value,(float,int)) else 'left',
 vertical='center')
            if isinstance(cell.value,(float,int)):
                cell.number_format='0.0000'
    sheet.column_dimensions['A'].width=14


def export_workbook(name, log, out):
    template=ROOT/'C题/附件/附件5'/OUTPUT_NAMES[name]
    book=load_workbook(template)
    book.properties.creator=''; book.properties.lastModifiedBy=''
    book.properties.title=CHINESE.get(name,'问题一')+'计算结果'
    plan=book['计划购电量']
    if name=='q1':
        for s in range(144):
            plan.cell(s+2,1,period(s,s+1)); plan.cell(s+2,2,float(log['plan'][s]))
        plan.column_dimensions['A'].width=22; plan.column_dimensions['B'].width=20
        storage=book['充放电量']
        for block in range(6):
            a,b=block*24,(block+1)*24
            storage.cell(block+2,1,period(a,b))
            storage.cell(block+2,2,float(log['charge'][a:b].sum()))
            storage.cell(block+2,3,float(log['discharge'][a:b].sum()))
        storage.cell(2,4,'00:00'); storage.cell(2,5,float(log['soc'][0]))
        storage.cell(3,4,'24:00'); storage.cell(3,5,float(log['soc'][-1]))
    else:
        for label,key in [('计划购电量','plan'),('调整购电量','effective')]:
            if label not in book.sheetnames: continue
            ws=book[label]
            for slot in range(144): ws.cell(1,slot+2,period(slot,slot+1))
            for day in range(31,365):
                row=day-29; a,b=day*144,(day+1)*144
                ws.cell(row,1,dt.datetime(2025,1,1)+dt.timedelta(days=day))
                ws.cell(row,1).number_format='yyyy-mm-dd'
                for slot,value in enumerate(log[key][a:b]): ws.cell(row,slot+2,float(value))
                ws.cell(row,146,float(log[key][a:b].sum()))
                cost=log['base_fee'][a:b].sum()
                if label=='调整购电量': cost+=log['up_fee'][a:b].sum()+log['down_fee'][a:b].sum()
                ws.cell(row,147,float(cost))
            for col in range(2,146): ws.column_dimensions[get_column_letter(col)].width=16
            ws.column_dimensions['EP'].width=20; ws.column_dimensions['EQ'].width=20
        storage=book['充放电量']; storage.delete_rows(2,storage.max_row)
        for day in range(31,365):
            date=dt.datetime(2025,1,1)+dt.timedelta(days=day)
            for block in range(6):
                a=day*144+block*24; b=a+24
                row=[date if block==0 else None,period(block*24,(block+1)*24),
                     float(log['charge'][a:b].sum()),float(log['discharge'][a:b].sum()),
                     '00:00' if block==0 else '24:00' if block==1 else None,
                     float(log['soc_start'][day*144]) if block==0 else float(log['soc_end'][(day+1)*144-1])
 if block==1 else None]
```

```
                storage.append(row)
                if block==0: storage.cell(storage.max_row,1).number_format='yyyy-mm-dd'
        urgent=book['紧急购电量']; urgent.delete_rows(2,urgent.max_row)
        for day in range(31,365):
            date=dt.datetime(2025,1,1)+dt.timedelta(days=day)
            entries=events(log['emergency'][day*144:(day+1)*144])
            if not entries: entries=[(None,None,0.)]
            for j,(start,end,value) in enumerate(entries):
                urgent.append([date if j==0 else None,'无' if start is None else period(start,end),value])
                if j==0: urgent.cell(urgent.max_row,1).number_format='yyyy-mm-dd'
        urgent.column_dimensions['B'].width=23; urgent.column_dimensions['C'].width=23
    for ws in book:
        numeric_style(ws)
        if ws.max_column<=6:
            for col in range(2,ws.max_column+1): ws.column_dimensions[get_column_letter(col)].width=20
    book.save(out/OUTPUT_NAMES[name])


def f(value, decimals=2):
    return format(float(value),f'.{decimals}f')


def latex_sci(value, decimals=3):
    value=float(value)
    if value == 0:
        return '$0$'
    mantissa,exponent=format(value,f'.{decimals}e').split('e')
    return '$'+mantissa+r'\times10^{'+str(int(exponent))+'}$'


def day_tables(name,log):
    lines=[]
    days=[int((np.datetime64(date)-np.datetime64('2025-01-01'))/np.timedelta64(1,'D')) for date in
     SELECTED_DATES]
    is_adjust='effective' in log and name in ('q3','q4_3')
    for date,day in zip(SELECTED_DATES,days):
        a,b=day*144,(day+1)*144
        lines += [r'\begin{table}[H]\centering\small\setlength{\tabcolsep}{5pt}',
                  rf'\caption{{{CHINESE[name]}：{date}购电结果（电量：kWh；费用：元）}}',
                  r'\begin{tabular}{lrlrlr}\toprule',
                  '时间段 & 购电量 & 时间段 & 购电量 & 时间段 & 购电量 '+r'\\\midrule']
        for hours in ((10,12,14),(16,18,20)):
            row=[]
            for h in hours:
                s=h*6; value=f(log['plan'][a+s])
                if is_adjust: value+=' / '+f(log['effective'][a+s])
                row += [period(s,s+1),value]
            lines.append(' & '.join(row)+r'\\')
        lines.append(r'\midrule 全天计划购电量 & \multicolumn{2}{r}{'+f(log['plan'][a:b].sum())+
                     r'} & 全天计划电费 & \multicolumn{2}{r}{'+f(log['base_fee'][a:b].sum())+r'}\\\
    bottomrule')
        lines += [r'\end{tabular}',r'\par\vspace{2pt}\footnotesize']
        if is_adjust:
            lines.append('表内“计划 / 调整”分别为午夜承诺与最终承诺；调整后计划电量'+f(log['effective'][a:b
    ].sum())+' kWh。')
        lines.append('含紧急补电的实际总购电量'+f((log['effective'][a:b]+log['emergency'][a:b]).sum())+
                     ' kWh，实际总电费'+f(log['cost'][a:b].sum())+'元。')
        lines += [r'\vspace{3pt}\small',rf'\caption{{{CHINESE[name]}：{date}储能充放电与端点储电量（kWh）}}'
    ,
```

```
                r'\begin{tabular}{lrrlrr}\toprule',
                '时间段 & 充电量 & 放电量 & 时间段 & 充电量 & 放电量 '+r'\\\midrule']
        for left,right in ((0,1),(2,3),(4,5)):
            row=[]
            for block in (left,right):
                x,y=a+24*block,a+24*(block+1)
                row += [period(24*block,24*(block+1)),f(log['charge'][x:y].sum()),f(log['discharge'][x:y].sum())]
            lines.append(' & '.join(row)+r'\\')
        lines.append(r'\midrule 00:00储电量 & \multicolumn{2}{r}{'+f(log['soc_start'][a])+r'} & 24:00储电量 & \multicolumn{2}{r}{'+f(log['soc_end'][b-1])+r'}\\\bottomrule')
        lines += [r'\end{tabular}\end{table}']
    # Four date columns as in the statement's Table 3; repeat headers over pages.
    evs=[events(log['emergency'][d*144:(d+1)*144]) for d in days]
    lines += [r'{\footnotesize\setlength{\tabcolsep}{3pt}',r'\begin{longtable}{lr|lr|lr|lr}',
              rf'\caption{{{CHINESE[name]}：指定日期紧急购电时段与电量（kWh）}}\\',r'\toprule',
              ' & '.join(r'\multicolumn{2}{c}{'+date+'}' for date in SELECTED_DATES)+r'\\',
              ' & '.join(['时间段 & 电量']*4)+r'\\\midrule\endfirsthead',
              rf'\caption[]{{{CHINESE[name]}：指定日期紧急购电时段与电量（续）}}\\',r'\toprule',
              ' & '.join(r'\multicolumn{2}{c}{'+date+'}' for date in SELECTED_DATES)+r'\\',
              ' & '.join(['时间段 & 电量']*4)+r'\\\midrule\endhead']
    for i in range(max(1,max(map(len,evs)))):
        row=[]
        for entries in evs:
            if i<len(entries):
                x,y,z=entries[i]; row += [period(x,y),f(z)]
            elif i==0 and not entries: row+=['无','0.00']
            else: row+=['','']
        lines.append(' & '.join(row)+r'\\')
    lines += [r'\bottomrule\end{longtable}}']
    return '\n'.join(lines)+'\n'


def q1_metrics(log,summary):
    eta=.9
    raw_waste=log['plan']+log['pv']+log['discharge']-log['load']-log['charge']
    waste=np.maximum(raw_waste,0.)
    balance=raw_waste-waste
    soc_residual=np.diff(log['soc'])-eta*log['charge']+log['discharge']/eta
    baseline=float(np.dot(log['price'],np.maximum(log['load']-log['pv'],0.)))
    saving=baseline-float(summary['cost'])
    saving_pct=100*saving/baseline
    overlap=float(np.max(np.minimum(log['charge'],log['discharge'])))
    return {'baseline':baseline,'saving':saving,'saving_pct':saving_pct,
            'waste':float(np.sum(waste)),'balance':float(np.max(np.abs(balance))),
            'soc_residual':float(np.max(np.abs(soc_residual))),'overlap':overlap}


def q1_tables(log,summary):
    lines=[r'\begin{table}[H]\centering\small',r'\caption{问题一指定时间段购电量及全天合计}',
           r'\begin{tabular}{lrlrlr}\toprule',
           '时间段 & 购电量 & 时间段 & 购电量 & 时间段 & 购电量 '+r'\\\midrule']
    for hours in ((10,12,14),(16,18,20)):
        row=[]
        for h in hours: row += [period(h*6,h*6+1),f(log['plan'][h*6])]
        lines.append(' & '.join(row)+r'\\')
    lines += [r'\midrule 全天购电量 & \multicolumn{2}{r}{'+f(summary['energy'])+r'} & 全天购电费 & \multicolumn{2}{r}{'+f(summary['cost'])+r'}\\\bottomrule',
              r'\end{tabular}\end{table}',r'\begin{table}[H]\centering\small',
```

```
            r'\caption{问题一储能设备分段充放电量及端点储电量（kWh）}',
            r'\begin{tabular}{lrrlrr}\toprule',
            '时间段 & 充电量 & 放电量 & 时间段 & 充电量 & 放电量 '+r'\\\midrule']
    for blocks in ((0,1),(2,3),(4,5)):
        row=[]
        for block in blocks:
            a,b=block*24,(block+1)*24
            row += [period(a,b),f(log['charge'][a:b].sum()),f(log['discharge'][a:b].sum())]
        lines.append(' & '.join(row)+r'\\')
    lines += [r'\midrule 00:00储电量 & \multicolumn{2}{r}{'+f(log['soc'][0])+r'} & 24:00储电量 & \
     multicolumn{2}{r}{'+f(log['soc'][-1])+r'}\\\bottomrule',
            r'\end{tabular}\end{table}']
    return '\n'.join(lines)+'\n'


def q1_validation_table(log,summary):
    metrics=q1_metrics(log,summary)
    lines=[r'\begin{table}[H]\centering\small',
            r'\caption{问题一可行性与经济性检验}\label{tab:q1-validation}',
            r'\begin{tabular}{lrlr}\toprule',
            r'检验指标 & 数值 & 检验指标 & 数值 \\\midrule',
            '无储能购电费/元 & '+f(metrics['baseline'])+' & 优化后购电费/元 & '+f(summary['cost'])+r'\\',
            '节省费用/元 & '+f(metrics['saving'])+' & 相对节费率 & '+f(metrics['saving_pct'])+r'\%\\',
            '最低储电量/kWh & '+f(np.min(log['soc']))+' & 最高储电量/kWh & '+f(np.max(log['soc']))+r'\\',
            '24:00储电量/kWh & '+f(log['soc'][-1])+' & 弃置余电/kWh & '+f(metrics['waste'])+r'\\',
            '最大能量平衡残差/kWh & '+latex_sci(metrics['balance'])+
            ' & 最大储电量递推残差/kWh & '+latex_sci(metrics['soc_residual'])+r'\\',
            '最大同段充放电重叠/kWh & '+f(metrics['overlap'])+' & LP--MILP费用差/元 & '+f(summary['
     absolute_lp_milp_gap'])+r'\\\bottomrule',
            r'\end{tabular}\end{table}']
    return '\n'.join(lines)+'\n'


def paired_bootstrap(first,second,seed=2026,repetitions=2000,block=7):
    """Paired circular moving-block bootstrap of daily cost differences."""
    diff=(first-second).reshape(-1,144).sum(axis=1)
    rng=np.random.default_rng(seed); n=len(diff)
    starts=rng.integers(0,n,size=(repetitions,math.ceil(n/block)))
    idx=(starts[:,:,None]+np.arange(block)[None,None,:])%n
    means=diff[idx.reshape(repetitions,-1)[:,:n]].mean(axis=1)
    return {'daily_mean_saving':float(diff.mean()), 'ci95':np.quantile(means,[.025,.975]).tolist(),
            'days':n,'block_days':block,'replicates':repetitions}


def explanatory_figures(plt, finish):
    """Draw conceptual diagrams; these are explanations, not measured results."""
    from matplotlib.patches import FancyBboxPatch
    blue, green, orange, purple = '#2d527a', '#497d58', '#cb743a', '#835a92'
    fig, ax = plt.subplots(figsize=(7.1,4.0))
    ax.set(xlim=(0,1),ylim=(-.08,1));ax.axis('off')
    def box(x,y,w,h,text,color):
        ax.add_patch(FancyBboxPatch((x,y),w,h,boxstyle='round,pad=.015',
                     linewidth=1.2,edgecolor=color,facecolor='white'))
        ax.text(x+w/2,y+h/2,text,ha='center',va='center',fontsize=10,color=color)
    def arrow(start,end,color=blue,style='-'):
        ax.annotate('',xy=end,xytext=start,arrowprops={'arrowstyle':'-|>',
                    'color':color,'lw':1.4,'linestyle':style,'mutation_scale':12})
    box(.03,.59,.23,.17,'外部电网\n计划购电',blue)
    box(.03,.27,.23,.17,'小区光伏\n提供发电量',green)
```

```
    box(.40,.43,.20,.16,'电能分配',blue)
    box(.74,.59,.23,.17,'小区负荷\n优先满足用电',blue)
    box(.74,.27,.23,.17,'未利用余电\n无售电收入',orange)
    box(.37,.04,.26,.17,'电池\n暂存电量，供以后使用',purple)
    box(.37,.81,.26,.14,'紧急购电\n只补负荷缺口',orange)
    arrow((.275,.65),(.385,.53));arrow((.275,.35),(.385,.47),green)
    arrow((.615,.53),(.725,.65));arrow((.615,.47),(.725,.35),orange)
    arrow((.645,.87),(.85,.775),orange,'--')
    arrow((.465,.415),(.465,.225),purple);arrow((.535,.225),(.535,.415),purple)
    ax.text(.43,.32,'充电',ha='right',va='center',fontsize=9,color=purple)
    ax.text(.57,.32,'放电',ha='left',va='center',fontsize=9,color=purple)
    ax.text(.5,-.055,'充电只使用满足负荷后的余电；紧急购电不用于给电池充电。',
            ha='center',fontsize=9,color='#444444')
    fig.tight_layout(pad=.6);finish(fig,'energy_flow')

    fig, ax = plt.subplots(figsize=(7.1,4.8))
    ax.set(xlim=(0,1),ylim=(0,1));ax.axis('off')
    def band(x,y,w,h,text,color,fill):
        ax.add_patch(FancyBboxPatch((x,y),w,h,boxstyle='square,pad=0',
                     edgecolor=color,facecolor=fill,linewidth=1.1))
        if text:ax.text(x+w/2,y+h/2,text,ha='center',va='center',fontsize=9,color=color)
    ax.text(.06,.965,'① 午夜联合预测并制定48小时计划',fontsize=11,color=blue)
    band(.06,.78,.44,.12,'当天24小时\n提交并承诺购电量',blue,'#eaf0f5')
    band(.50,.78,.44,.12,'次日24小时\n用于考虑之后的电池需求',blue,'#f5f6f7')
    for x,label in [(.06,'0小时'),(.50,'24小时'),(.94,'48小时')]:
        ax.text(x,.755,label,ha='center',va='top',fontsize=8)
    ax.text(.06,.66,'② 前24小时购电量锁定；实际运行不追溯修改',fontsize=10,color=orange)
    ax.annotate('',xy=(.955,.55),xytext=(.045,.55),
                arrowprops={'arrowstyle':'->','color':orange,'lw':1.2})
    for hour in (0,6,12,18,24):
        x=.06+.88*hour/24
        ax.plot(x,.55,'o',color=orange,ms=4)
        ax.text(x,.515,f'{hour}时',ha='center',va='top',fontsize=9)
    ax.text(.06,.365,'③ 每10分钟即时安全反馈：只读取当前区间实际余缺',fontsize=10,color=purple)
    band(.06,.19,.29,.10,'有余电：限功率、限容量充电',green,'#eef6ef')
    band(.355,.19,.29,.10,'有缺口：先按约束放电',purple,'#f4f0f7')
    band(.65,.19,.29,.10,'仍有缺口：紧急购电补足',orange,'#fbf1ea')
    ax.annotate('',xy=(.355,.24),xytext=(.35,.24),arrowprops={'arrowstyle':'->','color':purple,'lw':1})
    ax.annotate('',xy=(.65,.24),xytext=(.645,.24),arrowprops={'arrowstyle':'->','color':orange,'lw':1})
    ax.text(.5,.035,'区间末更新真实储电量，下一段继续反馈；紧急购电不用于充电。',ha='center',fontsize=9,color
     ='#444444')
    fig.tight_layout(pad=.6);finish(fig,'operation_timeline')


def create_figures(data,logs,summaries,experiments,selection):
    import matplotlib
    matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    from matplotlib.font_manager import FontProperties
    font_path=Path('/System/Library/Fonts/Supplemental/Songti.ttc')
    if font_path.exists():
        from matplotlib.font_manager import fontManager
        fontManager.addfont(str(font_path))
        plt.rcParams['font.family']=FontProperties(fname=str(font_path)).get_name()
    plt.rcParams.update({'font.size':9,'axes.spines.top':False,'axes.spines.right':False,
                         'axes.unicode_minus':False,'pdf.fonttype':42,'figure.dpi':140})
    colors=['#2d527a','#cb743a','#497d58','#835a92']
    figures=ROOT/'paper/figures'; figures.mkdir(parents=True,exist_ok=True)
    chart_data=ROOT/'output/support/chart_data'; chart_data.mkdir(parents=True,exist_ok=True)
```

```
    def finish(fig,name):
        fig.savefig(figures/(name+'.pdf'),bbox_inches='tight')
        fig.savefig(figures/(name+'.png'),bbox_inches='tight',dpi=150)
        plt.close(fig)
    explanatory_figures(plt,finish)
    fig,axes=plt.subplots(3,1,figsize=(7.1,5.5),sharex=True)
    for ax,values,label in zip(axes,[data.load,data.pv,data.price],['负载 / kW','光伏 / kW','实时电价 / (元/
     kWh)']):
        im=ax.imshow(values.T,origin='lower',aspect='auto',extent=[1,365,0,24],cmap='viridis')
        ax.set_ylabel('时刻 / h'); ax.set_title(label,loc='left',fontsize=10)
        fig.colorbar(im,ax=ax,pad=.02,aspect=30)
    axes[-1].set_xlabel('2025年日序');fig.tight_layout();finish(fig,'overview')
    fig,axes=plt.subplots(3,1,figsize=(7.1,5.8),sharex=True,
                          gridspec_kw={'height_ratios':[1.15,1,1]})
    q1=logs['q1']; hours=np.arange(144)/6
    for key,label,color in [('load','负载',colors[0]),('pv','光伏',colors[2]),('plan','计划购电',colors[1])
     ]:
        axes[0].plot(hours,q1[key]*6,label=label,color=color,lw=1.3)
    axes[0].set_ylabel('功率 / kW')
    axes[0].legend(ncol=3,frameon=False,loc='lower center',
                   bbox_to_anchor=(.5,1.03),borderaxespad=0)
    axes[1].plot(np.arange(145)/6,q1['soc'],color=colors[3],label='储电量')
    axes[1].axhline(1200,color='gray',ls='--',lw=.8);axes[1].axhline(10800,color='gray',ls='--',lw=.8)
    axes[1].set_ylabel('储电量 / kWh')
    width=1/6
    axes[2].bar(hours,q1['charge']*6,width=width,align='edge',color=colors[2],alpha=.7,label='充电功率')
    axes[2].bar(hours,-q1['discharge']*6,width=width,align='edge',color=colors[3],alpha=.7,label='放电功率')
    axes[2].axhline(0,color='gray',lw=.7)
    axes[2].set_xlabel('时刻 / h');axes[2].set_ylabel('充放电功率 / kW');axes[2].set_xlim(0,24)
    price_axis=axes[2].twinx()
    price_axis.step(hours,q1['price'],where='post',color=colors[1],lw=1.1,label='电价')
    price_axis.set_ylabel('电价 / (元/kWh)',color=colors[1])
    price_axis.tick_params(axis='y',labelcolor=colors[1])
    handles,labels=axes[2].get_legend_handles_labels()
    handles2,labels2=price_axis.get_legend_handles_labels()
    axes[2].legend(handles+handles2,labels+labels2,ncol=3,frameon=False,
                   loc='lower center',bbox_to_anchor=(.5,1.03),borderaxespad=0)
    fig.tight_layout(h_pad=1.8);finish(fig,'dispatch')
    pd.DataFrame({k:v for k,v in q1.items() if len(v)==144}).to_csv(chart_data/'q1_dispatch.csv',index=False
     )
    fig,axes=plt.subplots(1,2,figsize=(7.1,3.2))
    names=['q2','q3','q4_2','q4_3']; xs=np.arange(4)
    bottom=np.zeros(4)
    for fees,label,col in [(('base_fee',),'计划费',colors[0]),
                           (('up_fee','down_fee'),'调整净额',colors[1]),
                           (('emergency_fee',),'紧急费',colors[3])]:
        vals=np.array([sum(summaries[n]['evaluation'][fee] for fee in fees) for n in names])/1e6
        axes[0].bar(xs,vals,bottom=bottom,label=label,color=col);bottom+=vals
    axes[0].set_xticks(xs,['问题2','问题3','问题4-2','问题4-3'],rotation=15)
    axes[0].set_ylabel('费用 / 百万元')
    axes[0].legend(fontsize=7,frameon=False,ncol=3,loc='upper center',bbox_to_anchor=(.5,1.13))
    for name,col in zip(names,colors):
        daily=logs[name]['cost'][31*144:].reshape(-1,144).sum(axis=1)
        axes[1].plot(np.cumsum(daily)/1e6,label=name.replace('q','问题').replace('_','-'),lw=1.2,color=col)
    axes[1].set_xlabel('二月起日序');axes[1].set_ylabel('累计费用 / 百万元');axes[1].legend(fontsize=8,
     frameon=False)
    fig.tight_layout();finish(fig,'costs')
    # Report issued forecast errors by same forecast horizon, using only valid targets.
    records=[]
```

```
    for issue_hour in (0,6,12,18):
        for lead in range(1,25):
            errs=[]
            for day in range(31,365):
                end=day*144+issue_hour*6+lead*6
                if end>len(data.flat_pv):continue
                true=data.flat_pv[end-1]; forecast=data.forecasts[day,issue_hour//6,lead-1]
                if true>100:errs.append(abs(true-forecast))
            records.append({'issue_hour':issue_hour,'lead_hour':lead,'daylight_count':len(errs),
                            'mae_kw':float(np.mean(errs)) if errs else np.nan})
    fd=pd.DataFrame(records);fd.to_csv(chart_data/'issued_pv_forecast_errors.csv',index=False)
    fig,ax=plt.subplots(figsize=(7.1,2.9))
    for issue,col in zip((0,6,12,18),colors):
        rows=fd[fd.issue_hour==issue]
        ax.plot(rows.lead_hour,rows.mae_kw,label=f'{issue:02d}:00发布',color=col,marker='.',ms=3,lw=1)
    ax.set_xlabel('预测提前量 / h');ax.set_ylabel('白天光伏MAE / kW');ax.legend(ncol=4,frameon=False,
     fontsize=8)
    fig.tight_layout();finish(fig,'forecast')
    for plot_name,keys in [('ablation',['issue0','issue06','issue012','issue018']),
                           ('sensitivity',['point','k6','k24','h2','h6','roundtrip90','
     full_plan_plus_penalty','known_price'])]:
        rows=[]
        for suffix in keys:
            for base in ('q3','q4_3'):
                key=base+'_'+suffix
                if key in experiments:
                    rows.append({'policy':base,'variant':suffix,'cost':experiments[key]['evaluation']['cost'
     ],
                                 'change_pct':100*(experiments[key]['evaluation']['cost']/summaries[base]['
     evaluation']['cost']-1)})
        df=pd.DataFrame(rows);df.to_csv(chart_data/(plot_name+'.csv'),index=False)
        fig,ax=plt.subplots(figsize=(7.1,3.0))
        if rows:
            labels=list(dict.fromkeys(df.variant));x=np.arange(len(labels))
            for j,base in enumerate(('q3','q4_3')):
                vals=[next((r['change_pct'] for r in rows if r['policy']==base and r['variant']==label),np.
     nan) for label in labels]
                ax.bar(x+(j-.5)*.34,vals,width=.34,label={'q3':'问题3','q4_3':'问题4-3'}[base],color=colors[
     j])
            ax.set_xticks(x,[VARIANT_LABELS[label].replace('\\','') for label in labels],rotation=22,ha='
     right');ax.axhline(0,color='black',lw=.6)
            ax.legend(frameon=False);ax.set_ylabel('相对主策略费用变化 / %')
        fig.tight_layout();finish(fig,plot_name)


def day_interpretations(logs, generated):
    """Explain selected dates using the same executed ledgers as their tables."""
    values={}
    for name in ('q2','q3','q4_2','q4_3'):
        values[name]={}
        for date in SELECTED_DATES:
            day=int((np.datetime64(date)-np.datetime64('2025-01-01'))/np.timedelta64(1,'D'))
            a,b=day*144,(day+1)*144
            values[name][date]={k:float(logs[name][k][a:b].sum())
                                for k in ('cost','load','pv','emergency','charge','discharge')}
    summer=values['q2']['2025-06-21'];winter=values['q2']['2025-12-21']
    q2=('在四个展示日期中，6月21日实际电费最低，为'+f(summer['cost']/1e4)+'万元；当日负荷为'+
        f(summer['load']/1e4)+'万 kWh，光伏发电量为'+f(summer['pv']/1e4)+'万 kWh。12月21日负荷增至'+
        f(winter['load']/1e4)+'万 kWh，光伏发电量降至'+f(winter['pv']/1e4)+'万 kWh，实际电费为'+
```

```
        f(winter['cost']/1e4)+'万元。对比表明，外网购电决策同时受分时电价及净负荷水平影响。\n\n'+
        '储能表按4小时汇总多个10分钟区间，因此同一汇总时段可同时出现正的累计充电量与累计放电量。'+
        '单日累计充电量也可能超过电池容量，这反映日内多次循环；容量约束限制任一时刻的储电量，而非全天累计能量吞吐量。')
    september=values['q3']['2025-09-23'];march=values['q3']['2025-03-20']
    q3=('在问题三主策略中，9月23日紧急购电量为'+f(september['emergency'])+
        ' kWh、实际电费为'+f(september['cost'])+'元；3月20日紧急购电量为'+
        f(march['emergency'])+' kWh、实际电费为'+f(march['cost'])+'元。'+
        '不同日期的更新收益还取决于负荷、光伏、库存与调整价格，故总体经济性以“全部更新—仅0时”完整334天控制实验评价。\n\n'+
        '购电表斜线两侧分别表示午夜原计划与最终有效购电量，二者之差为最终调整量。'+
        '储能表记录实际充放电量，紧急购电表记录安全执行后仍需补足的电量，三类结果共同构成当日实际供电账本。')
    s0=values['q4_2']['2025-06-21'];s1=values['q4_3']['2025-06-21']
    w0=values['q4_2']['2025-12-21'];w1=values['q4_3']['2025-12-21']
    q4=('在相同实际电价序列下，6月21日允许调整策略的费用为'+f(s1['cost'])+'元，较不调整策略减少'+
        f(s0['cost']-s1['cost'])+'元，且当日未发生紧急购电。12月21日允许调整后，紧急购电量由'+
        f(w0['emergency'])+' kWh降至'+f(w1['emergency'])+' kWh，但实际电费增加'+f(w1['cost']-w0['cost'])+
        '元。经济性评价需综合原计划费、调整费与紧急购电费，不能仅依据紧急购电量。\n\n'+
        '两类策略的日初储电量分别由此前连续运行状态决定，指定展示日期不重新设定统一初值。'+
        '因此，日表用于解释实际运行特征，策略总体比较仍采用全期费用，并单独校正期初与期末储电量差异。')
    for name,content in [('q2',q2),('q3',q3),('q4',q4)]:
        (generated/('day_interpretations_'+name+'.tex')).write_text(content+'\n',encoding='utf-8')
    write_json(ROOT/'output/support/chart_data/selected_day_explanations.json',values)


def export_all(paper_only=False,results_only=False):
    data=load_data(); out=ROOT/'output/support';out.mkdir(parents=True,exist_ok=True)
    generated=ROOT/'paper/generated';generated.mkdir(parents=True,exist_ok=True)
    logs={};summaries={}
    for name in OUTPUT_NAMES:
        log,summary=fetch(name);logs[name]=log;summaries[name]=summary
        if not paper_only:export_workbook(name,log,out)
        if results_only:continue
        if name=='q1':
            content=q1_tables(log,summary)
            (generated/'validation_q1.tex').write_text(q1_validation_table(log,summary),encoding='utf-8')
        else:
            content=day_tables(name,log)
        (generated/('tables_'+name+'.tex')).write_text(content,encoding='utf-8')
    if results_only:
        print('Exported the five required result workbooks.',flush=True)
        return
    selection=json.loads((ROOT/'artifacts/selection.json').read_text())
    experiments={}
    for path in sorted((ROOT/'artifacts/experiments').glob('*/summary.json')):
        value=json.loads(path.read_text())
        name=path.parent.name
        if name=='q2_point':
            continue  # superseded by the explicit no-margin quantile experiment
        base=next((policy for policy in ('q4_2','q4_3','q2','q3') if name.startswith(policy+'_')),None)
        variant=name[len(base)+1:] if base else ''
        if value['evaluation'].get('days')==334 and variant in VARIANT_LABELS:
            experiments[name]=value
    q2=summaries['q2']['evaluation'];q3=summaries['q3']['evaluation']
    q42=summaries['q4_2']['evaluation'];q43=summaries['q4_3']['evaluation']
    q1_log=logs['q1']
    q1_baseline=float(np.dot(q1_log['price'],np.maximum(q1_log['load']-q1_log['pv'],0.)))
    q1_saving=q1_baseline-summaries['q1']['cost']
    q2ref=experiments['q2_scenario_reference']['evaluation']
```

```
    q270=experiments['q2_q70']['evaluation'];q290=experiments['q2_q90']['evaluation']
    q3issue0=experiments['q3_issue0']['evaluation']
    q3issue6=experiments['q3_issue06']['evaluation']
    q3issue12=experiments['q3_issue012']['evaluation']
    q3issue18=experiments['q3_issue018']['evaluation']
    q43issue0=experiments['q4_3_issue0']['evaluation']
    q42known=experiments['q4_2_known_price']['evaluation']
    q43known=experiments['q4_3_known_price']['evaluation']
    evaluation=slice(31*144,None)
    q3_delta=logs['q3']['effective'][evaluation]-logs['q3']['plan'][evaluation]
    q43_delta=logs['q4_3']['effective'][evaluation]-logs['q4_3']['plan'][evaluation]
    storage_saving_pct=[]
    for name,main in (('q2',q2),('q3',q3),('q4_2',q42),('q4_3',q43)):
        baseline=experiments[name+'_no_storage']['evaluation']['cost']
        storage_saving_pct.append(100*(1-main['cost']/baseline))
    macros={'QOneCost':summaries['q1']['cost'],'QOneEnergy':summaries['q1']['energy'],
            'QOneBaselineCost':q1_baseline,'QOneSaving':q1_saving,
            'QOneSavingPct':100*q1_saving/q1_baseline,
            'QOneEnergyWan':summaries['q1']['energy']/1e4,
            'QTwoCost':q2['cost'],'QThreeCost':q3['cost'],'QFourTwoCost':q42['cost'],'QFourThreeCost':q43['
     cost'],
            'QTwoPlanCost':q2['base_fee'],'QTwoEmergencyCost':q2['emergency_fee'],
            'QTwoInitialSOC':q2['initial_soc'],'QTwoFinalSOC':q2['final_soc'],
            'QTwoReferenceCost':q2ref['cost'],'QTwoReferenceEmergencyEnergy':q2ref['emergency'],
            'QTwoReferenceSaving':q2ref['cost']-q2['cost'],
            'QTwoReferenceEmergencySaving':q2ref['emergency']-q2['emergency'],
            'QTwoReferenceSavingPct':100*(1-q2['cost']/q2ref['cost']),
            'QTwoReferenceEmergencySavingPct':100*(1-q2['emergency']/q2ref['emergency']),
            'QTwoQSeventyCost':q270['cost'],'QTwoQSeventyEmergencyEnergy':q270['emergency'],
            'QTwoQNinetyCost':q290['cost'],'QTwoQNinetyEmergencyEnergy':q290['emergency'],
            'QTwoVsSeventyCostPct':100*(q2['cost']/q270['cost']-1),
            'QTwoVsSeventyEmergencyPct':100*(1-q2['emergency']/q270['emergency']),
            'QThreeUpdateSaving':q3issue0['cost']-q3['cost'],
            'QThreeUpdateSavingWan':(q3issue0['cost']-q3['cost'])/1e4,
            'QThreeSavingPct':100*(1-q3['cost']/q3issue0['cost']),
            'QThreeIssueZeroCost':q3issue0['cost'],
            'QThreeIssueSixCost':q3issue6['cost'],
            'QThreeIssueTwelveCost':q3issue12['cost'],
            'QThreeIssueEighteenCost':q3issue18['cost'],
            'QThreeAllVsTwelveSaving':q3issue12['cost']-q3['cost'],
            'QThreeEmergencySaving':q3issue0['emergency']-q3['emergency'],
            'QThreeEmergencySavingPct':100*(1-q3['emergency']/q3issue0['emergency']),
            'QThreeAdjustmentUp':float(np.maximum(q3_delta,0).sum()),
            'QThreeAdjustmentDown':float(np.maximum(-q3_delta,0).sum()),
            'QThreeAdjustmentNetDown':float(-q3_delta.sum()),
            'QThreeAdjustmentUpWan':float(np.maximum(q3_delta,0).sum()/1e4),
            'QThreeAdjustmentDownWan':float(np.maximum(-q3_delta,0).sum()/1e4),
            'QThreeAdjustmentNetDownWan':float(-q3_delta.sum()/1e4),
            'QFourSaving':q42['cost']-q43['cost'],
            'QFourSavingWan':(q42['cost']-q43['cost'])/1e4,
            'QFourSavingPct':100*(1-q43['cost']/q42['cost']),
            'QFourEmergencySaving':q42['emergency']-q43['emergency'],
            'QFourEmergencySavingPct':100*(1-q43['emergency']/q42['emergency']),
            'QFourUpdateSaving':q43issue0['cost']-q43['cost'],
            'QFourUpdateSavingPct':100*(1-q43['cost']/q43issue0['cost']),
            'QFourAdjustmentNetDown':float(-q43_delta.sum()),
            'QFourAdjustmentNetDownWan':float(-q43_delta.sum()/1e4),
            'QFourTwoKnownPriceSaving':q42['cost']-q42known['cost'],
            'QFourTwoKnownPriceSavingPct':100*(1-q42known['cost']/q42['cost']),
```

```
            'QFourThreeKnownPriceSaving':q43['cost']-q43known['cost'],
            'QFourThreeKnownPriceSavingPct':100*(1-q43known['cost']/q43['cost']),
            'StorageSavingMinPct':min(storage_saving_pct),
            'StorageSavingMaxPct':max(storage_saving_pct),
            'QTwoNoMarginCostWan':experiments['q2_no_margin']['evaluation']['cost']/1e4,
            'QTwoEmergencyEnergy':q2['emergency'],'QThreeEmergencyEnergy':q3['emergency'],
            'QFourTwoEmergencyEnergy':q42['emergency'],'QFourThreeEmergencyEnergy':q43['emergency']}
    for key in ('QOneCost','QTwoCost','QThreeCost','QFourTwoCost','QFourThreeCost'):
        macros[key+'Wan']=macros[key]/1e4
    (generated/'results.tex').write_text('\n'.join('\\newcommand{\\'+k+'}{'+f(v)+'}' for k,v in macros.items
     ())+'\n',encoding='utf-8')
    q2_sensitivity=[r'\begin{table}[H]\centering\small',
        r'\caption{问题二滚动残差分位数敏感性（2月至12月）}\label{tab:q2-quantile}',
        r'\begin{tabular}{crrr}\toprule',
        r'分位数 & 计划费用/元 & 紧急购电量/kWh & 总费用/元\\\midrule']
    for label,values in ((r'70\%',q270),(r'80\%',q2),(r'90\%',q290)):
        q2_sensitivity.append(label+' & '+f(values['base_fee'])+' & '+f(values['emergency'])+' & '+f(values[
     'cost'])+r'\\')
    q2_sensitivity += [r'\bottomrule\end{tabular}\end{table}']
    (generated/'q2_sensitivity.tex').write_text('\n'.join(q2_sensitivity)+'\n',encoding='utf-8')
    tests=json.loads((ROOT/'artifacts/verification.json').read_text()) if (ROOT/'artifacts/verification.json
     ').exists() else {}
    stats={}
    q2_reference_log,_=fetch('q2_scenario_reference','artifacts/experiments')
    q3_issue0_log,_=fetch('q3_issue0','artifacts/experiments')
    stats['q2_reference_vs_q2']=paired_bootstrap(q2_reference_log['cost'][31*144:],logs['q2']['cost'
     ][31*144:])
    stats['q3_issue0_vs_q3']=paired_bootstrap(q3_issue0_log['cost'][31*144:],logs['q3']['cost'][31*144:])
    stats['q4_2_vs_q4_3']=paired_bootstrap(logs['q4_2']['cost'][31*144:],logs['q4_3']['cost'][31*144:])
    write_json(out/'statistical_comparisons.json',stats)
    write_json(out/'summary.json',{'main':summaries,'experiments':experiments,'selection':selection,'
     statistics':stats})
    forecast_metrics=[]
    for name in ('q2','q3','q4_2','q4_3'):
        log=logs[name]; interval=slice(31*144,None)
        net_error=(log['expected_net']-log['load']+log['pv'])[interval]
        price_error=(log['expected_price']-log['price'])[interval]
        forecast_metrics.append({'policy':name,
            'pre_execution_reference_net_mae_kwh':float(np.abs(net_error).mean()),
            'pre_execution_reference_net_rmse_kwh':float(np.sqrt(np.square(net_error).mean())),
            'pre_execution_reference_price_mae_yuan_per_kwh':float(np.abs(price_error).mean()),
            'pre_execution_reference_price_rmse_yuan_per_kwh':float(np.sqrt(np.square(price_error).mean())),
            **{'midnight_24h_'+k+'_mae':v for k,v in summaries[name]['forecast_mae'].items()}})
    pd.DataFrame(forecast_metrics).to_csv(out/'forecast_metrics.csv',index=False,float_format='%.9f')
    rows=[]
    for name in ['q2','q3','q4_2','q4_3']:
        log=logs[name]
        for day in range(31,365):
            a,b=day*144,(day+1)*144
            rows.append({'policy':name,'date':str(data.dates[day]),**{k:float(log[k][a:b].sum()) for k in ('
     plan','effective','charge','discharge','emergency','waste','cost')},
                         'soc_start':float(log['soc_start'][a]),'soc_end':float(log['soc_end'][b-1])})
    if not paper_only:pd.DataFrame(rows).to_csv(out/'daily_summary.csv',index=False,float_format='%.9f')
    v=[r'\begin{table}[H]\centering\small\caption{二月至十二月实际运行汇总}',
       r'\begin{tabular}{lrrrr}\toprule',r'策略 & 总费用/万元 & 紧急量/MWh & 期初储电/kWh & 期末储电/kWh\\\
     midrule']
    for name in ['q2','q3','q4_2','q4_3']:
        z=summaries[name]['evaluation']
        v.append(' & '.join([DISPLAY_NAMES[name],f(z['cost']/1e4),f(z['emergency']/1e3),f(z['initial_soc']),
```

```
 f(z['final_soc'])])+r'\\')
v += [r'\bottomrule\end{tabular}\end{table}',
      r'\begin{table}[H]\centering\small\caption{储能与预测裕度的费用对照（万元）}',
      r'\begin{tabular}{lrrr}\toprule',r'策略 & 无储能 & 简化预测/无裕度 & 本文策略\\\midrule']
for name in ('q2','q3','q4_2','q4_3'):
    simplified='no_margin' if name=='q2' else 'point'
    vals=[experiments[name+'_'+variant]['evaluation']['cost']/1e4 for variant in ('no_storage',
 simplified)]
    vals.append(summaries[name]['evaluation']['cost']/1e4)
    v.append(DISPLAY_NAMES[name]+' & '+' & '.join(f(x) for x in vals)+r'\\')
v += [r'\bottomrule\end{tabular}\par\footnotesize 各列采用同一行对应的交易规则和实际数据；储电量差异的校
 正结果见附录。\end{table}',
      r'\begin{table}[H]\centering\small\caption{平均每日节费及其估计区间}',
      r'\begin{tabular}{lrrr}\toprule',r'比较 & 平均日节省/元 & 95\%下限 & 95\%上限\\\midrule']
comparisons={'q2_reference_vs_q2':r'问题二：80\%分位数模型相对12情境模型',
             'q3_issue0_vs_q3':'问题三：全部更新相对仅0时',
             'q4_2_vs_q4_3':'问题四：可调整相对不调整'}
for name,z in stats.items():v.append(comparisons[name]+' & '+f(z['daily_mean_saving'])+' & '+f(z['ci95'
 ][0])+' & '+f(z['ci95'][1])+r'\\')
all_positive=all(z['ci95'][0]>0 for z in stats.values())
interval_note=(r'三组区间下限均大于0，支持本年度样本中存在正节费；区间不代表跨年度保证。'
               if all_positive else r'至少一组区间包含0，不能据此认定存在稳定的节费优势。')
v += [r'\bottomrule\end{tabular}\par\footnotesize 正数表示节费，负数表示多花钱；'+interval_note+r'\end{
 table}']
extra=[r'\begin{table}[H]\centering\small\caption{情景滚动模型一月后半月预测方法选择（储电量校正后的费
 用，元）}',
      r'\begin{tabular}{lrrrl}\toprule',r'策略 & 季节朴素 & 岭回归 & 梯度提升 & 入选方法\\\midrule']
for name,scores in selection['validation_inventory_cost'].items():
    display='问题二12情境模型' if name=='q2' else DISPLAY_NAMES[name]
    extra.append(display+' & '+' & '.join(f(scores[m]) for m in ('naive','ridge','boost'))+' & '+{'naive
 ':'季节朴素','ridge':'岭回归','boost':'梯度提升'}[selection['selected'][name]]+r'\\')
extra += [r'\bottomrule\end{tabular}\end{table}']
v += [r'\begin{table}[H]\centering\small\caption{每次执行前对当前10分钟的预测误差（正式334日）}',
      r'\begin{tabular}{lrrrr}\toprule',
      r'策略 & 净负荷MAE/kWh & 净负荷RMSE/kWh & 电价MAE & 电价RMSE\\\midrule']
for row in forecast_metrics:
    v.append(DISPLAY_NAMES[row['policy']]+' & '+' & '.join(f(row[key],4) for key in
        ('pre_execution_reference_net_mae_kwh','pre_execution_reference_net_rmse_kwh',
         'pre_execution_reference_price_mae_yuan_per_kwh','
 pre_execution_reference_price_rmse_yuan_per_kwh'))+r'\\')
v += [r'\bottomrule\end{tabular}\par\footnotesize 电价误差单位为元/kWh；问题二记录午夜参考预测，其他策略
 记录当前区间实现前的控制层首步预测。\end{table}']
extra += [r'\begin{table}[H]\centering\small\caption{午夜对当天24小时的平均绝对预测误差}',
          r'\begin{tabular}{lrrr}\toprule',r'策略 & 负荷/kW & 光伏/kW & 电价/(元/kWh)\\\midrule']
for name in ('q2','q3','q4_2','q4_3'):
    errors=summaries[name]['forecast_mae']
    extra.append(DISPLAY_NAMES[name]+' & '+f(errors['load'])+' & '+f(errors['pv'])+' & '+f(errors['price
 '],6)+r'\\')
extra += [r'\bottomrule\end{tabular}\end{table}']
if experiments:
    extra += [r'{\small\begin{longtable}{lrrr}',rf'\caption{{{len(experiments)}组对照与参数检验完整结果
 （二月至十二月）}}\\\toprule',
              r'策略变体 & 实际费用/万元 & 紧急量/MWh & 库存校正费用/万元\\\midrule\endfirsthead',
              rf'\caption[]{{{len(experiments)}组对照与参数检验完整结果（续）}}\\\toprule',
              r'策略变体 & 实际费用/万元 & 紧急量/MWh & 库存校正费用/万元\\\midrule\endhead']
    for name,z in experiments.items():
        base=next(p for p in ('q4_2','q4_3','q2','q3') if name.startswith(p+'_'))
        label=DISPLAY_NAMES[base]+'：'+VARIANT_LABELS[name[len(base)+1:]]
        e=z['evaluation'];extra.append(label+' & '+f(e['cost']/1e4)+' & '+f(e['emergency']/1e3)+' & '+f(
```

```
 e['inventory_corrected_cost']/1e4)+r'\\')
659         extra += [r'\bottomrule\end{longtable}}']
660     if tests:
661         mantissa,exponent=format(tests.get('max_energy_residual_kwh',0),'.3e').split('e')
662         residual_latex='$'+mantissa+r'\times 10^{'+str(int(exponent))+'}$'
663         extra.append('逐时独立验证的最大能量平衡误差为'+residual_latex+' kWh；问题一LP与MILP费用差为'+f(
 summaries['q1']['absolute_lp_milp_gap'],9)+'元。')
664     bound_path=ROOT/'artifacts/perfect_bound.json'
665     if bound_path.exists():
666         bounds=json.loads(bound_path.read_text())
667         extra.append('全年365日完全信息松弛下界：固定电价'+f(bounds['fixed']['cost'])+'元，波动电价'+f(bounds
 ['variable']['cost'])+'元。该下界允许年末自由储电，与334日表内费用不能直接相减。')
668         extra.append('对应365日实际费用：问题二'+f(summaries['q2']['whole_year']['cost'])+'元，问题三'+
669                 f(summaries['q3']['whole_year']['cost'])+'元，波动电价下分别为'+
670                 f(summaries['q4_2']['whole_year']['cost'])+'元、'+f(summaries['q4_3']['whole_year']['cost'
 ])+
671                 '元。差额同时包含信息、交易限制和策略近似的影响。')
672         write_json(out/'perfect_information_bound.json',bounds)
673     extra += [r'\resultfigure{forecast}{附件3光伏预报的平均绝对误差随提前小时数变化：正式334日，仅取实际光伏
 大于100 kW的时段}',
674             r'\resultfigure{sensitivity}{情景数、控制范围及费用解释变化时，实际费用相对主策略的变化}',r'\
 FloatBarrier']
675     (generated/'validation.tex').write_text('\n'.join(v)+'\n',encoding='utf-8')
676     (generated/'experiment_details.tex').write_text('\n'.join(extra)+'\n',encoding='utf-8')
677     create_figures(data,logs,summaries,experiments,selection)
678     day_interpretations(logs,generated)
679     # Source appendix is generated only from shipped, first-party executable source.
680     source_files=[ROOT/'run.py']+sorted((ROOT/'src/microgrid').glob('*.py'))
681     app=[r'\section{完整程序代码}',r'本附录列示与支撑材料同名的完整自编程序。依赖环境、输入放置方式及运行顺序
 见支撑材料 README.md。']
682     for file in source_files:
683         rel=file.relative_to(ROOT).as_posix()
684         if file.name=='__init__.py':continue
685         app += [r'\subsection*{\texttt{\detokenize{'+rel+'}}}',r'\lstinputlisting[language=Python]{../'+rel+
 '}']
686     app += [r'\section{AI工具使用与核验说明}',
687             r'本次AI辅助涵盖资料检索、数学推导讨论、代码实现、自动验证、图表生成与论文草拟，具体过程见支撑材料
 “AI工具使用详情.pdf”。自动检验不能替代参赛队逐项人工核验；人工审查、修改与采纳记录由参赛队据实完成。']
688     (generated/'appendix.tex').write_text('\n'.join(app)+'\n',encoding='utf-8')
689     manifest={str(p.relative_to(ROOT)):hashlib.sha256(p.read_bytes()).hexdigest() for p in sorted((ROOT/'C题
 ').rglob('*')) if p.is_file() and p.suffix in ('.pdf','.xlsx')}
690     write_json(out/'input_manifest.json',manifest)
691     with (out/'time_mapping.csv').open('w',newline='',encoding='utf-8-sig') as stream:
692         writer=csv.writer(stream);writer.writerow(['slot','source_endpoint','original_template_label','
 adopted_interval'])
693         original=load_workbook(ROOT/'C题/附件/附件5/result1.xlsx',read_only=True,data_only=True)['计划购电量
 ']
694         for slot in range(144):writer.writerow([slot+1,clock(slot+1),original.cell(slot+2,1).value,period(
 slot,slot+1)])
695     print('Updated paper tables, figures, and explanations from existing results.' if paper_only else
696           'Exported five workbooks, paper tables, figures, and supporting ledgers.',flush=True)
```

src/microgrid/forecast.py

```
1  """Causal forecasts and finite historical residual scenarios.
2
3  Every actual observation is indexed by the interval it describes. An origin of
4  ``i`` can inspect observations ``[:i]`` only. No current-interval actual is an
```

```
optimization input. Historical forecast records are either predictions made at
their original origin or explicitly labelled causal naive reconstructions.
"""
from __future__ import annotations

from dataclasses import dataclass
from typing import Any
import time

import numpy as np

from .data import DataBundle


class _RidgeModel:
    """Small standardized ridge solve with explicitly checked finite arrays.

    Explicit contractions avoid spurious floating-point status warnings from
    the platform's NumPy/Accelerate matrix multiplication implementation.
    """

    def fit(self, x: np.ndarray, y: np.ndarray) -> None:
        from scipy.linalg import solve
        self.center = x.mean(axis=0)
        self.scale = x.std(axis=0)
        self.scale[self.scale < 1e-12] = 1.
        z = (x - self.center) / self.scale
        self.intercept = y.mean(axis=0)
        gram = np.einsum("ni,nj->ij", z, z, optimize=False)
        gram.flat[::len(gram) + 1] += 30.
        cross = np.einsum("ni,nj->ij", z, y - self.intercept, optimize=False)
        self.coef = solve(gram, cross, assume_a="pos", check_finite=True)
        if not np.isfinite(self.coef).all():
            raise FloatingPointError("non-finite ridge coefficients")

    def predict(self, x: np.ndarray) -> np.ndarray:
        z = (x - self.center) / self.scale
        return np.einsum("ni,ij->nj", z, self.coef, optimize=False) + self.intercept


@dataclass
class ForecastResult:
    net_s: np.ndarray
    price_s: np.ndarray
    mean_load: np.ndarray
    mean_pv: np.ndarray
    mean_price: np.ndarray
    provenance: dict[str, Any]

    def __getitem__(self, key: str) -> Any:
        return getattr(self, key)


class ForecastEngine:
    """Daily fitted point models and 12 past, completely observed error blocks.

    Ridge/boost learn corrections to the same causal seasonal baseline. A
    second-day forecast uses the origin's last observed day, never the target
    day's unobserved preceding day. Price is modelled in logarithms.
```

```
    """

    def __init__(self, data: DataBundle, family: str = "naive", seed: int = 2026):
        if family not in {"naive", "ridge", "boost"}:
            raise ValueError("family must be naive, ridge, or boost")
        self.data = data
        self.family = family
        self.seed = int(seed)
        self._records: dict[int, np.ndarray] = {}
        self._record_sources: dict[int, str] = {}
        self._daily: dict[int, np.ndarray] = {}
        self._baseline_cache: dict[tuple[int, int], np.ndarray] = {}
        self._day_fit: int | None = None
        self._model: Any = None
        self._price_mode: bool | None = None
        # Fixed-tariff problems may not use Attachment 4, even indirectly as
        # a feature for load/PV. The source is selected on the first query.
        self._price_daily = np.broadcast_to(data.prior_price, data.price.shape)
        self._price_flat = np.tile(data.prior_price, data.n_days)
        self._calls = 0
        self._max_actual_end = 0
        self.audit_records: list[dict[str, Any]] = []
        self._start_weekday = int((data.start_date - np.datetime64("1970-01-05", "D")).astype(int) % 7)

    def _weekday(self, day: int) -> int:
        return (self._start_weekday + day) % 7

    def _prior(self) -> np.ndarray:
        return np.column_stack((self.data.prior_load, self.data.prior_pv,
                                np.log(np.maximum(self.data.prior_price, 1e-6))))

    def _actual(self, start: int, end: int) -> np.ndarray:
        return np.column_stack((self.data.flat_load[start:end], self.data.flat_pv[start:end],
                                np.log(np.maximum(self._price_flat[start:end], 1e-6))))

    def _set_price_mode(self, variable_price: bool) -> None:
        if self._price_mode == variable_price:
            return
        self._price_mode = variable_price
        self._price_daily = (self.data.price if variable_price else
                             np.broadcast_to(self.data.prior_price, self.data.price.shape))
        self._price_flat = self.data.flat_price if variable_price else np.tile(self.data.prior_price, self.
 data.n_days)
        self._records.clear(); self._record_sources.clear(); self._daily.clear()
        self._baseline_cache.clear(); self._day_fit = None; self._model = None

    def _baseline_day(self, origin_day: int, target_day: int) -> np.ndarray:
        key = (origin_day, target_day)
        if key in self._baseline_cache:
            return self._baseline_cache[key]
        prior = self._prior()
        if origin_day <= 0:
            result = prior.copy()
        else:
            end = min(origin_day, self.data.n_days)
            begin = max(0, end - 28)
            recent_days = np.arange(max(0, end - 7), end)
            same_days = np.array([d for d in range(begin, end)
                                  if self._weekday(d) == self._weekday(target_day)], dtype=int)
```

```
            load_recent = self.data.load[recent_days].mean(axis=0)
            pv_recent = self.data.pv[recent_days].mean(axis=0)
            lp_recent = np.log(self._price_daily[recent_days]).mean(axis=0)
            if not len(same_days):
                same_days = recent_days
            load_same = self.data.load[same_days].mean(axis=0)
            lp_same = np.log(self._price_daily[same_days]).mean(axis=0)
            result = np.column_stack((
                .70 * load_same + .20 * load_recent + .10 * prior[:, 0],
                .85 * pv_recent + .15 * prior[:, 1],
                .60 * lp_same + .30 * lp_recent + .10 * prior[:, 2],
            ))
        self._baseline_cache[key] = result
        return result

    def _features(self, origin_day: int, target_day: int) -> tuple[np.ndarray, np.ndarray]:
        baseline = self._baseline_day(origin_day, target_day)
        prior = self._prior()
        last = self._actual((origin_day - 1) * 144, origin_day * 144) if origin_day > 0 else prior
        lag7_day = target_day - 7
        lag7 = self._actual(lag7_day * 144, (lag7_day + 1) * 144) if 0 <= lag7_day < origin_day else prior
        slot = np.arange(144)
        angle = 2 * np.pi * (slot + 1) / 144
        seasonal = 2 * np.pi * target_day / 365.25
        weekday = np.zeros((144, 7)); weekday[:, self._weekday(target_day)] = 1
        scaled = lambda x: x / np.array([5000., 5000., 1.])
        features = np.column_stack((
            scaled(baseline), scaled(last), scaled(lag7), scaled(prior),
            np.sin(angle), np.cos(angle), np.sin(2 * angle), np.cos(2 * angle),
            np.full(144, np.sin(seasonal)), np.full(144, np.cos(seasonal)),
            np.full(144, target_day - origin_day), weekday,
        ))
        return features, baseline

    def _fit(self, day: int) -> None:
        if self._day_fit == day:
            return
        self._day_fit, self._model = day, None
        if self.family == "naive" or day < 7:
            return
        features, targets = [], []
        for origin_day in range(max(1, day - 60), day):
            for ahead in (0, 1):
                target_day = origin_day + ahead
                if target_day >= day:
                    continue
                x, baseline = self._features(origin_day, target_day)
                y = self._actual(target_day * 144, (target_day + 1) * 144)
                features.append(x)
                targets.append((y - baseline) / np.array([5000., 5000., 1.]))
        if not features:
            return
        x, y = np.vstack(features), np.vstack(targets)
        if self.family == "ridge":
            self._model = _RidgeModel()
            self._model.fit(x, y)
        else:
            from sklearn.ensemble import HistGradientBoostingRegressor
            self._model = []
```

```
            # Subsampling time points bounds the daily fit cost without looking
            # at any test-day observation; all 144 points are still predicted.
            x, y = x[::2], y[::2]
            for j in range(3):
                model = HistGradientBoostingRegressor(
                    max_iter=60, max_leaf_nodes=15, min_samples_leaf=36,
                    learning_rate=.08, l2_regularization=5., early_stopping=False,
                    random_state=self.seed,
                )
                model.fit(x, y[:, j]); self._model.append(model)

    @staticmethod
    def _physical(transformed: np.ndarray) -> np.ndarray:
        if not np.isfinite(transformed).all():
            raise FloatingPointError("non-finite point forecast")
        result = transformed.copy()
        result[:, :2] = np.maximum(result[:, :2], 0)
        result[:, 2] = np.exp(np.clip(result[:, 2], -9, 3))
        return result

    def _ensure_day(self, day: int) -> np.ndarray:
        if day in self._daily:
            return self._daily[day]
        start = time.perf_counter()
        self._fit(day)
        parts = []
        # Three daily profiles cover a 48 h query issued after midnight.
        for target_day in (day, day + 1, day + 2):
            x, baseline = self._features(day, target_day)
            pred = baseline.copy()
            if self._model is not None:
                correction = (np.column_stack([m.predict(x) for m in self._model])
                              if isinstance(self._model, list) else self._model.predict(x))
                pred += correction * np.array([5000., 5000., 1.])
            parts.append(self._physical(pred))
        result = np.vstack(parts)
        self._daily[day] = result
        self._records[day] = result
        self._record_sources[day] = self.family if self._model is not None else "naive"
        self.audit_records.append({
            "day": day, "family": self.family,
            "effective_family": self._record_sources[day],
            "training_actual_end_exclusive": day * 144,
            "training_days_max": min(day, 60),
            "variable_price": self._price_mode,
            "fit_and_predict_seconds": time.perf_counter() - start,
        })
        return result

    def _historical_record(self, day: int) -> np.ndarray:
        if day not in self._records:
            # Lazy Feb-1 start: do not fit the selected February model on
            # January targets and then pretend those were January predictions.
            self._records[day] = np.vstack([
                self._physical(self._baseline_day(day, target_day))
                for target_day in (day, day + 1, day + 2)
            ])
            self._record_sources[day] = "causal_naive_initialization"
        return self._records[day]
```

```python

    def _latest_issue(self, origin: int, allowed: tuple[int, ...]) -> tuple[int, int, int] | None:
        if not allowed:
            return None
        day, phase = divmod(origin, 144)
        for candidate_day in (day, day - 1):
            if not 0 <= candidate_day < self.data.n_days:
                continue
            for hour in sorted(allowed, reverse=True):
                issue_origin = candidate_day * 144 + hour * 6
                if issue_origin <= origin:
                    return candidate_day, hour // 6, issue_origin
        return None

    def _official_curve(self, origin: int, horizon: int, allowed: tuple[int, ...],
                        fallback: np.ndarray) -> tuple[np.ndarray, int | None, np.ndarray]:
        latest = self._latest_issue(origin, allowed)
        result = fallback.copy()
        mask = np.zeros(horizon, dtype=bool)
        if latest is None:
            return result, None, mask
        day, issue_slot, issue_origin = latest
        endpoints = origin + np.arange(1, horizon + 1)
        targets = issue_origin + 6 * np.arange(1, 25)
        available_future = targets > origin
        if not available_future.any():
            return result, issue_origin, mask
        anchor = self.data.flat_pv[origin - 1] if origin > 0 else self.data.prior_pv[-1]
        xp = np.r_[origin, targets[available_future]]
        yp = np.r_[anchor, self.data.forecasts[day, issue_slot, available_future]]
        mask = endpoints <= targets[-1]
        result[mask] = np.interp(endpoints[mask], xp, yp)
        return np.maximum(result, 0), issue_origin, mask

    def _residuals(self, origin: int, horizon: int, count: int,
                   use_official: bool, allowed: tuple[int, ...]) -> tuple[np.ndarray, dict[str, Any]]:
        day, phase = divmod(origin, 144)
        if count == 1:
            return np.zeros((1, horizon, 3)), {
                "residual_block_count": 0, "residual_actual_end_exclusive": 0,
                "residual_initialization": "point_forecast_ablation_zero_residual",
            }
        # Entire historical target block must have ended by the current origin.
        candidates = [d for d in range(max(0, day - 60), day)
                      if d * 144 + phase + horizon <= origin]
        errors = np.zeros((count, horizon, 3))
        if not candidates:
            return errors, {"residual_block_count": 0, "residual_actual_end_exclusive": 0,
                            "residual_initialization": "zero_residual_cold_start"}
        # Fixed evenly-spaced historical blocks keep the scenario selection
        # deterministic, cover recent history, and avoid jitter between calls.
        selected = np.asarray(candidates)[np.round(np.linspace(0, len(candidates) - 1, count)).astype(int)]
        max_end = 0
        init = False
        for s, hist_day in enumerate(selected):
            start = int(hist_day) * 144 + phase
            end = start + horizon
            historical_prediction = self._historical_record(int(hist_day))[phase:phase + horizon].copy()
            if use_official:
```

```
            historical_prediction[:, 1], _, _ = self._official_curve(
                start, horizon, allowed, historical_prediction[:, 1])
        actual = self._actual(start, end)
        errors[s, :, :2] = actual[:, :2] - historical_prediction[:, :2]
        errors[s, :, 2] = actual[:, 2] - np.log(np.maximum(historical_prediction[:, 2], 1e-6))
        max_end = max(max_end, end)
        init |= self._record_sources[int(hist_day)] == "causal_naive_initialization"
    return errors, {"residual_block_count": len(np.unique(selected)),
                    "residual_actual_end_exclusive": max_end,
                    "residual_initialization": "contains_causal_naive_initialization" if init else "
recorded_origin_predictions"}

def forecast(self, origin: int, horizon: int, use_pv_forecasts: bool = False,
             variable_price: bool = False, scenario_count: int = 12,
             allowed_issues: tuple[int, ...] | None = None) -> ForecastResult:
    origin, horizon, scenario_count = int(origin), int(horizon), int(scenario_count)
    if not 0 <= origin <= self.data.n_steps:
        raise ValueError("origin outside observed year")
    if not 1 <= horizon <= 288 or scenario_count < 1:
        raise ValueError("horizon must be 1..288 and scenario_count must be positive")
    allowed = (0, 6, 12, 18) if allowed_issues is None else tuple(sorted(set(allowed_issues)))
    if not set(allowed).issubset({0, 6, 12, 18}):
        raise ValueError("allowed_issues must contain only 0, 6, 12, 18")
    self._set_price_mode(bool(variable_price))
    day, phase = divmod(origin, 144)
    point = self._ensure_day(day)[phase:phase + horizon].copy()
    latest_issue = None
    if use_pv_forecasts:
        point[:, 1], latest_issue, _ = self._official_curve(origin, horizon, allowed, point[:, 1])
    if not variable_price:
        point[:, 2] = self.data.prior_price[(origin + np.arange(horizon)) % 144]
    errors, residual_audit = self._residuals(origin, horizon, scenario_count, use_pv_forecasts, allowed)
    loads = np.maximum(point[None, :, 0] + errors[:, :, 0], 0)
    pvs = np.maximum(point[None, :, 1] + errors[:, :, 1], 0)
    prices = (np.exp(np.clip(np.log(point[None, :, 2]) + errors[:, :, 2], -9, 3))
              if variable_price else np.broadcast_to(point[None, :, 2], (scenario_count, horizon)).copy
())
    provenance = {
        "origin": origin, "actual_history_end": origin,
        "actual_history_end_exclusive": origin,
        "maximum_actual_index": origin - 1,
        "training_actual_end_exclusive": day * 144,
        "forecast_issue": latest_issue,
        "forecast_issue_origin": latest_issue,
        "allowed_issues": list(allowed), "family": self.family,
        "horizon": horizon, "scenario_count": scenario_count,
        "future_price_known": not variable_price,
        "residual_horizon_policy": "same historical midnight origin predicts entire block; official
residual uses historical query origin",
        "historical_price_source": "attachment4" if variable_price else "attachment1_fixed_daily_profile
",
        **residual_audit,
    }
    if residual_audit["residual_actual_end_exclusive"] > origin or (latest_issue is not None and
latest_issue > origin):
        raise AssertionError("causal information boundary violated")
    self._calls += 1; self._max_actual_end = max(self._max_actual_end, origin)
    return ForecastResult((loads - pvs) / 6, prices, point[:, 0], point[:, 1], point[:, 2], provenance)

```

```
    def diagnostics(self) -> dict[str, Any]:
        return {"family": self.family, "seed": self.seed, "forecast_calls": self._calls,
                "maximum_actual_history_end_exclusive": self._max_actual_end,
                "daily_fits": self.audit_records,
                "historical_record_count": len(self._records),
                "naive_initialization_record_count": sum(v == "causal_naive_initialization" for v in self._record_sources.values()),
                "interval_information_policy": "actual[:origin]; next interval actual excluded",
                "residual_policy": "whole past blocks mature before current origin; joint additive power and log-price errors"}
```

src/microgrid/optimize.py

```
"""Sparse reference scheduling; operational safety feedback is implemented elsewhere.

All flows are kWh per interval. Scenarios share a reference battery trajectory;
only emergency supply differs. This is not a full multistage stochastic policy.
"""

import time
from typing import Optional

import numpy as np
from scipy.optimize import Bounds, LinearConstraint, linprog, milp as scipy_milp
from scipy.sparse import coo_matrix, vstack


SOC_MIN = 1200.0
SOC_MAX = 10800.0
MAX_INTERVAL_ENERGY = 5000.0 / 6.0


class OptimizationError(RuntimeError):
    """The solver failed to certify an optimum or returned an invalid schedule."""

    def __init__(self, message, status=None, diagnostics=None):
        super().__init__(message)
        self.status = status
        self.diagnostics = diagnostics or {}


def adjustment_cost(q, original, price, settlement="adjusted"):
    """Total final settlement against the ORIGINAL midnight commitment.

    This includes the original purchase charge, not just the adjustment charge.
    Revisions are never summed against successive intermediate commitments.
    """
    if settlement not in ("adjusted", "full_plan_plus_penalty"):
        raise ValueError("settlement must be 'adjusted' or 'full_plan_plus_penalty'")
    q, original, price = np.broadcast_arrays(q, original, price)
    down_sign = -1.0 if settlement == "adjusted" else 1.0
    return price * (
        original + 1.5 * np.maximum(q - original, 0.0)
        + down_sign * 0.5 * np.maximum(original - q, 0.0)
    )


def recover_exclusive_dispatch(c, d, eta=0.9):
```

```
    """Remove simultaneous charge/discharge without changing the SOC path.

    AC demand decreases; unrestricted, free unused energy makes recovery feasible.
    The caller must recompute emergency supply and the objective after recovery.
    """
    if not 0.0 < eta <= 1.0:
        raise ValueError("eta must be in (0, 1]")
    c = np.maximum(np.asarray(c, dtype=float), 0.0).copy()
    d = np.maximum(np.asarray(d, dtype=float), 0.0).copy()
    if c.shape != d.shape:
        raise ValueError("charge and discharge must have the same shape")
    delta = np.minimum(c, d / (eta * eta))
    return np.maximum(c - delta, 0.0), np.maximum(d - eta * eta * delta, 0.0)


def _vector(value, horizon, name, required=False):
    if value is None:
        if required:
            raise ValueError("{} is required for the requested mode".format(name))
        return None
    result = np.asarray(value, dtype=float)
    if result.shape != (horizon,):
        raise ValueError("{} must have shape ({},)".format(name, horizon))
    return result


def solve_schedule(
    net_s: np.ndarray,
    price_s: np.ndarray,
    initial_soc: float,
    purchase: Optional[np.ndarray] = None,
    original: Optional[np.ndarray] = None,
    adjust_mask: Optional[np.ndarray] = None,
    terminal_soc: Optional[float] = None,
    eta: float = 0.9,
    capacity: bool = True,
    settlement: str = "adjusted",
    milp: bool = False,
    mode: Optional[np.ndarray] = None,
    allow_emergency: bool = True,
    time_limit: Optional[float] = 60.0,
):
    """Return an optimal finite-horizon reference schedule or raise an error.

    ``net_s`` is load minus PV, shape (K,H); ``price_s`` is (K,H) or (H,).
    Scenarios are equiprobable. ``mode`` optionally specifies 'free', 'fixed',
    or 'adjust' per interval. Without it, purchase=None means free purchases;
    a purchase vector means fixed purchases except where adjust_mask is true.
    An adjust interval is settled against ``original``. Other intervals use
    ordinary p*q, including fixed intervals (constant for this optimization).
    Irrelevant entries of purchase/original may be NaN; used entries must be
    finite and nonnegative. capacity=False disables the battery. Terminal SOC
    is an optional hard constraint. Time-limited incumbents are never silently
    accepted as optimal. Negative electricity prices need a different bounded
    market formulation and are explicitly rejected by this reference model.
    """
    start = time.perf_counter()
    net = np.asarray(net_s, dtype=float)
    if net.ndim == 1:
```

```
        net = net[None, :]
    if net.ndim != 2 or not net.shape[0] or not net.shape[1]:
        raise ValueError("net_s must be a nonempty (K,H) array")
    k_count, horizon = net.shape
    try:
        prices = np.broadcast_to(np.asarray(price_s, dtype=float), net.shape)
    except ValueError as exc:
        raise ValueError("price_s must broadcast to the shape of net_s") from exc
    if not np.all(np.isfinite(net)) or not np.all(np.isfinite(prices)):
        raise ValueError("net_s and price_s must contain finite values")
    if np.any(prices < 0):
        raise ValueError("negative prices require bounded purchases and a different recourse formulation")
    if not 0.0 < eta <= 1.0:
        raise ValueError("eta must be in (0, 1]")
    if not isinstance(capacity, (bool, np.bool_)):
        raise ValueError("capacity is a boolean battery-enable switch")
    if settlement not in ("adjusted", "full_plan_plus_penalty"):
        raise ValueError("settlement must be 'adjusted' or 'full_plan_plus_penalty'")
    if time_limit is not None and (not np.isfinite(time_limit) or time_limit <= 0):
        raise ValueError("time_limit must be positive or None")
    soc_input_tolerance = 1e-7
    if not np.isfinite(initial_soc) or not SOC_MIN - soc_input_tolerance <= initial_soc <= SOC_MAX +
     soc_input_tolerance:
        raise ValueError("initial_soc must be in [1200,10800]")
    initial_soc = float(np.clip(initial_soc, SOC_MIN, SOC_MAX))
    if terminal_soc is not None and (
        not np.isfinite(terminal_soc) or not SOC_MIN - soc_input_tolerance <= terminal_soc <= SOC_MAX +
     soc_input_tolerance
    ):
        raise ValueError("terminal_soc must be in [1200,10800]")
    if terminal_soc is not None:
        terminal_soc = float(np.clip(terminal_soc, SOC_MIN, SOC_MAX))

    planned = _vector(purchase, horizon, "purchase")
    midnight = _vector(original, horizon, "original")
    if mode is None:
        modes = np.full(horizon, "free" if planned is None else "fixed", dtype="U6")
        if adjust_mask is not None:
            mask = np.asarray(adjust_mask, dtype=bool)
            if mask.shape != (horizon,):
                raise ValueError("adjust_mask must have shape (H,)")
            modes[mask] = "adjust"
    else:
        modes = np.asarray(mode, dtype="U6")
        if modes.shape != (horizon,) or not np.all(np.isin(modes, ["free", "fixed", "adjust"])):
            raise ValueError("mode must contain H values: free, fixed, or adjust")
        if adjust_mask is not None:
            mask = np.asarray(adjust_mask, dtype=bool)
            if mask.shape != (horizon,) or not np.array_equal(mask, modes == "adjust"):
                raise ValueError("adjust_mask conflicts with explicit mode")
    fixed = modes == "fixed"
    adjusted = modes == "adjust"
    if np.any(fixed):
        if planned is None or np.any(~np.isfinite(planned[fixed])) or np.any(planned[fixed] < 0):
            raise ValueError("fixed intervals require finite nonnegative purchase values")
    if np.any(adjusted):
        if midnight is None or np.any(~np.isfinite(midnight[adjusted])) or np.any(midnight[adjusted] < 0):
            raise ValueError("adjust intervals require finite nonnegative original values")

```

```
    # q, c, d, S[0:H+1], emergency[K,H], up[A], down[A], binary[H].
    q_idx = np.arange(horizon)
    c_idx = q_idx + horizon
    d_idx = q_idx + 2 * horizon
    soc_idx = np.arange(horizon + 1) + 3 * horizon
    emergency_idx = np.arange(k_count * horizon).reshape(k_count, horizon) + 4 * horizon + 1
    offset = 4 * horizon + 1 + k_count * horizon
    adj_times = np.flatnonzero(adjusted)
    up_idx = np.arange(adj_times.size) + offset
    down_idx = up_idx + adj_times.size
    offset += 2 * adj_times.size
    binary_idx = np.arange(horizon) + offset if milp and capacity else np.array([], dtype=int)
    nvars = offset + binary_idx.size

    objective = np.zeros(nvars)
    mean_price = prices.mean(axis=0)
    objective[q_idx[~adjusted]] = mean_price[~adjusted]
    objective[emergency_idx.ravel()] = (5.0 * prices / k_count).ravel()
    objective[up_idx] = 1.5 * mean_price[adjusted]
    objective[down_idx] = (-0.5 if settlement == "adjusted" else 0.5) * mean_price[adjusted]
    constant_cost = float(np.dot(mean_price[adjusted], midnight[adjusted])) if np.any(adjusted) else 0.0
    lower = np.zeros(nvars)
    upper = np.full(nvars, np.inf)
    upper[c_idx] = upper[d_idx] = MAX_INTERVAL_ENERGY if capacity else 0.0
    lower[soc_idx], upper[soc_idx] = SOC_MIN, SOC_MAX
    lower[soc_idx[0]] = upper[soc_idx[0]] = initial_soc
    if terminal_soc is not None:
        lower[soc_idx[-1]] = upper[soc_idx[-1]] = terminal_soc
    if np.any(fixed):
        lower[q_idx[fixed]] = upper[q_idx[fixed]] = planned[fixed]
    if not allow_emergency:
        upper[emergency_idx.ravel()] = 0.0
    if binary_idx.size:
        upper[binary_idx] = 1.0

    eq_rows, eq_cols, eq_values, eq_rhs = [], [], [], []
    ub_rows, ub_cols, ub_values, ub_rhs = [], [], [], []

    def add_eq(cols, values, rhs):
        eq_rows.extend([len(eq_rhs)] * len(cols))
        eq_cols.extend(cols)
        eq_values.extend(values)
        eq_rhs.append(rhs)

    def add_ub(cols, values, rhs):
        ub_rows.extend([len(ub_rhs)] * len(cols))
        ub_cols.extend(cols)
        ub_values.extend(values)
        ub_rhs.append(rhs)

    for t in range(horizon):
        add_eq([soc_idx[t + 1], soc_idx[t], c_idx[t], d_idx[t]], [1.0, -1.0, -eta, 1.0 / eta], 0.0)
    for j, t in enumerate(adj_times):
        add_eq([q_idx[t], up_idx[j], down_idx[j]], [1.0, -1.0, 1.0], midnight[t])
    for s in range(k_count):
        for t in range(horizon):
            add_ub([c_idx[t], d_idx[t], q_idx[t], emergency_idx[s, t]], [1.0, -1.0, -1.0, -1.0], -net[s, t])
    if binary_idx.size:
        for t in range(horizon):
```

```
            add_ub([c_idx[t], binary_idx[t]], [1.0, -MAX_INTERVAL_ENERGY], 0.0)
            add_ub([d_idx[t], binary_idx[t]], [1.0, MAX_INTERVAL_ENERGY], MAX_INTERVAL_ENERGY)
    a_eq = coo_matrix((eq_values, (eq_rows, eq_cols)), shape=(len(eq_rhs), nvars)).tocsr()
    a_ub = coo_matrix((ub_values, (ub_rows, ub_cols)), shape=(len(ub_rhs), nvars)).tocsr()
    b_eq, b_ub = np.asarray(eq_rhs), np.asarray(ub_rhs)
    options = {"presolve": True}
    if time_limit is not None:
        options["time_limit"] = float(time_limit)
    if milp:
        integrality = np.zeros(nvars, dtype=int)
        integrality[binary_idx] = 1
        constraints = LinearConstraint(
            vstack([a_eq, a_ub], format="csc"),
            np.concatenate([b_eq, np.full(len(b_ub), -np.inf)]),
            np.concatenate([b_eq, b_ub]),
        )
        options["mip_rel_gap"] = 1e-8
        result = scipy_milp(objective, integrality=integrality, bounds=Bounds(lower, upper), constraints=
     constraints, options=options)
    else:
        options.update(primal_feasibility_tolerance=1e-8, dual_feasibility_tolerance=1e-8)
        result = linprog(objective, A_ub=a_ub, b_ub=b_ub, A_eq=a_eq, b_eq=b_eq, bounds=np.column_stack([
     lower, upper]), method="highs", options=options)
    diagnostics = {
        "solver": "HiGHS MILP" if milp else "HiGHS LP",
        "solver_status": int(result.status),
        "message": str(result.message),
        "elapsed_seconds": time.perf_counter() - start,
        "scenarios": k_count,
        "horizon": horizon,
    }
    if result.status != 0 or result.x is None:
        diagnostics["has_incumbent"] = result.x is not None
        raise OptimizationError("Reference optimization failed: {}".format(result.message), result.status,
     diagnostics)

    x = result.x
    q = np.maximum(x[q_idx], 0.0)
    c, d = recover_exclusive_dispatch(x[c_idx], x[d_idx], eta)
    soc = np.r_[float(initial_soc), initial_soc + np.cumsum(eta * c - d / eta)]
    emergency = np.maximum(net + c[None, :] - d[None, :] - q[None, :], 0.0)
    spill = q[None, :] + d[None, :] + emergency - net - c[None, :]
    costs = mean_price * q
    if np.any(adjusted):
        costs[adjusted] = adjustment_cost(q[adjusted], midnight[adjusted], mean_price[adjusted], settlement)
    actual_objective = float(costs.sum() + np.mean(np.sum(5.0 * prices * emergency, axis=1)))
    raw_objective = float(result.fun + constant_cost)
    tolerance = 2e-5
    violations = {
        "soc_lower": float(max(0.0, SOC_MIN - soc.min())),
        "soc_upper": float(max(0.0, soc.max() - SOC_MAX)),
        "charge_limit": float(max(0.0, c.max() - (MAX_INTERVAL_ENERGY if capacity else 0.0))),
        "discharge_limit": float(max(0.0, d.max() - (MAX_INTERVAL_ENERGY if capacity else 0.0))),
        "terminal": 0.0 if terminal_soc is None else float(abs(soc[-1] - terminal_soc)),
        "fixed_purchase": 0.0 if not np.any(fixed) else float(np.max(np.abs(q[fixed] - planned[fixed]))),
        "forbidden_emergency": 0.0 if allow_emergency else float(emergency.max()),
        "simultaneous": float(np.minimum(c, d).max()),
        "negative_spill": float(max(0.0, -spill.min())),
    }
```

```
    diagnostics.update(raw_objective=raw_objective, recovered_objective=actual_objective, max_violation=max(
     violations.values()), violations=violations)
    if max(violations.values()) > tolerance:
        raise OptimizationError("Recovered reference schedule violates physical constraints", result.status,
      diagnostics)
    if abs(actual_objective - raw_objective) > 1e-4 + 1e-7 * max(1.0, abs(raw_objective)):
        raise OptimizationError("Recovered objective does not match the solver certificate", result.status,
     diagnostics)
    gap = float(getattr(result, "mip_gap", 0.0) or 0.0)
    return {
        "q": q, "c": c, "d": d, "soc": soc,
        "emergency": emergency, "spill": spill,
        "objective": actual_objective, "gap": gap, "status": "optimal",
        "diagnostics": diagnostics,
    }
```

src/microgrid/quantile.py

```
"""Question 2 primary policy: causal rolling forecasts and quantile contracts.

The 48-hour linear program is used only for the day-ahead purchase contract.
During operation the current ten-minute observation is handled by a transparent
safety feedback: charge realized surplus, or discharge before emergency supply.
No unexecuted intraday MPC path is part of this implementation.
"""
from __future__ import annotations

from dataclasses import asdict
import gzip
import json
from pathlib import Path
import time

import numpy as np
from scipy.optimize import linprog
from scipy.sparse import csr_matrix, eye, hstack, lil_matrix, vstack

from .data import load_data
from .simulate import (CAP_MAX, CAP_MIN, INITIAL, MAX_ENERGY, STEPS, Policy,
                       settlement_components, summarize, write_json)


class JointForecaster:
    """Forecast load and PV using only observations before the operating day."""

    def __init__(self, load, pv, prior_load, prior_pv,
                 validation_days=14, residual_days=30):
        self.load = np.asarray(load, dtype=float)
        self.pv = np.asarray(pv, dtype=float)
        self.prior_load = np.asarray(prior_load, dtype=float)
        self.prior_pv = np.asarray(prior_pv, dtype=float)
        self.validation_days = validation_days
        self.residual_days = residual_days
        self._weights = {}
        self._forecast = {}

    def bases(self, values, day):
        if day <= 0:
```

```
            prior = self.prior_load if values is self.load else self.prior_pv
            return np.vstack([prior, prior, prior])
        previous = values[day - 1]
        if day < 7:
            return np.vstack([previous, previous, np.median(values[:day], axis=0)])
        return np.vstack([previous, values[day - 7],
                          np.median(values[day - 7:day], axis=0)])

    def weights(self, day):
        if day in self._weights:
            return tuple(value.copy() for value in self._weights[day])
        if day < 8:
            answer = (np.array([1., 0., 0.]), np.array([1., 0., 0.]))
            self._weights[day] = answer
            return tuple(value.copy() for value in answer)
        start = max(0, day - self.validation_days)
        load_basis, pv_basis, target = [], [], []
        for sample in range(start, day):
            load_basis.append(self.bases(self.load, sample).T)
            pv_basis.append(self.bases(self.pv, sample).T)
            target.append(self.load[sample] - self.pv[sample])
        load_basis = np.vstack(load_basis)
        pv_basis = np.vstack(pv_basis)
        target = np.concatenate(target)
        observations = len(target)
        prediction = hstack([csr_matrix(load_basis), -csr_matrix(pv_basis)], format="csr")
        identity = eye(observations, format="csr")
        inequality = vstack([hstack([prediction, -identity]),
                             hstack([-prediction, -identity])], format="csr")
        equality = lil_matrix((2, 6 + observations))
        equality[0, :3] = 1
        equality[1, 3:6] = 1
        solution = linprog(np.r_[np.zeros(6), np.ones(observations)],
                           A_ub=inequality, b_ub=np.r_[target, -target],
                           A_eq=equality.tocsr(), b_eq=[1, 1],
                           bounds=[(0, 1)] * 6 + [(0, None)] * observations,
                           method="highs")
        if solution.success:
            load_weights, pv_weights = solution.x[:3], solution.x[3:6]
            load_weights /= load_weights.sum()
            pv_weights /= pv_weights.sum()
        else:
            load_weights = pv_weights = np.array([1., 0., 0.])
        self._weights[day] = (load_weights, pv_weights)
        return load_weights.copy(), pv_weights.copy()

    def pair(self, day):
        if day not in self._forecast:
            load_weights, pv_weights = self.weights(day)
            load = np.maximum(load_weights @ self.bases(self.load, day), 0)
            pv = np.maximum(pv_weights @ self.bases(self.pv, day), 0)
            self._forecast[day] = (load, pv, load_weights, pv_weights)
        return tuple(value.copy() for value in self._forecast[day])

    def tomorrow(self, day, today_load, today_pv, load_weights, pv_weights):
        if day + 1 >= len(self.load):
            return today_load.copy(), today_pv.copy()

        def future_bases(values, today_prediction):
```

```
            weekly = values[day - 6] if day >= 6 else values[max(day - 1, 0)]
            known = [values[j] for j in range(max(0, day - 6), day)] + [today_prediction]
            return np.vstack([today_prediction, weekly, np.median(known, axis=0)])

        return (np.maximum(load_weights @ future_bases(self.load, today_load), 0),
                np.maximum(pv_weights @ future_bases(self.pv, today_pv), 0))

    def residual_margin(self, day, quantile):
        start = max(0, day - self.residual_days)
        residuals = []
        for sample in range(start, day):
            load, pv, _, _ = self.pair(sample)
            residuals.append((self.load[sample] - self.pv[sample]) - (load - pv))
        if not residuals:
            return np.zeros(STEPS)
        residuals = np.asarray(residuals)
        slot = np.quantile(residuals, quantile, axis=0)
        margin = np.empty(STEPS)
        for hour in range(24):
            section = slice(6 * hour, 6 * (hour + 1))
            margin[section] = (slot[section] +
                               np.quantile(residuals[:, section], quantile)) / 2
        return margin


def solve_contract_48h(net, price, initial_soc, eta=.9, storage=True):
    """Minimize 48-hour contract cost and restore 6000 kWh at horizon end."""
    net = np.asarray(net, dtype=float)
    price = np.asarray(price, dtype=float)
    if not storage:
        return np.maximum(net, 0), np.full(len(net) + 1, initial_soc)
    count = len(net)
    grid, charge, discharge, waste, state = 0, count, 2 * count, 3 * count, 4 * count
    variables = 5 * count + 1
    objective = np.zeros(variables)
    objective[grid:grid + count] = price
    objective[charge:charge + count] = 1e-7
    objective[discharge:discharge + count] = 1e-7
    objective[waste:waste + count] = 1e-8
    equality = lil_matrix((2 * count + 2, variables))
    target = np.zeros(2 * count + 2)
    for step in range(count):
        equality[step, grid + step] = 1
        equality[step, discharge + step] = 1
        equality[step, charge + step] = -1
        equality[step, waste + step] = -1
        target[step] = net[step]
        row = count + step
        equality[row, state + step + 1] = 1
        equality[row, state + step] = -1
        equality[row, charge + step] = -eta
        equality[row, discharge + step] = 1 / eta
    equality[2 * count, state] = 1
    target[2 * count] = initial_soc
    equality[2 * count + 1, state + count] = 1
    target[2 * count + 1] = INITIAL
    bounds = ([(0, None)] * count + [(0, MAX_ENERGY)] * count * 2 +
              [(0, None)] * count + [(CAP_MIN, CAP_MAX)] * (count + 1))
    solution = linprog(objective, A_eq=equality.tocsr(), b_eq=target,
```

```
                      bounds=bounds, method="highs")
    if not solution.success:
        raise RuntimeError("quantile contract LP failed: " + solution.message)
    return solution.x[grid:grid + count], solution.x[state:state + count + 1]


def simulate_quantile_q2(quantile=.8, data=None, end_day=365, output_dir=None,
                         name="q2", use_margin=True, storage=True, verbose=True):
    """Run the unique official Q2 primary implementation and common ledger."""
    if not 0 < quantile < 1:
        raise ValueError("quantile must lie strictly between zero and one")
    data = data or load_data()
    count = end_day * STEPS
    actual_load = data.load[:end_day] / 6
    actual_pv = data.pv[:end_day] / 6
    fixed_price = np.asarray(data.prior_price, dtype=float)
    forecaster = JointForecaster(actual_load, actual_pv,
                                 data.prior_load / 6, data.prior_pv / 6)
    keys = ["plan", "effective", "charge", "discharge", "emergency", "waste",
            "soc_start", "soc_end", "price", "load", "pv", "base_fee", "up_fee",
            "down_fee", "emergency_fee", "cost", "target_charge", "target_discharge",
            "issue_index", "expected_net", "expected_price"]
    log = {key: np.zeros(count) for key in keys}
    state = INITIAL
    start_time = time.perf_counter()
    maximum_balance = 0.
    forecast_error = {"load": 0., "pv": 0., "price": 0., "count": 0}
    fits = []
    for day in range(end_day):
        load_forecast, pv_forecast, load_weights, pv_weights = forecaster.pair(day)
        margin = forecaster.residual_margin(day, quantile) if use_margin else np.zeros(STEPS)
        today_net = np.maximum(load_forecast - pv_forecast + margin, 0)
        next_load, next_pv = forecaster.tomorrow(day, load_forecast, pv_forecast,
                                                 load_weights, pv_weights)
        tomorrow_net = np.maximum(next_load - next_pv + margin, 0)
        plan48, _ = solve_contract_48h(np.r_[today_net, tomorrow_net],
                                       np.tile(fixed_price, 2), state,
                                       storage=storage)
        plan = plan48[:STEPS]
        origin = day * STEPS
        fits.append({"day": day, "training_actual_end_exclusive": origin,
                     "validation_days": min(14, day),
                     "residual_days": min(30, day),
                     "load_weights": load_weights.tolist(),
                     "pv_weights": pv_weights.tolist()})
        for slot in range(STEPS):
            index = origin + slot
            load = actual_load[day, slot]
            pv = actual_pv[day, slot]
            available = plan[slot] + pv - load
            if storage and available >= 0:
                charge = min(available, MAX_ENERGY, max(0., (CAP_MAX - state) / .9))
                discharge = emergency = 0.
                waste = max(available - charge, 0.)
            elif storage:
                charge = waste = 0.
                discharge = min(-available, MAX_ENERGY, max(0., .9 * (state - CAP_MIN)))
                emergency = max(-available - discharge, 0.)
            else:
```

```
                charge = discharge = 0.
                emergency = max(-available, 0.)
                waste = max(available, 0.)
            next_state = state + .9 * charge - discharge / .9
            fees = settlement_components(plan[slot], plan[slot], fixed_price[slot], emergency)
            row = [plan[slot], plan[slot], charge, discharge, emergency, waste,
                   state, next_state, fixed_price[slot], load, pv, *fees, sum(fees),
                   charge, discharge, origin, load_forecast[slot] - pv_forecast[slot],
                   fixed_price[slot]]
            for key, value in zip(keys, row):
                log[key][index] = value
            residual = plan[slot] + pv + discharge + emergency - load - charge - waste
            maximum_balance = max(maximum_balance, abs(float(residual)))
            state = float(np.clip(next_state, CAP_MIN, CAP_MAX))
        if day >= 31:
            forecast_error["load"] += float(np.abs(data.load[day] - load_forecast * 6).sum())
            forecast_error["pv"] += float(np.abs(data.pv[day] - pv_forecast * 6).sum())
            forecast_error["price"] += 0.
            forecast_error["count"] += STEPS
        if verbose and (day < 2 or (day + 1) % 30 == 0 or day + 1 == end_day):
            print(f"{name} day {day + 1}/{end_day} q={quantile:.2f} SOC={state:.2f} ",
                  f"elapsed={time.perf_counter() - start_time:.1f}s", flush=True)
    policy = Policy(name=name, scenarios=1, control_hours=0, storage=storage,
                    family="joint_causal_quantile")
    summary = summarize(log, policy, data)
    configuration = asdict(policy)
    configuration.update(model="rolling_quantile", quantile=quantile,
                         quantile_margin=bool(use_margin), feedback="immediate_safe")
    summary.update(elapsed_seconds=time.perf_counter() - start_time,
                   solver_calls=end_day, maximum_balance_error_kwh=maximum_balance,
                   forecast_mae={key: value / max(1, forecast_error["count"])
                                 for key, value in forecast_error.items() if key != "count"},
                   configuration=configuration)
    if output_dir:
        destination = Path(output_dir) / name
        destination.mkdir(parents=True, exist_ok=True)
        np.savez_compressed(destination / "ledger.npz", **log)
        write_json(destination / "summary.json", summary)
        with gzip.open(destination / "revisions.json.gz", "wt", encoding="utf-8") as stream:
            json.dump([], stream, separators=(",", ":"))
        write_json(destination / "forecast_diagnostics.json",
                   {"joint_causal_quantile": {"model": "joint_causal_quantile",
                                              "daily_fits": fits}})
    return log, summary
```

src/microgrid/simulate.py

```
"""Chronological simulation. Only the physical safety layer sees current demand."""
from dataclasses import dataclass, asdict
from pathlib import Path
import json
import time
import numpy as np
import pandas as pd

from .data import load_data
from .forecast import ForecastEngine
from .optimize import solve_schedule
```

```
STEPS = 144
CAP_MIN, CAP_MAX, INITIAL = 1200., 10800., 6000.
MAX_ENERGY = 5000. / 6.


@dataclass
class Policy:
    name: str = 'q2'
    pv_forecasts: bool = False
    variable_price: bool = False
    updates: tuple = ()
    scenarios: int = 12
    control_hours: int = 4
    eta: float = .9
    storage: bool = True
    settlement: str = 'adjusted'
    family: str = 'naive'
    warmup_naive: bool = True
    perfect_price: bool = False
    seed: int = 2026


def settlement_components(original, effective, price, emergency, settlement='adjusted'):
    """Settle the final commitment against the midnight plan once, at delivery prices."""
    if settlement not in ('adjusted', 'full_plan_plus_penalty'):
        raise ValueError('settlement must be adjusted or full_plan_plus_penalty')
    original, effective, price, emergency = map(np.asarray, (original, effective, price, emergency))
    base = price * original
    upward = 1.5 * price * np.maximum(effective - original, 0.)
    downward = (-.5 if settlement == 'adjusted' else .5) * price * np.maximum(original - effective, 0.)
    urgent = 5. * price * emergency
    return base, upward, downward, urgent


def execute_safe(q, load, pv, c_target, d_target, soc, eta=.9, storage=True):
    """Interval-internal load-priority tracking under piecewise-constant power.

    load and pv are realized interval energies. They never change the pre-interval
    optimizer or purchase commitment. Emergency supply cannot charge the battery.
    """
    available = q + pv - load
    if storage:
        c = min(max(c_target, 0.), max(available, 0.), MAX_ENERGY, max(0., (CAP_MAX-soc)/eta))
        d = min(max(d_target, 0.), max(-available, 0.), MAX_ENERGY, max(0., eta*(soc-CAP_MIN)))
    else:
        c = d = 0.
    urgent = max(-available-d, 0.)
    waste = max(available-c, 0.)
    end = soc + eta*c - d/eta
    return c, d, urgent, waste, end


def _json_default(value):
    if isinstance(value, np.generic):
        return value.item()
    if isinstance(value, np.ndarray):
        return value.tolist()
    if isinstance(value, Path):
```

```
        return str(value)
    raise TypeError(type(value).__name__)


def write_json(path, value):
    Path(path).write_text(json.dumps(value, ensure_ascii=False, indent=2, default=_json_default), encoding='utf-8')


def base_policies(families=None):
    families = families or {}
    return [
        Policy('q2', family=families.get('q2', 'naive')),
        Policy('q3', pv_forecasts=True, updates=(6,12,18), family=families.get('q3', 'naive')),
        Policy('q4_2', variable_price=True, family=families.get('q4_2', 'naive')),
        Policy('q4_3', pv_forecasts=True, variable_price=True, updates=(6,12,18), family=families.get('q4_3', 'naive')),
    ]


def simulate(policy, data=None, end_day=365, output_dir=None, verbose=True):
    data = data or load_data()
    n = end_day * STEPS
    runtime = time.perf_counter()
    engines = {family: ForecastEngine(data, family=family, seed=policy.seed)
               for family in set([policy.family, 'naive'])}
    allowed = tuple(sorted(set((0,) + tuple(policy.updates)))) if policy.pv_forecasts else None
    keys = ['plan', 'effective', 'charge', 'discharge', 'emergency', 'waste',
            'soc_start', 'soc_end', 'price', 'load', 'pv', 'base_fee', 'up_fee',
            'down_fee', 'emergency_fee', 'cost', 'target_charge', 'target_discharge',
            'issue_index', 'expected_net', 'expected_price']
    log = {k: np.zeros(n) for k in keys}
    revisions = []
    soc = INITIAL
    max_eq_residual = 0.
    solver_calls = 0
    forecast_mae = {'load': 0., 'pv': 0., 'price': 0., 'count': 0}

    def predict(engine, origin, horizon):
        pred = engine.forecast(origin=origin, horizon=horizon,
                               use_pv_forecasts=policy.pv_forecasts,
                               variable_price=policy.variable_price,
                               scenario_count=policy.scenarios,
                               allowed_issues=allowed)
        net_s = np.array(pred.net_s, copy=True)
        price_s = np.array(pred.price_s, copy=True)
        if policy.perfect_price:
            # Explicit information-ablation only; never used by a main policy.
            stop = min(origin+horizon, len(data.flat_price))
            price_s[:, :stop-origin] = data.flat_price[origin:stop]
        return pred, net_s, price_s

    for day in range(end_day):
        origin = day * STEPS
        family = 'naive' if policy.warmup_naive and day < 31 else policy.family
        engine = engines[family]
        pred, net_s, price_s = predict(engine, origin, 288)
        ref = solve_schedule(net_s, price_s, initial_soc=soc, terminal_soc=INITIAL,
                             eta=policy.eta, capacity=policy.storage)
```

```
        solver_calls += 1
        plan = ref['q'][:STEPS].copy()
        effective = plan.copy()
        reference_q = ref['q'].copy()
        reference_soc = ref['soc'].copy()
        last_issue = origin
        midnight_mean = (pred.mean_load[:STEPS], pred.mean_pv[:STEPS], pred.mean_price[:STEPS])
        for slot in range(STEPS):
            t = origin + slot
            if slot and slot % 36 == 0 and slot//6 in policy.updates:
                remaining = STEPS-slot
                pred_new, net_new, price_new = predict(engine, t, 288-slot)
                mode = np.array(['adjust']*remaining + ['free']*144)
                original = np.r_[plan[slot:], np.zeros(144)]
                current = np.r_[effective[slot:], reference_q[STEPS:]]
                revised = solve_schedule(net_new, price_new, initial_soc=soc,
                                         purchase=current, original=original,
                                         mode=mode, terminal_soc=INITIAL,
                                         eta=policy.eta, capacity=policy.storage,
                                         settlement=policy.settlement)
                solver_calls += 1
                effective[slot:] = revised['q'][:remaining]
                reference_q[slot:] = revised['q']
                reference_soc[slot:] = revised['soc']
                last_issue = t
                # Archive the whole committed remaining-day vector, not a re-settlement.
                revisions.append({'issue': int(t), 'day': int(day), 'slot': int(slot),
                                  'q': np.round(effective[slot:], 9).tolist()})
            horizon = policy.control_hours * 6
            pred_rt, net_rt, price_rt = predict(engine, t, horizon)
            q_fixed = reference_q[slot:slot+horizon].copy()
            reference_end = float(np.clip(reference_soc[slot+horizon],
                                         max(CAP_MIN,soc-horizon*MAX_ENERGY/policy.eta),
                                         min(CAP_MAX,soc+horizon*MAX_ENERGY*policy.eta)))
            if policy.storage:
                control = solve_schedule(net_rt, price_rt, initial_soc=soc,
                                         purchase=q_fixed, terminal_soc=reference_end,
                                         eta=policy.eta, capacity=True)
                solver_calls += 1
            else:
                control = {'c':np.zeros(horizon), 'd':np.zeros(horizon)}
            q = effective[slot]
            load, pv = data.load[day, slot]/6., data.pv[day, slot]/6.
            p = data.price[day, slot] if policy.variable_price else data.prior_price[slot]
            c, d, u, w, next_soc = execute_safe(q, load, pv, control['c'][0],
                                                control['d'][0], soc, policy.eta, policy.storage)
            fees = settlement_components(plan[slot], q, p, u, policy.settlement)
            row = [plan[slot], q, c, d, u, w, soc, next_soc, p, load, pv,
                   *fees, sum(fees), control['c'][0], control['d'][0], last_issue,
                   float(np.mean(net_rt[:, 0])), float(np.mean(price_rt[:, 0]))]
            for key, value in zip(keys, row):
                log[key][t] = value
            residual = q+pv+d+u-load-c-w
            max_eq_residual = max(max_eq_residual, abs(residual))
            if not (CAP_MIN-1e-6 <= next_soc <= CAP_MAX+1e-6):
                raise AssertionError(('SOC out of bounds', policy.name, t, next_soc))
            if max_eq_residual > 1e-6 or c*d > 1e-8 or c*u > 1e-8:
                raise AssertionError(('physical violation', policy.name, t, residual, c, d, u))
            soc = next_soc
```

```
        for metric, real, estimate in zip(('load', 'pv', 'price'),
                                          (data.load[day], data.pv[day], data.price[day] if policy.variable_price else data.prior_price),
                                          midnight_mean):
            if day >= 31:
                forecast_mae[metric] += float(np.abs(real-estimate).sum())
        if day >= 31:
            forecast_mae['count'] += STEPS
        if verbose and (day < 2 or (day+1) % 14 == 0 or day+1 == end_day):
            print(f'{policy.name} day {day+1}/{end_day} family={family} SOC={soc:.2f} '
                  f'elapsed={time.perf_counter()-runtime:.1f}s', flush=True)
    summary = summarize(log, policy, data)
    summary.update({'elapsed_seconds': time.perf_counter()-runtime, 'solver_calls': solver_calls,
                    'maximum_balance_error_kwh': max_eq_residual,
                    'forecast_mae': {k: v/max(1,forecast_mae['count']) for k,v in forecast_mae.items() if k != 'count'},
                    'configuration': asdict(policy)})
    if output_dir:
        dest = Path(output_dir)/policy.name
        dest.mkdir(parents=True, exist_ok=True)
        np.savez_compressed(dest/'ledger.npz', **log)
        write_json(dest/'summary.json', summary)
        import gzip
        with gzip.open(dest/'revisions.json.gz', 'wt', encoding='utf-8') as stream:
            json.dump(revisions, stream, separators=(',', ':'))
        write_json(dest/'forecast_diagnostics.json', {k: v.diagnostics() for k,v in engines.items()})
    return log, summary


def summarize(log, policy, data):
    n = len(log['plan'])
    starts = {'whole_year': 0, 'evaluation': min(31*STEPS, n), 'january_validation': min(14*STEPS, n)}
    valuation = float(np.min(data.prior_price)/.9)
    result = {}
    for label, start in starts.items():
        stop = min(n,31*STEPS) if label == 'january_validation' else n
        if stop <= start:
            result[label] = {}
            continue
        totals = {k: float(log[k][start:stop].sum()) for k in
                  ('plan','effective','charge','discharge','emergency','waste','base_fee','up_fee','down_fee','emergency_fee','cost')}
        initial = float(log['soc_start'][start]); final = float(log['soc_end'][stop-1])
        urgent = log['emergency'][start:stop].reshape(-1, STEPS) > 1e-7
        events = urgent.copy(); events[:,1:] &= ~urgent[:,:-1]
        totals.update(initial_soc=initial, final_soc=final,
                      inventory_corrected_cost=totals['cost'] + valuation*(initial-final),
                      emergency_events=int(events.sum()), emergency_intervals=int(urgent.sum()),
                      soc_min=float(log['soc_end'][start:stop].min()),
                      soc_max=float(log['soc_end'][start:stop].max()), days=(stop-start)//STEPS)
        result[label] = totals
    return result


def solve_question_one(data, destination):
    path = Path(destination); path.mkdir(parents=True, exist_ok=True)
    net = ((data.prior_load-data.prior_pv)/6.)[None,:]
    price = data.prior_price[None,:]
    solved = solve_schedule(net,price,INITIAL,terminal_soc=INITIAL,allow_emergency=False)
```

```
    audited = solve_schedule(net,price,INITIAL,terminal_soc=INITIAL,allow_emergency=False,milp=True)
    np.savez_compressed(path/'ledger.npz', plan=solved['q'],charge=solved['c'],discharge=solved['d'],
                        soc=solved['soc'],price=data.prior_price,load=data.prior_load/6.,pv=data.prior_pv
     /6.)
    output = {'cost':float(data.prior_price@solved['q']), 'energy':float(solved['q'].sum()),
              'lp_objective':solved['objective'], 'milp_objective':audited['objective'],
              'absolute_lp_milp_gap':abs(solved['objective']-audited['objective']),
              'solver_diagnostics':solved.get('diagnostics',{}), 'milp_gap':audited['gap']}
    write_json(path/'summary.json',output)
    return output


def perfect_information_bound(data, destination):
    """Valid, deliberately optimistic full-information LP lower bound.

    A free final stock and known realized net demand/price relax the online problem.
    This model buys energy at base price without adjustment or emergency penalties.
    """
    outputs = {}
    for mode in ('fixed','variable'):
        net = ((data.flat_load-data.flat_pv)/6.)[None,:]
        prices = (data.flat_price if mode=='variable' else np.tile(data.prior_price,365))[None,:]
        result = solve_schedule(net,prices,INITIAL,terminal_soc=None,allow_emergency=False,time_limit=600.)
        outputs[mode] = {'cost':float(result['objective']), 'energy':float(result['q'].sum()),
                         'initial_soc':INITIAL,'final_soc':float(result['soc'][-1]),'gap':result['gap']}
    write_json(destination,outputs)
    return outputs
```

src/microgrid/verify.py

```
"""Independent physical, settlement, archive and workbook verification."""
from pathlib import Path
import datetime as dt
import gzip
import json

import numpy as np
from openpyxl import load_workbook

from .data import load_data
from .optimize import solve_schedule

ROOT = Path(__file__).resolve().parents[2]
NAMES = {"q1": "result1.xlsx", "q2": "result2.xlsx", "q3": "result3.xlsx",
         "q4_2": "result4-2.xlsx", "q4_3": "result4-3.xlsx"}
SELECTED = ("2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21")
TOL = 1e-6


class VerificationError(RuntimeError):
    pass


def _maxabs(values):
    a = np.asarray(values, dtype=float)
    return float(np.max(np.abs(a))) if a.size else 0.0


```

```
def _period(a, b):
    return "{:02d}:{:02d}–{:02d}:{:02d}".format(a // 6, a % 6 * 10, b // 6, b % 6 * 10)


def _date(day):
    return (dt.datetime(2025, 1, 1) + dt.timedelta(days=int(day))).date()


def _asdate(value):
    if isinstance(value, dt.datetime):
        return value.date()
    if isinstance(value, dt.date):
        return value
    return dt.datetime.fromisoformat(str(value)).date()


def _events(values):
    """Independent scan; adjacent events cannot merge across calendar days."""
    answer, start = [], None
    for i, value in enumerate(np.r_[values, 0.0]):
        if value > 1e-7 and start is None:
            start = i
        elif value <= 1e-7 and start is not None:
            answer.append((_period(start, i), float(np.sum(values[start:i]))))
            start = None
    return answer


def verify_ledger(log, summary, name, expected_steps=None):
    """Verify one ledger without calling the implementation's safety/fee helpers."""
    errors = []
    n = len(log["plan"])
    def check(condition, message):
        if not condition:
            errors.append(message)
    check(expected_steps is None or n == expected_steps, "wrong number of intervals")
    for key, value in log.items():
        check(np.asarray(value).ndim == 1 and len(value) == (n + 1 if key == "soc" else n), "wrong shape: "
 + key)
        check(np.isfinite(value).all(), "nonfinite values: " + key)
    eta = float(summary.get("configuration", {}).get("eta", .9))
    q = log.get("effective", log["plan"])
    c, d = log["charge"], log["discharge"]
    u = log.get("emergency", np.zeros(n))
    if name == "q1":
        begin, end = log["soc"][:-1], log["soc"][1:]
    else:
        begin, end = log["soc_start"], log["soc_end"]
    surplus = q + log["pv"] + d + u - log["load"] - c
    waste = log.get("waste", np.maximum(surplus, 0.0))
    metrics = {
        "max_energy_residual_kwh": _maxabs(surplus - waste),
        "max_soc_equation_residual_kwh": _maxabs(end - begin - eta * c + d / eta),
        "max_soc_continuity_error_kwh": _maxabs(begin[1:] - end[:-1]),
        "max_charge_discharge_overlap_kwh": float(np.minimum(c, d).max()),
        "max_emergency_charge_overlap_kwh": float(np.minimum(c, u).max()),
        "max_power_excess_kwh": float(max(0.0, c.max() - 5000 / 6, d.max() - 5000 / 6)),
        "soc_min_kwh": float(min(begin.min(), end.min())),
        "soc_max_kwh": float(max(begin.max(), end.max())),
```

```
        "initial_soc_kwh": float(begin[0]),
    }
    for metric in ("max_energy_residual_kwh", "max_soc_equation_residual_kwh", "max_soc_continuity_error_kwh
    ", "max_charge_discharge_overlap_kwh", "max_emergency_charge_overlap_kwh", "max_power_excess_kwh"):
        check(metrics[metric] <= TOL, metric + " exceeds tolerance")
    check(abs(begin[0] - 6000) <= TOL, "initial state is not 6000 kWh")
    check(metrics["soc_min_kwh"] >= 1200 - TOL and metrics["soc_max_kwh"] <= 10800 + TOL, "SOC bound
    violated")
    for key in ("plan", "effective", "charge", "discharge", "emergency", "waste"):
        if key in log:
            check(np.min(log[key]) >= -TOL, "negative physical flow: " + key)
    if name == "q1":
        check(abs(end[-1] - begin[0]) <= TOL, "Q1 terminal SOC differs from initial")
        check(np.min(surplus) >= -TOL, "Q1 supply below demand")
        check(abs(log["price"] @ q - summary["cost"]) <= 1e-5, "Q1 cost mismatch")
        check(abs(q.sum() - summary["energy"]) <= 1e-5, "Q1 purchase total mismatch")
        gap = abs(summary["lp_objective"] - summary["milp_objective"])
        metrics["q1_lp_milp_absolute_gap_yuan"] = float(gap)
        check(gap <= 1e-4 and abs(gap - summary["absolute_lp_milp_gap"]) <= 1e-6, "Q1 LP/MILP audit mismatch
    ")
        check(summary.get("milp_gap", 1) <= 1e-7, "Q1 MILP gap is not certified")
    else:
        config = summary["configuration"]
        natural = config["settlement"] == "adjusted"
        fees = {
            "base_fee": log["price"] * log["plan"],
            "up_fee": 1.5 * log["price"] * np.maximum(q - log["plan"], 0),
            "down_fee": (-.5 if natural else .5) * log["price"] * np.maximum(log["plan"] - q, 0),
            "emergency_fee": 5 * log["price"] * u,
        }
        fees["cost"] = sum(fees.values())
        metrics["max_fee_error_yuan"] = max(_maxabs(log[key] - value) for key, value in fees.items())
        check(metrics["max_fee_error_yuan"] <= 1e-5, "fees are not a single settlement against the midnight
    plan")
        if config["settlement"] == "full_plan_plus_penalty":
            check(np.min(q - log["plan"]) >= -TOL, "full-plan-plus-penalty settlement contains a dominated
    downward commitment")
        if not config.get("updates"):
            check(_maxabs(q - log["plan"]) <= TOL, "purchase changed despite no allowed revisions")
        # Recompute safety clipping from reference targets, independently.
        if "target_charge" in log and "target_discharge" in log:
            available = q + log["pv"] - log["load"]
            ec = np.minimum.reduce([np.maximum(log["target_charge"], 0), np.maximum(available, 0), np.full(n
    , 5000 / 6), np.maximum((10800 - begin) / eta, 0)])
            ed = np.minimum.reduce([np.maximum(log["target_discharge"], 0), np.maximum(-available, 0), np.
    full(n, 5000 / 6), np.maximum(eta * (begin - 1200), 0)])
            if not config["storage"]:
                ec[:] = ed[:] = 0
            metrics["max_safe_tracking_error_kwh"] = max(_maxabs(ec - c), _maxabs(ed - d), _maxabs(u - np.
    maximum(-available - ed, 0)))
            check(metrics["max_safe_tracking_error_kwh"] <= TOL, "actual controls do not follow the
    specified safety policy")
        if "issue_index" in log:
            check(np.all(log["issue_index"] <= np.arange(n)), "a procurement revision occurs before its
    release")
        for section, a, b in (("whole_year", 0, n), ("evaluation", min(31 * 144, n), n), ("
    january_validation", min(14 * 144, n), min(31 * 144, n))):
            if b <= a:
                continue
```

```
            saved = summary[section]
            check(saved["days"] == (b - a) // 144, "wrong number of days in " + section)
            for key in ("plan", "effective", "charge", "discharge", "emergency", "waste", "base_fee", "up_fee", "down_fee", "emergency_fee", "cost"):
                check(abs(log[key][a:b].sum() - saved[key]) <= 1e-4, "summary total mismatch: " + section + "/" + key)
            check(abs(saved["initial_soc"] - begin[a]) <= TOL and abs(saved["final_soc"] - end[b - 1]) <= TOL, "summary inventory mismatch: " + section)
            count = sum(len(_events(u[t:t + 144])) for t in range(a, b, 144))
            check(saved["emergency_events"] == count, "summary event count mismatch: " + section)
    return {"passed": not errors, "intervals": n, "errors": errors, **metrics}


def verify_workbook(path, name, log):
    errors, comparisons = [], 0
    def same(actual, expected, label):
        nonlocal comparisons
        comparisons += 1
        try:
            match = np.isfinite(float(actual)) and abs(float(actual) - float(expected)) <= 1e-5
        except (TypeError, ValueError):
            match = False
        if not match:
            errors.append(label)
    def check(condition, label):
        if not condition:
            errors.append(label)
    book = load_workbook(path, read_only=True, data_only=True)
    expected_sheets = ["计划购电量", "充放电量"] if name == "q1" else ["计划购电量", "充放电量", "紧急购电量"]
    if name in ("q3", "q4_3"):
        expected_sheets.append("调整购电量")
    check(set(book.sheetnames) == set(expected_sheets), "unexpected or missing sheets")
    try:
        if name == "q1":
            rows = list(book["计划购电量"].values)
            check(len(rows) == 145, "Q1 purchase sheet must contain 144 intervals")
            for slot in range(144):
                check(rows[slot + 1][0] == _period(slot, slot + 1), "Q1 interval label")
                same(rows[slot + 1][1], log["plan"][slot], "Q1 purchase value")
            rows = list(book["充放电量"].values)
            for block in range(6):
                check(rows[block + 1][0] == _period(24 * block, 24 * (block + 1)), "Q1 storage period")
                same(rows[block + 1][1], log["charge"][block * 24:(block + 1) * 24].sum(), "Q1 charge bin")
                same(rows[block + 1][2], log["discharge"][block * 24:(block + 1) * 24].sum(), "Q1 discharge bin")
            same(rows[1][4], log["soc"][0], "Q1 initial inventory")
            same(rows[2][4], log["soc"][-1], "Q1 final inventory")
        else:
            for title, key in (("计划购电量", "plan"), ("调整购电量", "effective")):
                if title not in book.sheetnames:
                    continue
                rows = list(book[title].values)
                check(len(rows) == 335 and len(rows[0]) == 147, title + " must have 334 dates and 144 interval columns plus two totals")
                check(list(rows[0][1:145]) == [_period(s, s + 1) for s in range(144)], title + " interval labels")
                for j, day in enumerate(range(31, 365), 1):
                    row = rows[j]
```

```
                    check(_asdate(row[0]) == _date(day), title + " date order")
                    a, b = day * 144, (day + 1) * 144
                    for slot in range(144):
                        same(row[slot + 1], log[key][a + slot], title + " interval value")
                    same(row[145], log[key][a:b].sum(), title + " daily quantity")
                    cost = log["base_fee"][a:b].sum()
                    if title == "调整购电量":
                        cost += log["up_fee"][a:b].sum() + log["down_fee"][a:b].sum()
                    same(row[146], cost, title + " daily commitment fee")
            rows = list(book["充放电量"].values)[1:]
            check(len(rows) == 334 * 6, "storage sheet must contain six rows per date")
            for j, day in enumerate(range(31, 365)):
                check(_asdate(rows[6 * j][0]) == _date(day), "storage date")
                for block in range(6):
                    row = rows[6 * j + block]
                    a, b = day * 144 + block * 24, day * 144 + (block + 1) * 24
                    check(row[1] == _period(block * 24, (block + 1) * 24), "storage interval")
                    same(row[2], log["charge"][a:b].sum(), "charge bin")
                    same(row[3], log["discharge"][a:b].sum(), "discharge bin")
                same(rows[6 * j][5], log["soc_start"][day * 144], "daily initial inventory")
                same(rows[6 * j + 1][5], log["soc_end"][(day + 1) * 144 - 1], "daily final inventory")
            rows = list(book["紧急购电量"].values)[1:]
            expected = []
            for day in range(31, 365):
                day_events = _events(log["emergency"][day * 144:(day + 1) * 144]) or [("无", 0.0)]
                expected.extend([(_date(day), label, value) for label, value in day_events])
            check(len(rows) == len(expected), "emergency sheet event count")
            carried_date = None
            for row, (date, label, value) in zip(rows, expected):
                if row[0] is not None:
                    carried_date = _asdate(row[0])
                check(carried_date == date and row[1] == label, "emergency date or interval")
                same(row[2], value, "emergency event energy")
    except (KeyError, IndexError, TypeError, ValueError) as exc:
        errors.append("workbook structure: " + str(exc))
    finally:
        book.close()
    check(Path(path).stat().st_size < 20_000_000, "workbook exceeds conservative 20 MB limit")
    return {"passed": not errors, "errors": errors[:30], "error_count": len(errors), "numeric_cells_compared
     ": comparisons}


def verify_all(root=None, audit_milp=True, require_workbooks=False, strict=True):
    """Write verification.json, then fail loudly if any required check fails."""
    root = Path(root) if root is not None else ROOT
    data = load_data(root / "C题/附件")
    results, workbooks, audits, errors = {}, {}, [], []
    for name, filename in NAMES.items():
        base = root / "artifacts/runs" / name
        if not (base / "ledger.npz").exists() or not (base / "summary.json").exists():
            errors.append("missing run: " + name)
            continue
        with np.load(base / "ledger.npz") as archive:
            log = {key: archive[key].copy() for key in archive.files}
        summary = json.loads((base / "summary.json").read_text())
        result = verify_ledger(log, summary, name, 144 if name == "q1" else 365 * 144)
        results[name] = result
        errors.extend([name + ": " + message for message in result["errors"]])
        if name == "q1":
```

```
            source_load, source_pv, source_price = data.prior_load / 6, data.prior_pv / 6, data.prior_price
        else:
            source_load, source_pv = data.flat_load / 6, data.flat_pv / 6
            source_price = data.flat_price if summary["configuration"]["variable_price"] else np.tile(data.
    prior_price, 365)
        for key, expected in (("load", source_load), ("pv", source_pv), ("price", source_price)):
            if log[key].shape != expected.shape or _maxabs(log[key] - expected) > TOL:
                errors.append(name + ": source data / energy conversion mismatch: " + key)
        revisions_path = base / "revisions.json.gz"
        if name != "q1" and revisions_path.exists():
            with gzip.open(revisions_path, "rt", encoding="utf-8") as stream:
                revisions = json.load(stream)
            reconstructed = log["plan"].copy()
            reconstructed_issue = np.repeat(np.arange(365) * 144, 144)
            prior_issue = -1
            allowed_slots = {int(hour) * 6 for hour in summary["configuration"]["updates"]}
            if len(revisions) != 365 * len(allowed_slots):
                errors.append(name + ": missing or duplicate scheduled revision snapshots")
            for item in revisions:
                issue, day, slot = item["issue"], item["day"], item["slot"]
                valid = 0 <= day < 365 and issue == day * 144 + slot and issue > prior_issue and slot in
    allowed_slots and len(item["q"]) == 144 - slot and np.isfinite(item["q"]).all() and np.min(item["q"])
    >= -TOL
                if not valid:
                    errors.append(name + ": invalid revision timestamp or vector")
                    continue
                reconstructed[issue:(day + 1) * 144] = item["q"]
                reconstructed_issue[issue:(day + 1) * 144] = issue
                prior_issue = issue
            result["max_revision_archive_error_kwh"] = _maxabs(reconstructed - log["effective"])
            if result["max_revision_archive_error_kwh"] > TOL:
                errors.append(name + ": archived revisions do not reproduce final commitments")
            if not np.array_equal(reconstructed_issue, log["issue_index"]):
                errors.append(name + ": archived revision issue times differ from execution log")
        elif name != "q1":
            errors.append(name + ": missing revision archive")
        path = root / "output/support" / filename
        if path.exists():
            workbooks[name] = verify_workbook(path, name, log)
            errors.extend([filename + ": " + message for message in workbooks[name]["errors"]])
        elif require_workbooks:
            errors.append("missing workbook: " + filename)
        if audit_milp and name != "q1" and result["passed"]:
            for date in SELECTED:
                day = (_asdate(date) - dt.date(2025, 1, 1)).days
                a, b = day * 144, (day + 1) * 144
                kwargs = dict(net_s=(log["load"][a:b] - log["pv"][a:b])[None, :], price_s=log["price"][a:b][
    None, :], initial_soc=log["soc_start"][a], purchase=log["plan"][a:b], terminal_soc=log["soc_end"][b -
    1], eta=summary["configuration"]["eta"], capacity=summary["configuration"]["storage"], time_limit=60)
                try:
                    lp = solve_schedule(**kwargs)
                    mip = solve_schedule(**kwargs, milp=True)
                except (ValueError, RuntimeError) as exc:
                    errors.append(name + " " + date + ": numerical audit failed: " + str(exc))
                    continue
                gap = abs(lp["objective"] - mip["objective"])
                audits.append({"policy": name, "date": date, "lp_objective": lp["objective"], "
    milp_objective": mip["objective"], "absolute_gap_yuan": gap, "milp_relative_gap": mip["gap"], "
    interpretation": "Ex-post numerical audit with actual net load, fixed midnight purchases and matched
```

```
        terminal SOC; never used for execution decisions."})
297                 if gap > 1e-4 + 1e-7 * max(1, abs(lp["objective"])):
298                     errors.append(name + " " + date + ": LP/MILP audit gap")
299     report = {
300         "passed": not errors, "errors": errors, "runs_checked": len(results),
301         "workbooks_checked": len(workbooks), "workbooks_required": require_workbooks,
302         "max_energy_residual_kwh": max([value["max_energy_residual_kwh"] for value in results.values()] or
        [0]),
303         "max_soc_equation_residual_kwh": max([value["max_soc_equation_residual_kwh"] for value in results.
        values()] or [0]),
304         "max_soc_continuity_error_kwh": max([value["max_soc_continuity_error_kwh"] for value in results.
        values()] or [0]),
305         "tolerance_kwh": TOL, "runs": results, "workbooks": workbooks,
306         "representative_milp_audits": audits,
307     }
308     target = root / "artifacts/verification.json"
309     target.parent.mkdir(parents=True, exist_ok=True)
310     target.write_text(json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
311     if errors and strict:
312         raise VerificationError("{} validation failures; see {}".format(len(errors), target))
313     return report
```

src/microgrid/verify_experiments.py

```
1  """Audit completed experiment ledgers without running any optimization.
2
3  Run ``python -m microgrid.verify_experiments`` with ``PYTHONPATH=src``. Missing
4  experiments are reported as pending; --require-complete fails until all 33 exist.
5  """
6  from __future__ import annotations
7
8  import argparse
9  from datetime import datetime, timezone
10 import gzip
11 import hashlib
12 import json
13 from pathlib import Path
14
15 import numpy as np
16
17 from .data import load_data
18 from .verify import TOL, verify_ledger
19
20 ROOT = Path(__file__).resolve().parents[2]
21 EXPECTED = ["q2_" + e for e in
22             ("no_storage", "no_margin", "q70", "q90", "scenario_reference")]
23 EXPECTED += [p + "_" + e for p in ("q3", "q4_2", "q4_3")
24              for e in ("point", "no_storage")]
25 EXPECTED += [p + "_" + e for p in ("q3", "q4_3")
26              for e in ("issue0", "issue06", "issue012", "issue018", "k6", "k24", "h2", "h6", "roundtrip90",
       "full_plan_plus_penalty")]
27 EXPECTED += [p + "_known_price" for p in ("q4_2", "q4_3")]
28
29
30 def verify_experiments(root: str | Path = ROOT, require_complete: bool = False) -> dict:
31     root = Path(root)
32     data = load_data(root / "C题/附件")
33     results, pending, failures = {}, [], []
```

```
    maximum = lambda values: float(np.max(np.abs(values))) if np.size(values) else 0.
    for name in EXPECTED:
        folder = root / "artifacts/experiments" / name
        paths = {key: folder / value for key, value in (
            ("ledger", "ledger.npz"), ("summary", "summary.json"),
            ("revisions", "revisions.json.gz"), ("diagnostics", "forecast_diagnostics.json"))}
        if not all(path.exists() for path in paths.values()):
            pending.append(name)
            continue
        try:
            summary = json.loads(paths["summary"].read_text())
            with np.load(paths["ledger"]) as archive:
                log = {key: archive[key].copy() for key in archive.files}
            with gzip.open(paths["revisions"], "rt", encoding="utf-8") as stream:
                revisions = json.load(stream)
            diagnostics = json.loads(paths["diagnostics"].read_text())
        except (EOFError, json.JSONDecodeError):
            # The simulator writes the final diagnostics after the ledger and
            # summary. An experiment that is still closing files is not ready.
            pending.append(name)
            continue
        report = verify_ledger(log, summary, name, 365 * 144)
        errors = report["errors"]
        config = summary["configuration"]
        sources = {"load": data.flat_load / 6, "pv": data.flat_pv / 6,
                   "price": data.flat_price if config["variable_price"] else np.tile(data.prior_price, 365)}
        source_errors = {key: maximum(log[key] - value) for key, value in sources.items()}
        if max(source_errors.values()) > TOL:
            errors.append("ledger values do not match the supplied actual inputs")
        reconstructed = log["plan"].copy()
        allowed = {int(hour) * 6 for hour in config["updates"]}
        previous = -1
        for item in revisions:
            issue, day, slot = item["issue"], item["day"], item["slot"]
            if not (issue == day * 144 + slot and issue > previous and slot in allowed
                    and len(item["q"]) == 144 - slot):
                errors.append("invalid revision timestamp, order, or vector length")
                continue
            reconstructed[issue:(day + 1) * 144] = item["q"]
            previous = issue
        revision_error = maximum(reconstructed - log["effective"])
        if revision_error > TOL:
            errors.append("revision archive does not reconstruct effective commitments")
        if len(revisions) != 365 * len(allowed):
            errors.append("revision archive count does not match all permitted releases")
        inventory_error = 0.
        for label, start, stop in (("whole_year", 0, 365 * 144),
                                   ("evaluation", 31 * 144, 365 * 144),
                                   ("january_validation", 14 * 144, 31 * 144)):
            z = summary[label]
            corrected = log["cost"][start:stop].sum() + float(data.prior_price.min() / .9) * (
                log["soc_start"][start] - log["soc_end"][stop - 1])
            inventory_error = max(inventory_error, abs(corrected - z["inventory_corrected_cost"]))
            if int(np.count_nonzero(log["emergency"][start:stop] > 1e-7)) != z["emergency_intervals"]:
                errors.append(label + " emergency interval count mismatch")
            if abs(log["soc_end"][start:stop].min() - z["soc_min"]) > TOL or abs(log["soc_end"][start:stop].
     max() - z["soc_max"]) > TOL:
                errors.append(label + " inventory extrema mismatch")
        if inventory_error > 1e-5:
```

```
            errors.append("inventory-adjusted cost does not reconcile")
        for engine in diagnostics.values():
            for fit in engine.get("daily_fits", []):
                if fit["training_actual_end_exclusive"] > fit["day"] * 144:
                    errors.append("daily training audit exceeds its information boundary")
        report.update(passed=not errors, input_value_errors=source_errors,
                      max_revision_archive_error_kwh=revision_error,
                      max_inventory_cost_error_yuan=float(inventory_error),
                      revision_count=len(revisions),
                      configuration=config,
                      file_sha256={key: hashlib.sha256(path.read_bytes()).hexdigest()
                                   for key, path in paths.items()})
        results[name] = report
        failures.extend(name + ": " + error for error in errors)
    report = {
        "generated_utc": datetime.now(timezone.utc).isoformat(),
        "expected_experiments": len(EXPECTED), "verified_experiments": len(results),
        "complete": not pending, "passed": not failures and (not require_complete or not pending),
        "pending": pending, "errors": failures,
        "max_energy_residual_kwh": max((z["max_energy_residual_kwh"] for z in results.values()), default=0.),
        "max_soc_equation_residual_kwh": max((z["max_soc_equation_residual_kwh"] for z in results.values()), default=0.),
        "max_soc_continuity_error_kwh": max((z["max_soc_continuity_error_kwh"] for z in results.values()), default=0.),
        "max_fee_error_yuan": max((z["max_fee_error_yuan"] for z in results.values()), default=0.),
        "max_inventory_cost_error_yuan": max((z["max_inventory_cost_error_yuan"] for z in results.values()), default=0.),
        "max_revision_archive_error_kwh": max((z["max_revision_archive_error_kwh"] for z in results.values()), default=0.),
        "tolerance_kwh": TOL, "optimization_calls": 0, "results": results,
    }
    output = root / "artifacts/experiment_verification.json"
    output.write_text(json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
    return report


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--require-complete", action="store_true")
    args = parser.parse_args()
    report = verify_experiments(require_complete=args.require_complete)
    print(json.dumps({k: v for k, v in report.items() if k != "results"}, ensure_ascii=False, indent=2))
    if not report["passed"]:
        raise SystemExit(1)
```

附录 D：AI 工具使用与核验说明

本次 AI 辅助涵盖资料检索、数学推导讨论、代码实现、自动验证、图表生成与论文草拟，具体过程见支撑材料“AI 工具使用详情.pdf”。自动检验不能替代参赛队逐项人工核验；人工审查、修改与采纳记录由参赛队据实完成。